The AutoPhOT Photometric Pipeline: Architecture, Algorithms, and Configuration Reference

November 14, 2026

Abstract

AutoPhOT is a target-centric photometric pipeline that reduces CCD and near-infrared imaging of transients and variable sources from arbitrary instruments to calibrated light curves (Brennan and Fraser, 2022). Given a directory of heterogeneous FITS frames and a sky position, it ingests and sanitises the metadata, solves or verifies the astrometry, removes instrumental signatures and the sky background, constructs an empirical point-spread function (ePSF) from stars in each frame, performs aperture and PSF photometry at the target coordinates, calibrates the measurements against reference catalogues with fitted colour terms, optionally performs template subtraction, and reports detections, calibrated magnitudes, and empirical limiting magnitudes. This document describes the processing chain implemented in the current codebase, stage by stage, including the noise model, the quality-control gates applied at each step, and the detection and limiting-magnitude statistics, and provides a complete reference to the pipeline's configuration space (approximately 500 parameters, all defined in the single-source configuration file `autophot/data/default_input.yml`). It is intended both as a methods reference for publications that use AutoPhOT reductions and as the authoritative map of what the code does and what each parameter controls.

Contents

1 Introduction **4**

2 Architecture and execution model **4**
2.1 Entry points 4
2.2 The driver 4
2.3 The per-image orchestrator 5
2.4 Configuration model 6

3 Running the pipeline **6**
3.1 A minimal run 7
3.2 Finding and changing parameters 8
3.3 How much tuning is needed 8
3.4 A run file for the command line 8
3.5 Template subtraction 9

4 Ingestion and image metadata **9**

5 Pre-processing **9**

6 Astrometric calibration (WCS) **10**

7 Source detection and image FWHM **10**

8 Cosmic rays and defects **11**

9 Background estimation **11**

10 Aperture photometry **13**

11 Empirical PSF construction **15**
11.1 Candidate pool and vetoes . . . 15
11.2 ePSF build . . . 16
11.3 Spatially varying models . . . 16

12 PSF photometry **18**

13 Reference catalogues and photometric systems **19**

14 Photometric calibration (zero-point) **19**

15 Template subtraction **21**
15.1 Alignment cascade . . . 21
15.2 Flux matching . . . 22
15.3 Subtraction backends . . . 22
15.4 Difference-image quality . . . 23

16 Target photometry and detection **24**
16.1 Detection decision . . . 24

17 Limiting magnitudes **25**

18 Host-galaxy modelling **27**

19 Outputs and post-processing **27**

20 Error-budget summary **28**

21 Reproducibility and performance **28**

A Configuration parameter reference **28**
A.1 Parameter usage audit . . . 29
A.2 General and run-control parameters . . . 30
A.3 Pre-processing . . . 32
A.4 Photometry . . . 32
A.4.1 Detection and forced-photometry decision . . . 32
A.4.2 Centroid window of the PSF fit . . . 34
A.4.3 Aperture photometry . . . 35
A.4.4 Instrumental colour terms . . . 37
A.4.5 Local background infill (LPI) . . . 37
A.4.6 Sampling regime . . . 38
A.4.7 Field FWHM estimation . . . 38
A.4.8 ePSF construction . . . 39
A.4.9 ePSF star vetoes: cutout contamination . . . 48
A.4.10 ePSF star vetoes: ensemble shape outliers . . . 50
A.4.11 ePSF star vetoes: Fourier-domain outliers . . . 50
A.4.12 ePSF star vetoes: normalisation mask . . . 51
A.4.13 Analytic composite-Moffat PSF . . . 51
A.4.14 Spatially varying (gridded) ePSF . . . 53
A.4.15 PSF fit and fitters . . . 55

A.4.16 Diagnostics . . . 57
A.4.17 Parallelism . . . 57
A.4.18 Other . . . 58
A.5 World coordinate system . . . 58
A.6 Reference catalogues . . . 62
A.7 Photometric zero-point . . . 65
A.8 Cosmic rays and defects . . . 68
A.9 Source detection and FWHM . . . 70
A.10 Limiting magnitudes . . . 71
A.11 Host-galaxy modelling . . . 75
A.12 Image reprojection . . . 76
A.13 Template subtraction . . . 77
A.14 Background estimation . . . 100
A.15 Post-processing . . . 103

B Auxiliary YAML schemas **104**
B.1 `telescope.yml` . . . 104
B.2 `catalog.yml` . . . 104
B.3 `filters.yml` . . . 105
B.4 TNS and API credentials . . . 105

1 Introduction

AutoPhOT (the AUTOmated Photometry Of Transients pipeline; Brennan and Fraser, 2022) reduces optical and near-infrared CCD imaging of point-like transients to calibrated light curves with minimal user intervention. Its defining feature is that it is *target-centric*: the user supplies a sky position (or an object name resolvable through the Transient Name Server (Transient Name Server, n.d.) or SIMBAD (Wenger et al., 2000)) and a directory of science frames, and the pipeline returns, per image, forced or free aperture and PSF photometry at that position, calibrated against survey catalogues, together with detection diagnostics and local limiting magnitudes.

The pipeline was designed around three constraints of real survey follow-up. First, input data are heterogeneous: frames arrive from many telescopes with inconsistent header conventions, pixel scales, and filter curves, so instrument-specific behaviour is confined to a telescope database (`telescope.yml`) rather than to the code. Second, photometry must degrade gracefully: every stage carries explicit quality gates and fallbacks, and the outputs carry structured quality flags rather than silent failures. Third, the photometric error budget must be honest: pixel-variance propagation is benchmarked against measured sky-aperture scatter, and a per-image empirical noise factor rescales reported uncertainties.

Since the publication of Brennan and Fraser (2022), the pipeline has been heavily extended. The current implementation adds, among other things, a vetting-heavy empirical PSF construction with an analytic composite-Moffat fallback and optional spatially varying gridded models, a least-squares / Poisson / MCMC family of PSF fitters with adaptive forced-photometry centroid windows, MCMC or robust zero-point fits with colour terms and optional spatial and magnitude-dependent corrections, template subtraction through SFFT (Russell et al., 2022; Hu et al., 2022), HOTPANTS (Becker, 2015), or a self-contained ZOGY implementation (Zackay et al., 2016), a matched-filter detection statistic with an empirical false-alarm veto, multi-threshold injection-recovery limiting magnitudes with logistic completeness fits, and optional host-galaxy surface-photometry subtraction. This document describes that implementation. The pipeline is written in Python on Astropy (Astropy Collaboration et al., 2022), NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), Matplotlib (Hunter, 2007), and pandas (McKinney, 2010), with catalogue and archive access through `astroquery` (Ginsburg et al., 2019).

Section 2 gives the execution model and the order of processing stages, and Section 3 is a worked guide to configuring and starting a run. Sections 4–18 describe each stage and its algorithm. Section 20 summarises the error budget, Section 21 covers reproducibility and parallelism, and Appendix A tabulates every configuration parameter. Appendix B documents the auxiliary YAML databases (`telescope.yml`, `catalog.yml`, `filters.yml`).

Throughout, "the code" refers to the `autophot` Python package. Entry-point module names are given in typewriter font; the authoritative configuration file is `autophot/data/default_input.yml`, which is parsed by the driver and by `autophot-main --config-help`.

2 Architecture and execution model

2.1 Entry points

The package exposes a unified command line (`autophot`, `autophot/cli.py`) with the subcommands listed in Table 1, plus the historical console scripts `autophot-driver`, `autophot-main`, `autophot-batch`, and `autophot-inspect-telescope`. Programmatic use goes through the `AutomatedPhotometry` class in `autophot/driver.py`: `AutomatedPhotometry.load()` returns the bundled defaults as one nested dictionary that the caller edits in place, and `AutomatedPhotometry.run_photometry()` executes the run (Section 3).

2.2 The driver

`driver.py::AutomatedPhotometry.run_photometry` orchestrates a run:

Table 1: Command-line entry points.

Command	Function
`autophot driver`	Full driver over a directory of images
`autophot run`	Per-image pipeline on a single FITS file
`autophot batch`	Batch driver over many datasets
`autophot inspect-telescope`	Build/update `telescope.yml` from headers
`autophot gaia-curves`	Build a custom catalogue from Gaia XP curves
`autophot color-image`	RGB composite from per-band products
`autophot periodogram`	Lomb-Scargle period search of a light curve

1. **Discovery.** Recursively scan `fits_dir` for `.fits/.fts/.fit` files. Sub-directories are treated as datasets; a per-dataset `templates/` folder overrides the top-level one. Byte-identical inputs are deduplicated when `preprocessing.deduplicate_input_files` is set, preventing doubled light-curve epochs.
2. **Header interpretation.** Each frame's `TELESCOP` (and `INSTRUME`) is matched against `wdir/telescope.yml` (`pipeline/telescope.py::inspect_telescope` can rebuild or update it). The database supplies the header keywords for gain, read noise, saturation, airmass, epoch, exposure time, the filter keyword chain (`filter_key_0`, `filter_key_1`, ...), the instrument pixel scale, and a per-instrument header-value → canonical-band map. Frames are then grouped by telescope, instrument, and band, and restricted to `selected_filters` if requested.
3. **Target resolution.** A real `target_name` is resolved to coordinates through TNS (using the `TNS_BOT_*` credentials) with a SIMBAD fallback; alternatively `target_ra`/`target_dec` are used directly, or per-image header keys via `use_header_radec`. A real name and explicit coordinates together are a configuration error. `additional_targets` (names or RA/Dec pairs) are resolved the same way and are fitted simultaneously with the primary target in a grouped PSF fit, which handles blending self-consistently.
4. **Variable-source census.** `find_variable_sources` queries SIMBAD for variable sources (SN*, No*, etc.) across the footprint of the run. These positions seed the masked background pass and, optionally (`do_other_transients`), additional per-image photometry.
5. **Template acquisition.** For each band, a reference template is located in the dataset's `templates/` directory, or downloaded when `template_subtraction.download_templates` is set (Pan-STARRS1, Legacy Survey, or SkyMapper cutouts sized to the science footprint).
6. **Catalogue prefetch.** The reference photometric catalogue is downloaded once per (catalogue, band, footprint) combination and cached under `wdir` so that per-image workers do not repeat network queries.
7. **Dispatch.** With `nCPU`> 1 each image runs in its own subprocess (round-robin GPU pinning across `nGPU` devices), the console shows an `[i/N]` counter plus failures, and a per-image `LOG_<base>.log` records the full DEBUG stream. With `nCPU=1` the same function runs serially in-process.
8. **Aggregation.** After all images complete, per-image CSVs are merged into a light curve per band and plotted; optional post-processing (transient colour correction, RGB composite, Lomb-Scargle periodogram) runs at driver level.

2.3 The per-image orchestrator

Each science frame is processed by `pipeline/main.py::run_photometry` (`autophot-main -f frame.fits -c input.yml`). The stages execute in a fixed order; Table 2 summarises them and Figure 1 shows the data flow. A run flag, `template_subtraction.do_subtraction`, controls

whether the subtraction branch executes; when it does, photometry is repeated on the difference image and both direct-image and difference-image measurements are reported.

Table 2: Ordered per-image processing stages in `pipeline/main.py`.

#	Stage	Principal code	Config section
1	Header & metadata	`telescope.py`, `functions.py`	top level
2	Pre-processing (trim, dedup)	`main.py`	`preprocessing`
3	Distortion correction	SCAMP+SWarp	`wcs`, `alignment`
4	Source detection & FWHM	`fwhm.py::Find_FWHM`	`source_detection`
5	Background, pass 1	`background.py::BackgroundSubtractor`	`background`
6	Cosmic rays & defects	`cosmic.py::RemoveCosmicRays`	`cosmic_rays`
7	WCS solve/verify	`wcs.py::WCSSolver`	`wcs`
8	Background, masked pass	`background.py`	`background`
9	Template locate/align	`templates.py::Templates.align`	`template_subtraction`
10	Calibration image/weight	`background.py`, `main.py`	`background`
11	Reference catalogue	`catalog.py::Catalog`	`catalog`
12	Detectors on cal. image	`fwhm.py`, SExtractor	`source_detection`
13	Optimum radius & AP phot.	`aperture.py::Aperture`	`photometry`
14	Aperture correction	`aperture.py`	`photometry`
15	Catalogue↔frame match	`catalog.py`	`catalog`
16	ePSF construction	`psf.py::PSF.build`	`photometry`
17	PSF photometry (field)	`psf.py::PSF.fit`	`photometry`
18	Zero-point & colour terms	`zeropoint.py::Zeropoint`	`zeropoint`
19	Difference image	`templates.py::subtract`, `run_sfft.py`	`template_subtraction`
20	Target AP+PSF photometry	`aperture.py`, `psf.py`	`photometry`
21	Detection decision	`main.py::_decide_detection`	`photometry`
22	Calibrated magnitudes	`main.py`	`zeropoint`
23	Limiting magnitudes	`limits.py::Limits`	`limiting_magnitude`
24	Outputs & quality flags	`functions.py`, `quality_flags.py`	—

2.4 Configuration model

The single source of truth for configuration is `autophot/data/default_input.yml`. Every key carries an inline `# type -- description` comment, which is what the tables in Appendix A are generated from (by `scripts/generate_parameter_appendix.py`). Users edit the loaded dictionary in place (Section 3) or write the same structure to a YAML file for the `-c` command-line option; keys left unset fall back to their defaults at point of use. Unrecognised keys abort the run with a close-match suggestion (downgradable to a warning via `AUTOPHOT_STRICT_CONFIG=0`), contradictory settings (e.g. `forceconv: AUTO` with SFFT) produce warnings, and `--`-prefixed or documented runtime keys (`fpath`, `write_dir`, `imageFilter`, `science_fwhm`, `aperture_correction` and its error, `pixel_scale`, `gain`, `read_noise`, `saturate`, `fwhm`, `exposure_time`) are set internally per image; they exist in the schema so that every `.get()` falls back to a declared type.

CLI flags `-v`, `-q`, and `--verbose-level` override `global_verbose_level` at runtime; the per-image log file always records DEBUG. Environment overrides include `AUTOPHOT_NCPU`, `AUTOPHOT_NGPU`, and the TNS credentials, which may also live in the YAML (`wcs.TNS_BOT_*`) or in `autophot_tokens.py`.

3 Running the pipeline

This section shows how a run is configured and started in practice; the worked notebook `examples/run_autophot.ipynb` walks through the same steps on real data. Everything reduces to editing one nested dictionary – the individual keys are tabulated in Appendix A and discussed in the

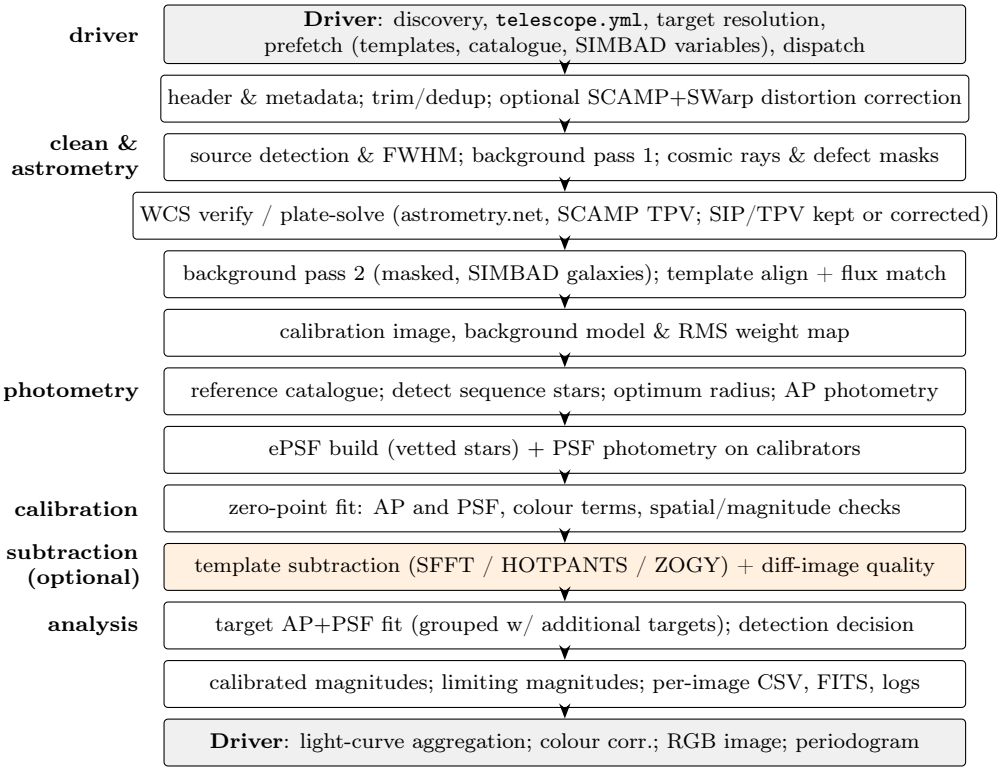


Figure 1: Per-image data flow. The subtraction branch (stage 10) is optional; without it the difference-image photometry is skipped and all results refer to the direct (sky-subtracted) frame.

stage they control (Sections 4–18), where short Python snippets also reproduce the key idea of each stage as a figure.

3.1 A minimal run

`AutomatedPhotometry.load()` returns the complete default configuration as a nested dictionary; the run is configured by editing it in place and started by passing it to `run_photometry`:

```
from autophot import AutomatedPhotometry

config = AutomatedPhotometry.load()

config["fits_dir"]    = "/data/SN2026abc/images"  # scanned recursively
config["target_name"] = "SN2026abc"   # resolved to RA/Dec via TNS/SIMBAD
# Or set the position directly (degrees, FK5/J2000); a real name
# together with explicit coordinates is a configuration error:
# config["target_ra"]  = 123.456789
# config["target_dec"] = -12.345678

config["catalog"]["use_catalog"] = "auto"  # pick the best survey per band

output_csv = AutomatedPhotometry.run_photometry(default_input=config)
```

The call returns the path of the aggregated light-curve CSV. Products land under `<fits_dir>_<outdir_name>` (default suffix `REDUCED`), one directory per image, each carrying a DEBUG-level log (`LOG_<base>.log`) and the configuration that image was processed with (`input.yaml`); `wdir` (default: the current directory) holds the telescope database and the catalogue caches. A first run needs only three further decisions: the catalogue (`catalog.use_catalog`, Section 13), whether to subtract a template (`template_subtraction.do_subtraction`, Section 15), and `nCPU` when reducing more than a few images.

3.2 Finding and changing parameters

Keys are addressed as `config["section"]["key"]`; run-level options (`fits_dir`, `target_name`, `nCPU`, ...) sit at the top level. To find a key without opening the YAML file,

```
from autophot import list_parameters
list_parameters(contains="aperture")  # matching keys and their defaults
```

or, from the shell, `autophot run --config-help` prints every accepted key with its inline description. Unrecognised or misspelled keys abort the run before processing, with a close-match suggestion (`AUTOPHOT_STRICT_CONFIG=0` demotes that to a warning). The edits most often needed:

```
config["catalog"]["use_catalog"] = "panstarrs"  # one survey, all bands
# ...or a per-band mapping; band names are case-significant (r != R):
config["catalog"]["use_catalog"] = {
    "griz": "refcat", "UBVRI": "apass", "default": "gaia",
}

config["wcs"]["redo_wcs"] = False           # trust the existing header WCS
config["zeropoint"]["fit_method"] = "odr"   # mcmc (default) | odr | ransac

config["photometry"]["use_poisson_likelihood_fitter"] = True
config["photometry"]["perform_emcee_fitting_s2n"] = 10  # MCMC below S/N 10
config["photometry"]["check_inverted_image"] = True   # fading dips on diffs
config["photometry"]["force_photometry"] = True  # pin to catalogue coords

config["limiting_magnitude"]["snr_thresholds"] = [3.0, 5.0]

config["selected_filters"] = ["g", "r"]  # restrict the run to these bands
config["nCPU"] = 8          # image-level workers; keep nCPU x the
                            # per-image n_jobs under the core count
config["restart"] = False   # reuse finished outputs (True reprocesses)
config["rng_seed"] = 42     # reproducible injection draws for limits
```

3.3 How much tuning is needed

The shipped values are the tuned defaults: for a typical run the only keys that must change are the run-defining ones – where the frames are, where to work, which target, whether to subtract. The rest of the schema is exposed for experimentation: crowded fields, saturated stars, nonstandard filters, faint hosts, and borderline detections each have a gate or threshold to match, so a problematic image can be coaxed rather than re-coded. Experimenting is expected, and the strict-key check and contradictory-setting warnings of Section 2.4 catch mistakes early. Two cautions: record the overrides behind any published result (a saved YAML file doubles as that record), and set `rng_seed` when comparing configurations so the injection draws stay fixed.

3.4 A run file for the command line

The single-image and batch entry points read the same dictionary as a YAML file, so an edited configuration can be saved once and reused:

```
import yaml
with open("my_run.yml", "w") as f:
    yaml.safe_dump(config, f, sort_keys=False)
```

```
autophot run -f image.fits -c my_run.yml             # a single frame
autophot batch --file-list files.txt -c my_run.yml --jobs 8
autophot inspect-telescope /data/images --recursive  # rebuild telescope.yml
autophot gaia-curves --ra 123.45 --dec -12.34 \
```

```
    --curve myband=myband.dat --out-csv calib.csv    # custom-band catalogue
```

The historical scripts `autophot-main`, `autophot-batch`, `autophot-inspect-telescope`, and `autophot-gaia-curves` take the same arguments. Console verbosity follows `-v`/`-q`/`--verbose-level` over the YAML value; the per-image log always records DEBUG.

3.5 Template subtraction

Subtraction is off by default; enabling it takes one flag plus one template per band. A template is picked up from `fits_dir/templates/<band>_template/` – an adjacent `.weight` FITS serves as its noise map, and a `templates/` folder inside a nested dataset overrides the top-level one – or fetched automatically:

```
ts = config["template_subtraction"]
ts["do_subtraction"]     = True
ts["download_templates"] = "panstarrs"  # "legacy" | "sdss"; omit when
                                        # supplying your own templates
ts["method"]             = "auto"       # sfft | hotpants | zogy | auto
ts["alignment_method"]   = "spalipy"    # cascade falls through on failure
```

`download_templates` stops the run rather than overwrite when user templates already exist anywhere under `fits_dir`; remove them or leave it `False`. Leave `forceconv` at its `REF` default unless the difference-image PSF has been re-derived – it is the only direction under which the science ePSF applies directly to the difference image (Section 15).

4 Ingestion and image metadata

`pipeline/telescope.py::FitsInfo` reads each header and resolves the instrument through `telescope.yml` (Appendix B): the `TELESCOP` value selects a top-level entry, `INSTRUME` selects the instrument block, and that block supplies the header keywords for gain, read noise, saturation level, airmass, exposure time, and epoch (MJD or DATE-OBS), plus the instrument pixel scale and a map from raw filter header values to canonical bands. Missing critical keywords (`TELESCOP`, `INSTRUME`, `FILTER`) reject a file when `validate_fits_headers` is on; missing important keywords (`EXPTIME`, `GAIN`, epoch) warn and, in non-interactive mode, are filled with defaults.

Canonical band naming is case-significant throughout the pipeline and the databases: lowercase `u,g,r,i,z` denote Sloan-family (AB) bands, uppercase `U,B,V,R,I` denote Johnson-Cousins (Vega) bands, and `J,H,K` the 2MASS system. Single-letter tokens are never case-folded in normalisation, `telescope.yml` matching, catalogue column mapping (`catalog.yml`), or light-curve grouping, so `I` can never resolve onto `i`. Multi-letter names (`open`, `L`, `clear`) are normalised to an open/unfiltered pseudo-band.

The stage produces the image metadata that everything downstream consumes: exposure time, gain g (e^-/ADU), read noise (e^-), saturation level (ADU), MJD, pixel scale (arcsec/pixel), and the canonical band (`imageFilter`). When a keyword is missing and a documented fallback exists (e.g. `exposure_time` on survey coadds), a warning is logged.

5 Pre-processing

Three light-touch operations run before anything else. (i) NaN-boundary trimming removes no-coverage edges from resampled inputs while protecting the target footprint (a configurable pixel buffer is kept; the WCS is refreshed and the target pixel position recomputed in place). (ii) Optional `trim_image` recrops the frame to a ($2 \times$ `trim_image`) arcmin box centred on the target, skipping the operation when the box would cover the whole frame. (iii) Byte-level deduplication of inputs was already applied at driver level (Section 2.2).

6 Astrometric calibration (WCS)

Astrometry is handled by `pipeline/wcs.py::WCSSolver`. The design is verify-first: when `wcs.verify_input_wcs` is set, the input world coordinate system (WCS; Calabretta and Greisen, 2002) is validated by matching detected sources to a Gaia DR3 cone search (Gaia Collaboration et al., 2023) under configurable tolerances (`verify_max_median_arcsec`, `verify_max_p95_arcsec`, `verify_min_matches`, `verify_match_radius_arcsec`, `verify_gmag_limit`); a passing input WCS is kept unchanged.

When a solve is required (`redo_wcs` or failed verification), the pipeline shells out to `astrometry.net`'s `solve-field` (Lang et al., 2010). The solve is guided by header hints (position from the telescope DB or header coordinates, pixel-scale bounds from `wide_scale_low_arcsec`/`wide_scale_high_arcsec`, optional parity from `solve_field_parity`), by an internal source list written as an `.xyls` file (`solve_field_use_xylist`), and by a sequence of SIP tweak orders (`solve_field_tweak_orders`, default $[4, 2, 1, 0]$) tried in turn under separate CPU budgets (`cpulimit`, `late_cpulimit`, `blind_cpulimit`, `solve_field_total_budget`). Detections used for solving come from SExtractor run with the `wcs`-section extraction parameters, with a sparse-field fallback (`solve_field_sparse_*`) that lowers the detection threshold on starved frames. A crowded-field profile (`wcs.crowded_auto` and the `wcs.crowded` sub-table) switches extraction and solving to dense-field parameters automatically.

Solutions are sanity-checked: the solved pixel scale must agree with the expected scale to `solved_scale_tolerance`, the solved centre with the hinted position to `solved_position_tolerance_arcsec`, and, when both an input and a solved WCS exist, the better of the two is kept via `compare_input_vs_solved_wcs` / `keep_input_wcs_if_solved_not_better`, evaluated by rematching detected sources against the catalogue.

Distortion is represented either as SIP polynomials (Shupe et al., 2005) in the solved header (`projection_type: SIP`) or refined as TPV by SCAMP (Bertin, 2006) when `prefer_scamp_tpv_after_solve` is set and SCAMP is available; `refit_wcs_from_points` re-fits the WCS directly from matched pixel/catalogue positions. With `correct_distortion` enabled the image itself is resampled through SWarp (Bertin et al., 2002) (`correct_distortion_resampling_type: LANCZOS3`), after which the measured FWHM is rechecked against inflation beyond `fwhm_inflation_max_factor`. The WCS cache is invalidated after every header edit (`invalidate_wcs_cache`), which the codebase treats as an invariant: headers and the WCS object may never disagree.

7 Source detection and image FWHM

Initial source detection is performed by SExtractor (Bertin and Arnouts, 1996) with the `wcs.sextractor_*` parameters, supplemented by photutils' `IRAFStarFinder`/`DAOStarFinder` (`source_detection.finder: dao`, `dao_nsigma`). `pipeline/fwhm.py::Find_FWHM.measure_image` then estimates the global seeing: a background map (`photutils.Background2D`, median estimator, biweight RMS, 3σ clipping) feeds the detection threshold, and the reported FWHM is the sigma-clipped median of SExtractor's `FWHM_IMAGE` over the vetted sources (clipping iterated `fwhm_sigma_clip_maxiters` times). Vetting removes constant-region sources (`sextractor_reject_constant_regions`), elongated objects (`source_detection.elongation_max`), and sources whose `FLUX_`/FWHM ratio exceeds `fwhm_flux_radius_max_frac` (galaxy rejection); a 2-D Gaussian fit on selected cutouts provides a cross-check bounded by `fwhm_max_sigma_px`. Linearity (`check_linearity`) fits peak versus total flux to flag the saturation knee and vet nonlinear calibrators.

The measured FWHM sets the sampling regime that calibrates many downstream heuristics: undersampled at FWHM≤ 2.5 px (`undersampled_fwhm_threshold`), critically sampled at ≤ 3 px, and oversampled above 5 px. Each regime adjusts detection parameters, aperture sizing, ePSF oversampling, convergence criteria, and candidate-cut strictness.

8 Cosmic rays and defects

When `cosmic_rays.remove_cmrays` is enabled, `pipeline/cosmic.py::RemoveCosmicRays` flags and repairs cosmic-ray hits. `cr_backend` selects the detector: `astroscrappy` (default; the fast Cython L.A.Cosmic variant of van Dokkum (2001), McCully and others, 2018), `lacosmic` (the `ccdproc` implementation), or `dfcosmic` (Dragonfly's PyTorch port of the original algorithm), with the `cr_sigclip`, `cr_sigfrac`, and `cr_objlim` detection parameters. Detected hits are interpolated over and also added to a separate, dilated defect mask (`cr_dilate_factor`, `cr_dilate_iterations`, `cr_defect_margin_px`) so that repaired pixels never influence background statistics or PSF candidates within `psf_defect_exclusion_fwhm`. The target itself is protected: detections overlapping `cr_protect_target_radius_fwhm` around the target are dropped, since a transient's PSF-like profile invites false positives (`cr_protect_target`, `cr_defect_target_*`). Streak-like artifacts are found through a Hough-transform line search (`fwhm.py::hough_transform_streak_mask`) and folded into the same defect mask; defective column segments are detected separately (`background.mask_defect_columns` family). Optionally the MaxiMask convolutional classifier (Paillassa et al., 2020) (`background.use_maximask`) labels hot pixels, diffraction spikes, satellite trails, saturation, and persistence (`maximask_classes`), which feed both the PSF-star vetoes (`psf_spike_*`) and the target defect warning (`target_defect_warning_fwhm`).

9 Background estimation

The sky model is computed by `background.py::BackgroundSubtractor.remove`. It follows the SExtractor philosophy with modern hardening: (i) a source mask is grown iteratively from segmented detections (`source_mask_*`: dilation by `source_mask_dilate_factor`, two iterations, `source_mask_max_fraction` bounds the maskable fraction of the frame), (ii) the frame is binned into meshes of a few to ten FWHM (`mesh_scale_min/max`), (iii) sigma-clipped statistics per mesh cell (median background, biweight RMS; Beers et al., 1990) are evaluated, (iv) the mesh map is median-filtered and interpolated back to full resolution with a cubic spline zoom (fixed under photutils ≥ 3), and (v) masked edges are flattened (`global_mask_edge_flatten`). The returned products are the background surface, a per-pixel RMS map, a defects mask, and a hardware-defects mask; near-zero pixels (below `zero_threshold`) are treated as invalid throughout.

The stage runs twice. The first pass (before WCS) gives the statistics needed for detection and FWHM; the second runs after astrometry with the defect and SIMBAD-galaxy masks applied (`mask_simbad_galaxies`), so that extended sources cannot imprint their ghosts on the mesh. A local background surface can additionally be fitted around the target (`local_*`) for the LPI-style infill described in Section 16. The RMS map is written beside the science frame as `<base>.weight.fits` (RMS in ADU) and, where an external weight map exists (`science_extweight`), it is reprojected onto the current grid and folded in. Because resampling correlates pixel noise, summed variances under-count the true scatter; the empirical noise factor of Section 10 compensates.

The idea in miniature is Figure 2: a toy frame with a smooth sky gradient and a handful of sources is divided into mesh cells, and inside one cell the source flux shows up only as a bright tail that the sigma clip removes.

```
"""Sky background estimation: the mesh grid and the clipped sky histogram.

A mesh cell's pixels are mostly sky; the source population only adds a
bright tail. Sigma clipping the cell histogram recovers the sky level.
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
```

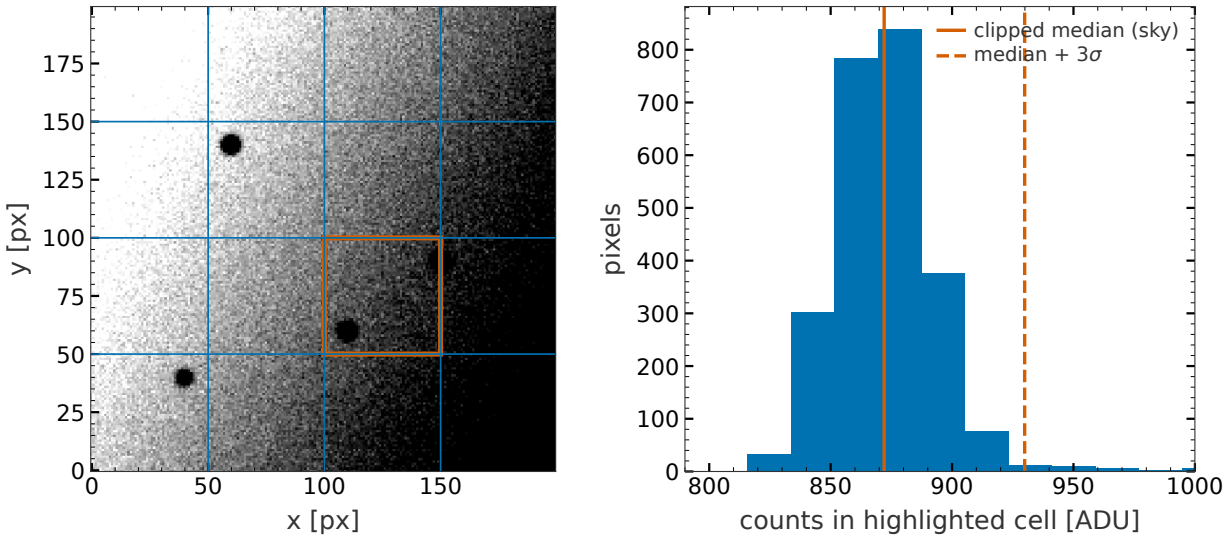


Figure 2: Left: a toy frame (smooth gradient, noise, four sources) divided into background meshes; one cell is highlighted. Right: the pixel histogram of that cell. The sky is the narrow Gaussian body; the sources contribute only the bright tail past the 3σ clip.

```
from autophot.core.plotting_utils import apply_autophot_mplstyle

rng = np.random.default_rng(42)

ny, nx = 200, 200
yy, xx = np.mgrid[:ny, :nx]
sky = 800 + 0.8 * xx - 0.4 * yy  # smooth sky gradient [ADU]
img = rng.normal(sky, 12.0)
for x0, y0, amp in [(60, 140, 900), (150, 90, 700), (110, 60, 500), (40, 40, 400)]:
    r2 = (xx - x0) ** 2 + (yy - y0) ** 2
    img += amp * np.exp(-r2 / (2 * 2.0**2))

apply_autophot_mplstyle()
plt.rcParams["pdf.fonttype"] = 42  # keep text editable in the PDF
fig, (ax_img, ax_hist) = plt.subplots(1, 2, figsize=(6.378, 2.9))

cell = 50  # mesh cell size [px]
ax_img.imshow(img, cmap="gray_r", vmin=780, vmax=900)
for k in range(1, nx // cell):
    ax_img.axvline(k * cell, color="#0072B2", lw=0.6)
    ax_img.axhline(k * cell, color="#0072B2", lw=0.6)
ax_img.add_patch(
    plt.Rectangle((100, 50), cell, cell, fill=False, edgecolor="#D55E00", lw=1.8)
)
ax_img.set(xlabel="x [px]", ylabel="y [px]")

pix = img[50 : 50 + cell, 100 : 100 + cell].ravel()  # highlighted cell
ax_hist.hist(pix, bins=40, color="#0072B2")
med = np.median(pix)
sig = 1.4826 * np.median(np.abs(pix - med))
ax_hist.axvline(med, color="#D55E00", lw=1.2, label="clipped median (sky)")
ax_hist.axvline(
    med + 3 * sig, color="#D55E00", lw=1.2, ls="--", label=r"median + 3$\sigma$"
)
ax_hist.set(
```

```
    xlabel="counts in highlighted cell [ADU]", ylabel="pixels", xlim=(790, 1000)
)
ax_hist.legend(frameon=False, loc="upper right", fontsize=7)
fig.tight_layout()

out = Path(__file__).resolve().parent.parent / "figures" / "fig_background.pdf"
fig.savefig(out)
plt.close(fig)
print(f"wrote {out}")
```

10 Aperture photometry

`photometry/aperture.py::Aperture.measure` performs circular-aperture photometry on every source. The aperture radius is the per-image `photometry.aperture_radius` when set, else the data-driven optimum radius (below), else `aperture_size` × `FWHM` (default 1.7×FWHM), floored at `min_aperture_radius_fwhm` on undersampled data. Local sky is the sigma-clipped median of an annulus separated from the aperture by `annulus_gap_fwhm` and of width `annulus_width_fwhm` (crowded fields use tighter defaults); if the annulus holds fewer than `annulus_min_pixels` usable pixels it is widened.

Flux and magnitude convention. The integrated counts in electrons within the aperture are $F_{\rm ap}$, and flux_AP $= F_{\rm ap}/t_{\rm exp}$ in $\mathrm{e^-\ s^{-1}}$; magnitudes are $m = -2.5\log_{10}(\text{flux})$. The same rate convention applies to PSF fluxes.

Noise model. The propagated variance sums the source Poisson term and the sky term over the effective (fractional-pixel) aperture area $A_{\rm eff}$, preferring photutils' `aperture_sum_err` evaluated on the error map, with an annulus-estimator term added in quadrature (Stetson, 1987):

$$\sigma_F^2 \;=\; \underbrace{F_{\rm src}}_{\text{source}} \;+\; \underbrace{A_{\rm eff}\,\max\!\left(\sigma_{\rm ann}^2,\,R^2\right)}_{\text{sky+read noise}} \;+\; \underbrace{\left(1.2533\,\frac{\sigma_{\rm ann}}{\sqrt{N_{\rm ann}}}\,A_{\rm eff}\right)^2}_{\text{annulus estimator}}, \tag{1}$$

where R is the read noise, $\sigma_{\rm ann}$ the MAD-scaled scatter of the clipped annulus, and $N_{\rm ann}$ the usable annulus pixel count. Resampling and template processing correlate pixel noise, so the propagated variance is multiplied by an empirical noise factor η (`empirical_noise_factor`): the ratio of measured empty-sky aperture-sum scatter to the propagated prediction, measured on ∼100 random sky apertures per image. It only inflates ($\eta \geq 1$); $\eta > 1.5$ additionally warns of confusion noise, and per-image η values are ledgered and summarised at driver level. Magnitude errors use the linear approximation

$$\sigma_m = \frac{2.5}{\ln 10}\,\frac{\sigma_F}{F} = \frac{1.0857}{\mathrm{S/N}}, \tag{2}$$

consistent between AP and PSF outputs.

Optimum radius. `measure_optimum_radius` selects the aperture from the curve of growth (CoG; Stetson, 1990) of isolated stars: per-source SNR-optimal radii (Howell, 1989) are combined into a sigma-clipped median after a concentration gate on the enclosed-flux fraction inside 1.7×FWHM and a tail-stability screen (`optimum_radius_*`), with a minimum kept-source count before the estimate is trusted (`optimum_radius_min_keep_*`).

Aperture correction. `compute_aperture_correction` measures the offset between the adopted aperture and the CoG total on bright isolated stars (stored as `aperture_correction` and its error). The correction is embedded in the zero-point rather than applied to individual fluxes: $\mathrm{ZP_{AP}}$ is fitted on raw `flux_AP` and therefore already contains the aperture loss, while the measured correction is stored for the AP/PSF scale cross-check of Section 14.

The mechanism is small enough to fit in a snippet (Figure 3): the net flux is the aperture sum minus the annulus sky per pixel times the aperture area, and the S/N peaks at an intermediate

radius – too small an aperture throws away signal, too large an aperture admits more sky noise than signal.

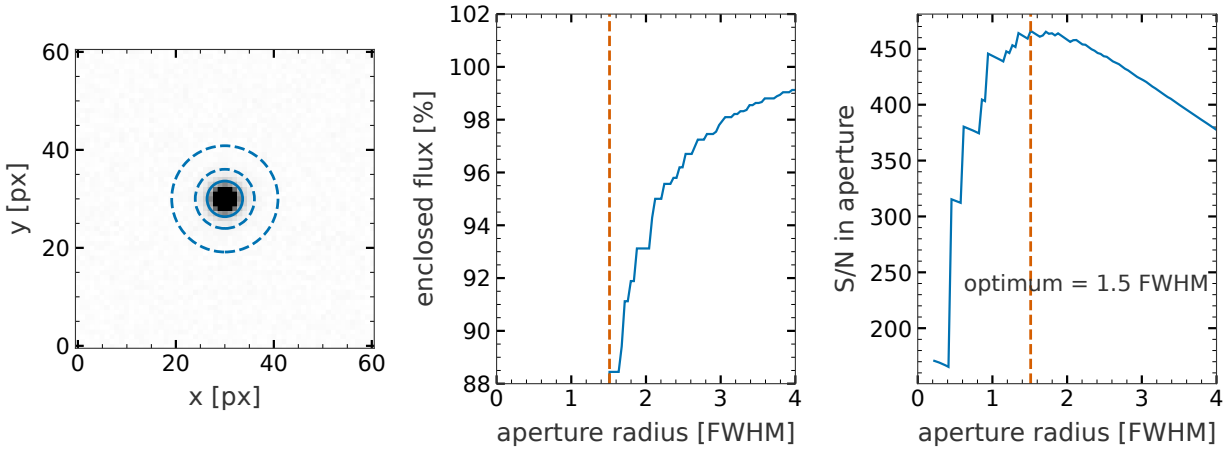


Figure 3: Left: the source aperture (solid) and the sky annulus (dashed). Centre: the curve of growth of a Moffat-profile star. Right: the aperture S/N, which peaks at ≈ 1.5 FWHM – the per-image optimum the pipeline derives from its own stars (its default is 1.7×FWHM).

```
"""Aperture photometry: source aperture, sky annulus, and the curve of
growth that sets the optimum radius.

Net flux = counts inside the aperture minus (sky per pixel x area).
S/N rises with radius until the added sky noise beats the added flux.
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

from autophot.core.plotting_utils import apply_autophot_mplstyle

rng = np.random.default_rng(7)

n = 61
c = (n - 1) / 2
yy, xx = np.mgrid[:n, :n]
r = np.hypot(xx - c, yy - c)

alpha, beta = 2.127, 2.5  # Moffat shape parameters
fwhm = 2 * alpha * np.sqrt(2 ** (1 / beta) - 1)  # = 2.4 px
psf = 30000 * (1 + (r / alpha) ** 2) ** -beta
sky_level = 800.0  # ADU per pixel
img = rng.normal(sky_level + psf, np.sqrt(sky_level + psf))

# Curve of growth and aperture S/N as a function of radius.
radii = np.linspace(0.5, 20, 200)
enclosed = np.array([psf[r <= rr].sum() for rr in radii])
frac = enclosed / enclosed[-1]
snr = enclosed / np.sqrt(enclosed + np.pi * radii**2 * (sky_level + 12.0**2))
r_opt = radii[np.argmax(snr)]
r_in, r_out = r_opt + fwhm, r_opt + 3 * fwhm  # annulus: 1 FWHM gap, 2 FWHM wide

apply_autophot_mplstyle()
plt.rcParams["pdf.fonttype"] = 42  # keep text editable in the PDF
```

```
fig, (ax0, ax1, ax2) = plt.subplots(1, 3, figsize=(6.378, 2.5))

ax0.imshow(img, cmap="gray_r", vmin=750, vmax=5000)
for rad, ls in [(r_opt, "-"), (r_in, "--"), (r_out, "--")]:
    ax0.add_patch(
        plt.Circle((c, c), rad, fill=False, edgecolor="#0072B2", ls=ls, lw=1.0)
    )
ax0.set(xlabel="x [px]", ylabel="y [px]")

ax1.plot(radii / fwhm, 100 * frac, color="#0072B2")
ax1.axvline(r_opt / fwhm, color="#D55E00", ls="--", lw=1.0)
ax1.set(
    xlabel="aperture radius [FWHM]",
    ylabel="enclosed flux [%]",
    ylim=(88, 102),
    xlim=(0, 4),
)

ax2.plot(radii / fwhm, snr, color="#0072B2")
ax2.axvline(r_opt / fwhm, color="#D55E00", ls="--", lw=1.0)
ax2.annotate(
    f"optimum = {r_opt / fwhm:.1f} FWHM",
    (3.9, snr.max() * 0.5),
    fontsize=8,
    ha="right",
)
ax2.set(xlabel="aperture radius [FWHM]", ylabel="S/N in aperture", xlim=(0, 4))

fig.tight_layout()
out = Path(__file__).resolve().parent.parent / "figures" / "fig_aperture.pdf"
fig.savefig(out)
plt.close(fig)
print(f"wrote {out}")
```

11 Empirical PSF construction

`photometry/psf.py::PSF.build` constructs the point-spread model in three tiers: a vetted empirical ePSF through photutils' `EPSFBuilder` (Bradley et al., 2022; Anderson and King, 2000), a fitted composite-Moffat analytic model (Moffat, 1969) used as build seed and as fallback, and an optional spatially varying `GriddedPSFModel`.

11.1 Candidate pool and vetoes

PSF stars are drawn from the detection catalogues and must clear an extensive battery of cuts, each with keep-floor logic so that sparse fields never end up with an empty pool (`psf_min_candidates`, `psf_pool_preferred_min`):

- *Catalogue level:* S/N bounds (`psf_snr_min/hard_min/rescue_min`, `psf_threshold_*`), saturation margin (`psf_saturate_fraction`), elongation (`psf_elongation_max`), SExtractor flags (`psf_flags_max`), isolation (`psf_isolation_radius_fwhm`, neighbour S/N floor), pairwise blend/dedup checks (`psf_min_pair_separation_fwhm`, `psf_dup_radius_px`), profile-shape cuts in FWHM, `FLUX_RADIUS`, `A_/B_IMAGE`, sharpness, and peak-to-aperture ratio, proximity to masked or defect pixels (`psf_mask_core_min_dist_fwhm`, `psf_defect_exclusion_fwhm`, `psf_spike_*`), and a defect-dominance diagnostic. A candidate failing `psf_defect_veto_min_fails` independent cuts is removed unconditionally.

- *Ensemble level:* an FFT power-spectrum outlier test (`psf_fft_*`, relaxed on undersampled data where subpixel phase jitter dominates high-frequency power), and a flux-normalised shape-outlier test against the median stack (`psf_shape_outlier_*`).
- *Cutout level* (`psf_contam_*`): quadrant-asymmetry, edge-ring and border excess, radial-profile reversal, secondary-peak detection in the core, row/column/diagonal streak medians, annulus local-maxima, total annulus flux excess, core ellipticity from second moments, a hot-pixel concentration test, and a normalised curve-of-growth test against both the clean-star ensemble and a Moffat encircled-energy envelope. Unrepairable classes (core secondary peaks, growth-curve failures, near-core masked pixels) are always rejected; when the keep-floor binds, contaminated annulus pixels are masked in place (`psf_contam_mask_ring_sigma`) and the star is kept.

11.2 ePSF build

Surviving cutouts (half-size $\geq$ `psf_cutout_size_scale_fwhm`×FWHM, enclosing `psf_cutout_ee_fraction` of the flux) are flux-normalised inside the aperture radius and passed to `EPSFBuilder`. Key behaviours:

- **Oversampling.** `psf_oversample` fixes the factor; `psf_auto_oversample_undersampled` adapts it to the sampling and star count so that each oversampled gridpoint keeps $\geq$ `psf_oversample_min_samples_per_pixel` star samples, bounded by `psf_oversample_max`.
- **Smoothing.** The Anderson & King quartic kernel (`psf_smoothing_kernel: quartic`, with a quadratic basis for $< 5 \times 5$ footprints, or `auto`/`gaussian`/`none`) is sized from the native-pixel scale `psf_smoothing_kernel_size_scale_px`, boosted when the effective samples per gridpoint fall below `psf_smoothing_kernel_target_samples`, and capped by `psf_smoothing_kernel_max_fwhm_frac`×FWHM.
- **Iteration.** Adaptive iterations (`psf_adaptive_iterations`) start at `psf_maxiters_initial` and retry to `psf_maxiters_max`; the per-star centre accuracy is relaxed on undersampled frames (`psf_center_accuracy_undersampled` ≈ 0.05 px) where the default 10^{-3} px is unreachable and continued iteration walks the ePSF core narrow. Stars that fail to settle are pruned across `psf_prune_unsettled_rounds` rounds; unsettled centroids are capped at `psf_epsf_max_star_shift_px`.
- **Seed and fallback.** A composite-Moffat model (`psf_analytic_components` core/mid/halo terms, optional shared centre offsets chosen by BIC) is fitted jointly over all star cutouts and used as the ePSF seed and residual diagnostic; when too few vetted stars reach the build (`psf_build_min_stars`) or the empirical model fails its quality gates, the analytic composite (or a canonical Moffat built from the image FWHM alone) is delivered instead and the model is flagged `analytic-composite`/`analytic-canonical` in the output.
- **Post-build gates.** The delivered ePSF's own FWHM must lie within $[0.85, 1.5]\times$ the image FWHM (`psf_epsf_fwhm_min/max_frac`), its negative-pixel fraction below `psf_epsf_negfrac_max`, and its residuals against the star ensemble below `psf_epsf_residual_ratio_max` (`psf_epsf_residual_gate`); failures fall back to the analytic model or to aperture-only photometry.

11.3 Spatially varying models

With `psf_spatial_grid` (`auto`, or an integer N), the field is split into detector grid cells (starved cells are grown into or merged with neighbours, bounded by `psf_grid_max_cells`), each cell runs the full vetting/build pipeline with $\geq$ `psf_grid_min_stars_per_cell` candidates, and the per-cell stamps are combined into a `GriddedPSFModel`. Cell stamps must pass per-cell FWHM, negative-fraction, and ensemble-consistency gates (`psf_grid_cell_*`); failed cells are filled from the nearest populated neighbour or the whole grid is rejected (`psf_spatial_fill_missing`). With `psf_grid_target_aware`, cell boundaries are chosen to place the target at the centre of its own populated

cell, and `psf_grid_require_target_cell` demands a real build there. A shared oversampling factor is picked for the whole grid and can descend when too few cells build (`psf_grid_osamp_descent`).

The reason an empirical model is needed at all is visible in Figure 4: on the detector's native pixels a star is a handful of noisy squares, but stars land at different sub-pixel phases, so stacking many of them on a finer grid recovers the smooth profile – that stacked model is the ePSF.

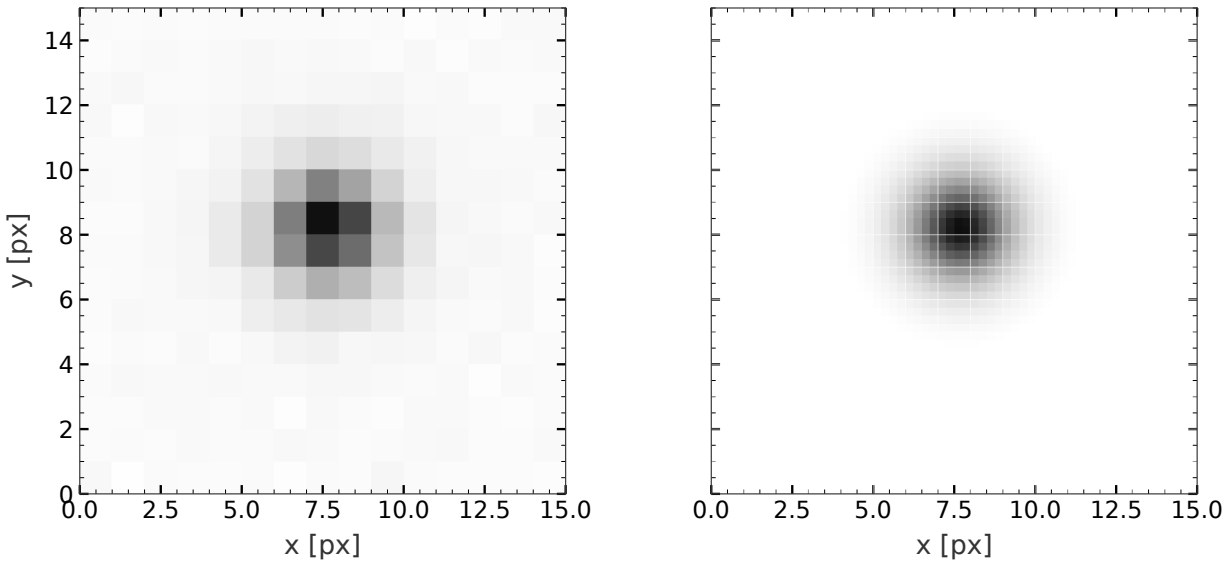


Figure 4: Left: one star on the native pixel grid. Right: the same profile on a grid four times finer (the white lines mark the native pixel boundaries) – the super-resolution the ePSF build recovers by stacking cutouts at different sub-pixel phases.

```
"""The empirical PSF: stars land at different sub-pixel phases, so
stacking many undersampled cutouts recovers a smooth oversampled model.
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

from autophot.core.plotting_utils import apply_autophot_mplstyle

rng = np.random.default_rng(3)


def moffat(yy, xx, x0, y0, alpha=2.127, beta=2.5, amp=2000):
    return amp * (1 + ((xx - x0) ** 2 + (yy - y0) ** 2) / alpha**2) ** -beta


n = 15  # native stamp size [px]
yy, xx = np.mgrid[:n, :n]
x0, y0 = n / 2 - 0.3, n / 2 + 0.2
obs = rng.normal(moffat(yy, xx, x0, y0) + 100, np.sqrt(moffat(yy, xx, x0, y0) + 100))

osamp = 4  # the ePSF grid is osamp times finer than the pixels
yyf, xxf = np.mgrid[: n * osamp, : n * osamp] / osamp - 0.375
epsf = moffat(yyf, xxf, x0, y0)

apply_autophot_mplstyle()
plt.rcParams["pdf.fonttype"] = 42  # keep text editable in the PDF
```

```
fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(6.378, 2.9), sharey=True)

ax0.imshow(obs, cmap="gray_r", vmin=60, vmax=2100, extent=(0, n, 0, n))
ax0.set(xlabel="x [px]", ylabel="y [px]")

ax1.imshow(epsf, cmap="gray_r", vmin=60, vmax=2100, extent=(0, n, 0, n))
for k in range(1, n):  # native pixel boundaries
    ax1.axvline(k, color="w", lw=0.3, alpha=0.6)
    ax1.axhline(k, color="w", lw=0.3, alpha=0.6)
ax1.set(xlabel="x [px]")

fig.tight_layout()
out = Path(__file__).resolve().parent.parent / "figures" / "fig_epsf.pdf"
fig.savefig(out)
plt.close(fig)
print(f"wrote {out}")
```

12 PSF photometry

`photometry/psf.py::PSF.fit` fits the delivered model to all sources with `photutils.PSFPhotometry`: cutouts are grouped when sources lie within `psf_group_separation_fwhm` of one another, fitted simultaneously so blends are self-consistent, and the target is always grouped with `additional_targets` for the same reason. A constant local background is estimated per group (the LPI option of Section 16 replaces it with a regression infill under the target). All sources share one fit box scaled from `psf_fit_shape_bright_scale_fwhm` with undersampled boosts, and the fitted amplitude is bounded ≥ 0 on direct frames (`psf_fit_positive_flux`); on difference images (`psf_diff_negative_flux`) signed amplitudes can be kept for fading sources.

Centroid windows make the target fit adaptive between free and forced photometry. Below `fitting_xy_bounds_detect_snr` (default S/N$=4$) the target's centroid is held within `fitting_xy_bounds_hold_px` of the catalogue seed (widened by the measured astrometric uncertainty when `fitting_xy_bounds_astrometric`), i.e. the measurement degrades gracefully to forced photometry. The in-window detection S/N is an aperture statistic measured on the image: flux inside `aperture_radius` at the peak found within the search disk, minus the median of window pixels outside that disk, over an aperture-sum noise of local `background_rms` scaled by $\sqrt{A + (1.2533\,A)^2/N_{\rm sky}}$ – the same pixel-plus-annulus budget as aperture photometry – multiplied by the empirical noise factor η to cover resampled-pixel correlation. A bright peak found beyond $\sim$0.5 FWHM of the seed is treated as a neighbour rather than the target, and its Gaussian wing contribution is subtracted from the seed aperture before the S/N is evaluated. Between that floor and `fitting_xy_bounds_full_snr` the bound grows linearly to the free window (`fitting_xy_bounds` arcsec, additionally floored and capped by `fitting_xy_bounds_target_*`). A fit that latches onto the bound edge (`fitting_xy_bounds_edge_frac`) is refit held at the seed so the reported flux cannot belong to a neighbour; `force_photometry` pins every target regardless of S/N.

Fitters. Three engines are available. The default is bounded Levenberg-Marquardt least squares. `use_poisson_likelihood_fitter` selects a Poisson maximum-likelihood fit (step-projected to honour the positive-flux bound, `poisson_*`). Above `perform_emcee_fitting_s2n` (S/N>10 by default) the LSQ solution is refined by an affine-invariant MCMC (Foreman-Mackey et al., 2013) that samples position, flux, and local sky with proper uncertainty propagation, using `emcee_nwalkers`, adaptive length (`emcee_adaptive_tau_target`, `emcee_early_stop`), burn-in fraction, thinning, and convergence diagnostics that feed the quality flags. Reported flux errors carry the fit covariance inflated by $\sqrt{\chi^2_{\rm red}}$ where appropriate, plus the SFFT kernel-model term of Section 15.

Detection statistic. Alongside photometry, the code evaluates a matched-filter probability

(Zackay et al., 2016): for the PSF-plus-sky model H_1 against the sky-only model H_0 in a box around the fitted position,

$$a = \frac{\sum_i w_i p_i d_i}{\sum_i w_i p_i^2}, \qquad z = a\sqrt{\sum_i w_i p_i^2}, \qquad \text{prob_detect} = \Phi\Big(\frac{z}{\kappa}\Big), \tag{3}$$

with $w_i = 1/\sigma_i^2$, p_i the unit-flux ePSF, and $\kappa = \text{clip}(\sqrt{\chi^2_{\text{red}}}, 1, 10)$. The statistic is signed (oversubtraction dips drive prob$\to 0$) and is suppressed when a narrower, broader, or dipole-shaped template fits the box better than the PSF. It feeds both the detection rescue of Section 16 and the empirical false-alarm veto, which evaluates the same statistic at ∼60 blank annulus sites (3–8×FWHM) around the target.

13 Reference catalogues and photometric systems

`calibration/catalog.py::Catalog` downloads the field sequence once per (catalogue, band, footprint region) via `build_complete_catalog`, querying through `astroquery` (Ginsburg et al., 2019), including VizieR (Ochsenbein et al., 2000), and serves it to every image of the run. Supported backends: `gaia` (Gaia DR3, including synthetic photometry from XP spectra via GaiaXPy (Weiler et al., 2023) and SVO filter transmission curves (Rodrigo and Solano, 2012; Gaia Collaboration et al., 2023)), `refcat` (ATLAS Refcat2; Tonry et al., 2018), `pan_starrs`/`panstarrs` (Pan-STARRS1 DR2 mean; Chambers et al., 2016; Tonry et al., 2012), `sdss` (York et al., 2000), `apass` (Henden et al., 2016), `2mass` (Skrutskie et al., 2006), `legacy` (DESI Legacy Imaging Surveys; Dey et al., 2019), `skymapper` (Wolf et al., 2018), and `custom` (user CSV; `catalog_custom_fpath`). Query geometry follows the image footprint rather than a blind cone around the target (`region_*`), with coverage-aware recentering when the catalogue does not fill the frame (`coverage_recenter_min`).

Catalogue rows are cleaned before calibration: magnitude range (`zeropoint.faint_mag_limit`, `bright_mag_limit`), saturation/nonlinearity against the image's own linearity curve (`reject_nonlinear_sources`, `nonlinear_peak_frac`, `saturation_peak_frac`), SExtractor flags (`zeropoint.max_flags`), FWHM-outlier and defect rejection, blends (`blend_neighbor_*`), star-galaxy separation on catalogue Kron magnitudes (Kron, 1980) where the backend provides them, known-variable matching, and a minimum surviving count (`min_source_no`, `clean_min_keep`).

Photometric systems. Band names are case-significant (Section 4); the catalogue column map is `autophot/data/catalog.yml` and the default colour pair per band is `filters.yml:default_dmag`. When a catalogue does not natively carry the image band, published transformations can synthesize it (`catalog.allow_filter_transformations`): SDSS$ugriz \to UBVRI$ (Lupton, 2005; Jester et al., 2005), PS1→ Johnson-Cousins $BVRI$ (Tonry et al., 2012), Legacy Survey natural-system inversions (Legacy Surveys Collaboration, 2023), and DECaLS/MzLS+BASS→SDSS piecewise relations (Toptun and et al., 2024). Each transformation's published scatter is folded into the synthesised magnitude error so transformed calibrators are weighted correctly. `include_IR_sequence_data` adds 2MASS JHK calibrators to optical fits.

14 Photometric calibration (zero-point)

`calibration/zeropoint.py::Zeropoint` fits, separately for the aperture and the PSF measurements of each matched calibrator,

$$\Delta m_i \;=\; m_{\text{cat},i} - m_{\text{inst},i} \;=\; \text{ZP} + \sum_{k=1}^{n} c_k \,(C_{1,i} - C_{2,i})^k \tag{4}$$

(polynomial orders $n = 1, 2$ via `color_term_poly_order`, or a piecewise-linear form with `color_term_n_segments`> 1 segments and a fitted break), where $C_1 - C_2$ is the default colour for the

band from `filters.yml`. Zero-points are fitted on raw instrumental magnitudes, so the aperture ZP embeds the aperture correction; a pure intercept (no colour term) is fitted when `apply_colorterms: false` or colour data are missing.

Fit methods (`zeropoint.fit_method`): `mcmc` (the default; affine-invariant sampling of the locus with errors in both coordinates, `mcmc_n_walkers/burn/steps` with an autocorrelation stop at `mcmc_tau_factor`), `odr` (orthogonal distance regression; Boggs et al., 1987), and `ransac` (robust inlier search; Fischler and Bolles, 1981). Calibrator outliers are rejected by iterative sigma clipping capped at `sigma_clip_max` seeded at a robust mode; a fast path accepts a clean fit early (`fast_path_max_mad`); a PSF-vs-AP disagreement gate (`ap_psf_disagree_min_mag`) and a minimum inlier fraction (`min_inlier_fraction`, `max_inlier_mad`) guard against systematic loci.

Extensions. An optional spatial ZP surface (`spatial_zp_surface`: low-order polynomial in detector coordinates adopted only on a significant F-test improvement and bounded by `max_correction_mag`) models vignetting/illumination residuals, and a magnitude-dependent diagnostic (`magnitude_dependent_zp`) bins the residual locus to flag nonlinear response beyond `max_zp_trend_mag`.

Zero-point error model. The reported `zeropoint_error` combines in quadrature the statistical error on the mean (formal weighted error inflated by χ^2/dof when residuals exceed claimed errors, floored at the standard error of the inlier median) and `zeropoint.systematic_err` (default 0.02 mag) for field-correlated calibration systematics; `zeropoint_scatter` reports the robust dispersion MAD $\times$ 1.4826 of the inlier locus and `zeropoint_error_stat` the pre-systematic component. Bootstrap resampling (`bootstrap_resamples`) provides the sampling distribution. When a spatial surface is used, the effective number of independent calibrators (`spatial_corr_min_neff_frac`) inflates the error.

Target magnitudes. For each photometry channel, $m_{\rm cal} = m_{\rm inst} + {\rm ZP} + c\,(C_{\rm target})$. When the target has catalogue colours (a known variable) the actual colour term applies; for a transient with unknown colour the median calibrator colour is adopted and both $|c|\,\sigma_{\rm colour}$ and $|c_{\rm err}|\,|\bar{C}|$ are added as systematics (Section 20).

At its simplest the fit is one number (Figure 5): the offset between catalogue and instrumental magnitudes over the surviving calibrators, with the handful of discrepant points clipped away first.

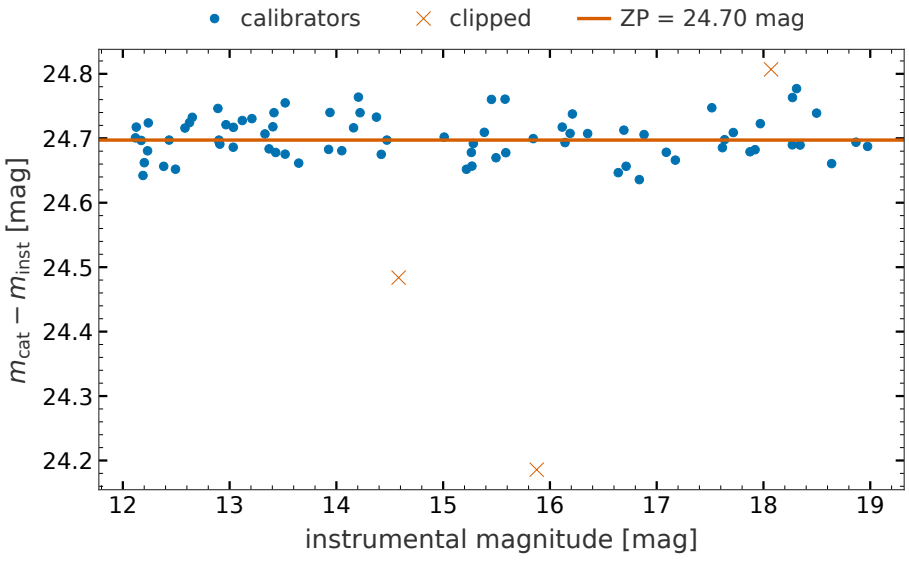


Figure 5: Catalogue minus instrumental magnitude versus instrumental magnitude for toy calibrators. The flat locus is the zero point; the marked outliers are what iterative sigma clipping removes before the offset is taken.

```
"""Photometric zero point: catalogue magnitude minus instrumental
magnitude over the matched calibrators gives the offset ZP. Sigma
```

```
clipping removes the handful of bad calibrators before the fit.
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

from autophot.core.plotting_utils import apply_autophot_mplstyle

rng = np.random.default_rng(11)
zp_true = 24.7

m_inst = rng.uniform(12, 19, 80)  # -2.5 log10 of instrumental flux
m_cat = m_inst + zp_true + rng.normal(0, 0.03, m_inst.size)
bad = rng.choice(m_inst.size, 5, replace=False)  # blends, variables, ...
m_cat[bad] += rng.normal(0, 0.35, bad.size)

dm = m_cat - m_inst
med = np.median(dm)
mad = 1.4826 * np.median(np.abs(dm - med))
keep = np.abs(dm - med) < 3 * mad
zp = np.median(dm[keep])

apply_autophot_mplstyle()
plt.rcParams["pdf.fonttype"] = 42  # keep text editable in the PDF
fig, ax = plt.subplots(figsize=(4.7, 3.0))
ax.plot(m_inst[keep], dm[keep], "o", ms=3, color="#0072B2", label="calibrators")
ax.plot(m_inst[~keep], dm[~keep], "x", ms=5, color="#D55E00", label="clipped")
ax.axhline(zp, color="#D55E00", lw=1.2, label=f"ZP = {zp:.2f} mag")
ax.set(
    xlabel="instrumental magnitude [mag]", ylabel=r"$m_{\rm cat} - m_{\rm inst}$
    [mag]"
)
ax.legend(frameon=False, ncol=3, loc="lower center", bbox_to_anchor=(0.5, 1.0))
fig.tight_layout()

out = Path(__file__).resolve().parent.parent / "figures" / "fig_zeropoint.pdf"
fig.savefig(out)
plt.close(fig)
print(f"wrote {out}")
```

15 Template subtraction

With `template_subtraction.do_subtraction: true`, `subtraction/templates.py::Templates` locates the band-matched reference (local `templates/` directory or downloaded survey cutout), aligns it, flux-matches it, and subtracts it.

15.1 Alignment cascade

`Templates.align` tries a per-method cascade; the first method to pass the quality gates wins:

Requested	Cascade order
`spalipy`	spalipy → SCAMP+SWarp → WCS reproject → AstroAlign
`swarp`	SCAMP+SWarp → reproject → AstroAlign
`reproject`	reproject → AstroAlign → SCAMP+SWarp
`astroalign`	AstroAlign → reproject → SCAMP+SWarp
`tweakwcs`	tweakwcs → SCAMP+SWarp → reproject → AstroAlign
`chi2_shift`	chi2_shift → SCAMP+SWarp → reproject → AstroAlign

spalipy performs quad-hash geometric matching and spline-warp resampling (spalipy); SCAMP+SWarp refits the template WCS against Gaia and resamples with `reproject`-family interpolation (Bertin, 2006; Bertin et al., 2002; Robitaille et al., 2020); `tweakwcs` adjusts the WCS directly from matched sources (STScI, n.d.); `chi2_shift` cross-correlates for extended fields (Guizar-Sicairos et al., 2008); AstroAlign matches source triangles (Beroiz et al., 2020). Quality gates require `alignment_min_matched_sources` matches and bound the matched offset RMS, 90th-percentile, and the local offset near the target (`alignment_max_rms_px`, `alignment_max_p90_px`, `alignment_target_*`); a post-SWarp verification pass and per-attempt metrics (`Alignment_<base>.json`) record which cascade step produced the accepted alignment.

15.2 Flux matching

A scale factor FSCALE is derived from pipeline-vetted sources common to both frames (the flux-consistent subset selected by `find_flux_consistent_sources` after S/N, morphology, defect, isolation, and stamp-level cuts, with the target region excluded). The difference image therefore inherits the science frame's photometric scale, and the ratio $f = \mathrm{FSCALE}_{\mathrm{obs}}/\mathrm{FSCALE}_{\mathrm{exp}}$ is propagated as a flux-scale systematic (`photometry.detection_*`, `sfft_fscal_*` guards).

15.3 Subtraction backends

`Templates.subtract` runs one of three backends (`template_subtraction.method`, with `auto` choosing by FWHM match):

- **SFFT** (Hu et al., 2022; Russell et al., 2022) (default, `sfft`): the matching kernel K in $\mathrm{SCI} = \mathrm{REF} \otimes K$ is solved in a delta-function (or B-spline) basis (Alard and Lupton, 1998) over stamp regions centred on the vetted kernel prior sources, via the `Easy_SparsePacket`/`Easy_CrowdedPacket` APIs in a `run_sfft.py` subprocess (numpy or CUDA backend, `sfft_backend`, `sfft_num_threads`). The convolution direction follows `forceconv`: `REF` (default; convolve the template to the science PSF, preserving science-frame photometry), `SCI`, or `AUTO` within `sfft_forceconv_auto_tol`, and deconvolution can be vetoed (`sfft_veto_deconvolution`). Kernel order (`kernel_order`), half-width sizing (`kernel_hw_*` or `sfft_auto_kernel_size`), regularisation (`sfft_regularize_*`), crowded-mode parameters (`sfft_crowded_*`, `sfft_bg_order`, `sfft_const_phot_ratio`), and prior-source vetoes (`sfft_psf_*`, `sfft_flux_linearity_*`, `sfft_max_*`) are configurable; kernel anomalies trigger a retry ladder (noise, negative fraction, spatial order) and post-subtraction checks feed back into the source list (`sfft_use_post_anomaly_feedback`).
- **HOTPANTS** (Becker, 2015): external executable (`hotpants_exe_loc`, `hotpants_timeout`), stamp selection from the same vetted sources, external noise maps via `-ini/-tni`.
- **ZOGY** (Zackay et al., 2016): a self-contained numpy implementation with per-pixel noise maps (`zogy_per_pixel_noise`, box-estimated `zogy_noise_map_box_size`), an optional least-squares kernel refinement over stamp sources (`zogy_lsq_*`), a residual astrometric-shift search and small warp correction (`zogy_astrometric_shift`, `zogy_residual_warp`), an astrometric noise floor (`zogy_astrometric_noise_floor_px`), adaptive Wiener filtering, score-image output (`zogy_write_score_images`), and NaN filling.

15.4 Difference-image quality

Post-subtraction checks include the flux-scale discrepancy (`sfft_fscal_phot_sanity_frac`, aperture-level cross-checks), static-source residual statistics, and a residual-artifact masker that locates PSF-mismatch dipoles (positive/negative lobe pairs) and masks them for subsequent photometry (`residual_artifact_*`). The difference header records `CONVD`/`FORCECON` so downstream code knows which frame was convolved (this matters for the correct PSF to fit on the difference image). The SFFT kernel's spatial model contributes a fractional flux error folded into PSF photometry (`psf_kernel_model_err_frac`), and the difference image carries Var $\approx \sigma_{\rm sci}^2 + \sigma_{\rm tpl}^2$ noise.

The end result is the familiar three-panel picture of Figure 6: once the template is aligned, flux-matched, and PSF-matched, the difference carries only what changed.

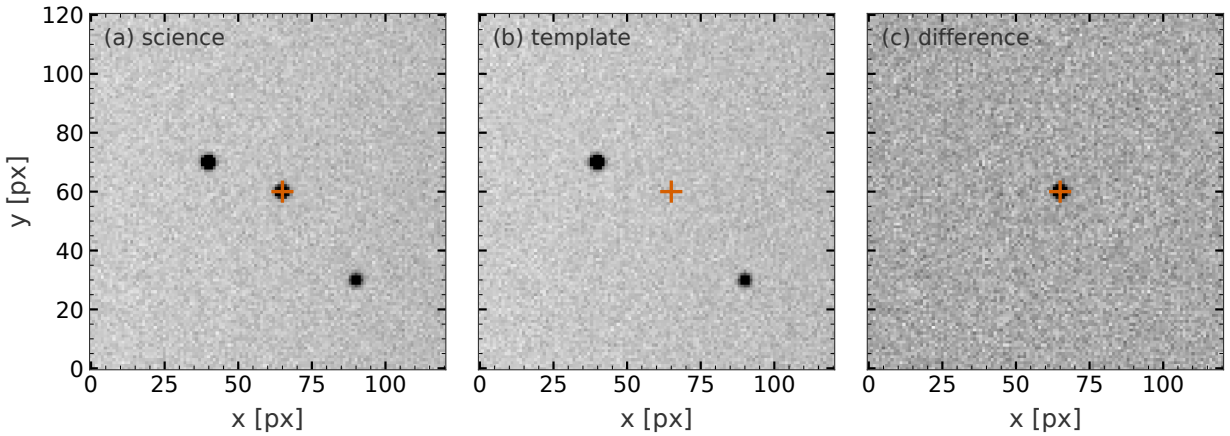


Figure 6: Toy subtraction. (a) Science frame: sky, two static sources, and a transient (marked). (b) The aligned template. (c) The difference image: sky and static sources cancel, leaving the transient's PSF – and doubled pixel noise.

```
"""Image subtraction: matching the template's flux scale and PSF to the
science frame, then subtracting, removes the sky and every static source
- only the transient survives.
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

from autophot.core.plotting_utils import apply_autophot_mplstyle

rng = np.random.default_rng(9)

n = 121
yy, xx = np.mgrid[:n, :n]


def gauss(x0, y0, amp, fwhm=3.0):
    s = fwhm / 2.355
    return amp * np.exp(-((xx - x0) ** 2 + (yy - y0) ** 2) / (2 * s**2))


sky = 500 + 0.3 * xx
template = sky + gauss(40, 70, 2000) + gauss(90, 30, 800)
science = template + gauss(65, 60, 1500)  # the transient
```

```
sci_obs = rng.normal(science, np.sqrt(science))
tpl_obs = rng.normal(template, np.sqrt(template))
diff = sci_obs - tpl_obs

apply_autophot_mplstyle()
plt.rcParams["pdf.fonttype"] = 42  # keep text editable in the PDF
fig, axs = plt.subplots(1, 3, figsize=(6.378, 2.5), sharex=True, sharey=True)
panels = [
    (sci_obs, 400, 900, "(a) science"),
    (tpl_obs, 400, 900, "(b) template"),
    (diff, -150, 300, "(c) difference"),
]
for ax, (data, lo, hi, label) in zip(axs, panels):
    ax.imshow(data, cmap="gray_r", vmin=lo, vmax=hi)
    ax.plot(65, 60, "+", ms=8, color="#D55E00", mew=1.2)
    ax.annotate(label, (0.04, 0.96), xycoords="axes fraction", va="top", fontsize=8)
    ax.set_xlabel("x [px]")
axs[0].set_ylabel("y [px]")

fig.tight_layout()
out = Path(__file__).resolve().parent.parent / "figures" / "fig_subtraction.pdf"
fig.savefig(out)
plt.close(fig)
print(f"wrote {out}")
```

16 Target photometry and detection

Aperture and PSF photometry run at the target's catalogue position on both the direct and the difference image. The target PSF fit shares the field fit's conventions (Section 12) with the adaptive centroid windows described there; `check_inverted_image` repeats the fit on the sign-flipped difference image to measure transients that faded below the template. An optional LPI-style local background infill (`lpi_background_for_target` and the `lpi_*` parameters, in development) replaces the annulus estimate under the target with a ridge-regression prediction conditioned on shifted samples of the surrounding sky, tolerating structured host-galaxy backgrounds.

16.1 Detection decision

`main.py::_decide_detection` combines independent gates; a measurement is reported as a detection only when it survives all applicable ones:

1. **S/N** $\geq$ `detection_limit` (default 3; the forced-photometry SNT convention).
2. **Positive flux** in the primary channel (catches oversubtraction dips; inverted fits reverse the sign convention).
3. **Fit quality**: reduced $\chi^2 \leq$ `detection_max_chi2` for the PSF channel.
4. **AP/PSF sign consistency** (`detection_flux_consistency`): opposite signs flag a subtraction artifact.
5. **Centroid veto**: the fitted position must lie within `detection_max_offset_arcsec` of the target (larger offsets mean the fit locked onto a neighbour or artifact).
6. **Empirical FAP veto**: `prob_detect` is evaluated at `detection_empirical_trials` blank annulus sites (3–8×FWHM) on the same image; if the fraction reaching the target's value exceeds `detection_empirical_max_pfalse` (and S/N $<$ `detection_empirical_snr_max`), the point is rejected as a correlated-noise fluctuation.

Two rescue channels counter false rejections: an aperture-SNR fallback (`detection_ap_fallback_ratio`× threshold, AP being less sensitive to PSF-model mismatch on faint sources) and a matched-filter rescue (`detection_mf_snr_min` with `prob_detect`≥`detection_mf_prob_min` and positive flux; the empirical-FAP and centroid vetoes still apply). Non-detections report an upper limit at the second entry of `limiting_magnitude.snr_thresholds` (SNU convention, default 5): the multi-threshold limiting magnitude machinery of Section 17 when available, else the 50%-completeness limit. Every epoch also carries the `quality_flags` bitmask (Section 19).

17 Limiting magnitudes

`analysis/limits.py::Limits.get_injected_limits_multi_snr` estimates local depth empirically. ePSF-shaped sources of known instrumental magnitude are injected at `injection_n_sites` random positions in an annulus around the target (`inject_min_radius_fwhm` to `inject_max_radius_fwhm`; sites avoid neighbours, bright wings, zero/variance-poor pixels, and non-finite annuli through the `inject_*` gates, and each site is repeated with subpixel jitter `injection_jitter_pix`× `injection_jitter_repetitions`). Poisson noise can be added (`inject_source_poisson_noise`). Recovery then applies the *same* photometry path and criteria as the real target (`recovery_method: auto`), so the completeness estimate inherits the pipeline's own systematics.

For each threshold in `snr_thresholds` (default $[3, 5]$), the recovery fraction as a function of magnitude is fitted with the logistic

$$P(\mathrm{det} \mid m) = \frac{1}{1 + \exp\big((m - m_{50})/s\big)}, \quad s > 0, \tag{5}$$

sampled with emcee (`logistic_emcee_*`) to give the completeness curve, the m_{50} point, and the magnitude at each S/N threshold; `completeness_solver: bisect` provides a non-MCMC alternative and `adaptive_snr_selection` chooses which thresholds to report. A bootstrap (`m50_bootstrap_n`) distributes m_{50}. Finally, an uncertainty calibration (`uncal_*`) compares the claimed S/N of recovered injections with their measured scatter and rescales the reported errors when they systematically under- or over-estimate the true noise (`uncal_apply`, clipped to `uncal_factor_clip`).

The recipe in miniature is Figure 7: injected sources fade through the detection threshold, the recovered fraction as a function of magnitude is the completeness curve, and its 50% point is the reported limit.

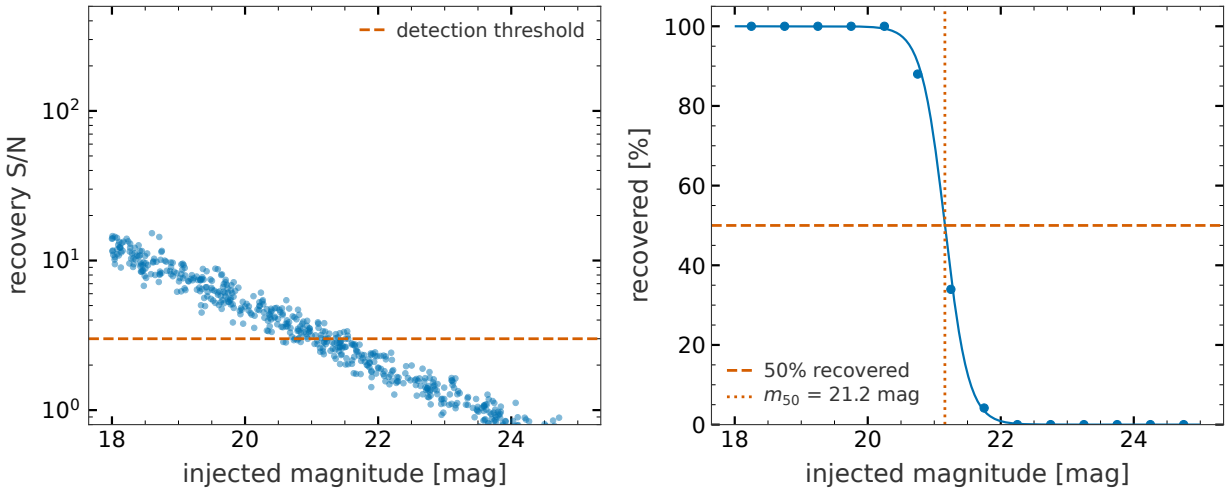


Figure 7: Left: recovery S/N of injected toy sources versus their true magnitude, with the detection threshold dashed. Right: the fraction recovered per magnitude bin and the fitted logistic completeness curve; the limit is the 50% point, m_{50}.

```
"""Injection-recovery limits: artificial sources are added at known
magnitudes, and the magnitude at which 50% are recovered is the limiting
magnitude.
"""

from itertools import pairwise
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from scipy.optimize import curve_fit

from autophot.core.plotting_utils import apply_autophot_mplstyle

rng = np.random.default_rng(5)

m_true = rng.uniform(18, 25, 600)
snr = 10 ** (0.2 * (23.5 - m_true)) * rng.lognormal(0, 0.15, m_true.size)
detected = snr > 3.0

bins = np.arange(18, 25.5, 0.5)
cen = 0.5 * (bins[1:] + bins[:-1])
frac = np.array(
    [detected[(m_true >= lo) & (m_true < hi)].mean() for lo, hi in pairwise(bins)]
)


def logistic(m, m50, k):
    return 1.0 / (1.0 + np.exp(k * (m - m50)))


(m50, k), _ = curve_fit(logistic, m_true, detected.astype(float), p0=[23, 1.5])
grid = np.linspace(18, 25, 300)

apply_autophot_mplstyle()
plt.rcParams["pdf.fonttype"] = 42  # keep text editable in the PDF
fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(6.378, 2.7))

ax0.semilogy(m_true, snr, "o", ms=2, color="#0072B2", alpha=0.5)
ax0.axhline(3.0, color="#D55E00", ls="--", lw=1.0, label="detection threshold")
ax0.set(xlabel="injected magnitude [mag]", ylabel="recovery S/N", ylim=(0.8, 500))
ax0.legend(frameon=False, fontsize=7, loc="upper right")

ax1.plot(grid, 100 * logistic(grid, m50, k), color="#0072B2")
ax1.plot(cen, 100 * frac, "o", ms=3, color="#0072B2")
ax1.axhline(50, color="#D55E00", ls="--", lw=1.0, label="50% recovered")
ax1.axvline(m50, color="#D55E00", ls=":", lw=1.0, label=f"$m_{{50}}$ = {m50:.1f}
    mag")
ax1.legend(frameon=False, fontsize=7, loc="lower left")
ax1.set(xlabel="injected magnitude [mag]", ylabel="recovered [%]", ylim=(0, 105))

fig.tight_layout()
out = Path(__file__).resolve().parent.parent / "figures" / "fig_limiting.pdf"
fig.savefig(out)
plt.close(fig)
print(f"wrote {out}")
```

18 Host-galaxy modelling

An optional host-surface-photometry stage (`host_model.enabled`) subtracts a smooth galaxy model before photometry: SExtractor seeds feed a GALFIT run (Peng et al., 2002) (`engine: galfit`, `galfit_path`) fitting `n_sersic` Sérsic components plus an optional nuclear point source within `fit_radius_fwhm`, the target position being held fixed or excluded from the model seeds (`hold_target_position`, `seed_exclude_target_fwhm`). The residual frame then carries the transient on a flattened host. This complements (and is distinct from) the LPI local infill of Section 16: GALFIT models the host globally, the LPI infill models only the sky under the target's PSF.

19 Outputs and post-processing

Per-image products land next to the science frame under `outdir_name`:

- the science FITS with updated header (WCS, `FWHM`, `CONVD`/`FORCECON`, ZP keywords), the background model and the `<base>.weight.fits` RMS map, the aligned template and the difference image;
- a per-image CSV with the target row: positions (catalogue seed and fitted centroid), `flux_ap`/`flux_AP`, `flux_psf`/`flux_PSF` and errors, S/N per channel, instrumental and calibrated magnitudes in the image band, `reduced_chi2`, `prob_detect`, `mf_snr`, empirical `p_false`, detection flag, limiting magnitudes at each threshold, the `psf_model` used, and the `quality_flags` bitmask;
- diagnostic plots (aperture curve of growth, ePSF build residuals, ZP locus, alignment vector maps, injection-recovery completeness), written as `plot_format`;
- the per-image `LOG_<base>.log` (full DEBUG) and `Alignment_<base>.json` metrics.

The `quality_flags` integer combines per-epoch conditions (Table 3); `is_fatal()` and `is_degraded()` classify unusable versus cautionary measurements for light-curve consumers.

Table 3: `quality_flags` bitmask layout (`core/quality_flags.py`).

Bit range	Flags
0–4	data: non-finite pixels, $> 20\%$ masked, edge, saturated, bad background
8–12	PSF: few stars, poor fit, spatial variation, undersampled, model mismatch
16–25	fitting: low S/N, non-convergence (LSQ/MCMC), low ESS, at bound, suspicious residuals, crowded, negative flux, MCMC fallback, degeneracy
26+	pipeline: no/few-star zero-point, poor alignment

At driver level the per-image rows are merged into a per-band `<target>_<band>.csv` light curve and plotted; an instrument column (`lightcurve_instrument_column`, with `lightcurve_instrument_aliases`) keeps mixed-facility epochs distinguishable. Optional post-processing computes an inter-band transient colour correction (`postprocessing.transient_color_correction`), an RGB composite from the per-band products (`color_image`: asinh stretch with configurable softening, percentile clipping, gamma), and a Lomb-Scargle periodogram (Lomb, 1976; Scargle, 1982) of the light curve (`periodogram_*`), in PSF or AP flavour.

The light-curve CSV is long-form (one row per image, a `filter` column). The usual next steps are one-liners:

```
from autophot.analysis.lightcurve import (
    plot_lightcurve, generate_photometry_table,
)
from autophot.photometry.differential_photometry import (
    differential_photometry,
```

```
)

plot_lightcurve(output_csv, snr_limit=3, method="PSF")  # detections + limits
generate_photometry_table(output_csv, snr_limit=3, method="PSF")  # ASCII table
differential_photometry(output_csv, method="PSF")      # vs reference ensemble
```

20 Error-budget summary

The reported magnitude error on a target epoch combines, in quadrature, the terms in Table 4. The convention $\sigma_m = 1.0857/(\mathrm{S/N})$ is uniform across aperture, PSF, and limiting-magnitude outputs, and every term is quality-gated rather than silently dropped: missing terms are absent only when the corresponding stage did not run.

Table 4: Terms folded into the reported photometric error.

Term	Origin	Configuration
source Poisson + sky + read noise	aperture/PSF fit	header `GAIN`, `RDNOISE`
annulus estimator noise	local-sky subtraction	annulus sizing
resampling/noise correlation	empirical factor η	`empirical_noise_factor`
PSF fit covariance	LSQ/MCMC posterior	fitter, χ^2 inflation
SFFT kernel model	spatial kernel systematics	`psf_kernel_model_err_frac`
zero-point	stat $\oplus$ `systematic_err`	`zeropoint.*`
colour term (transient)	median-colour assumption	`color_scatter`, slope error
aperture correction	CoG total-vs-aperture	stored `aperture_correction_err`
systematic floor	flat-fielding, model error	`systematic_error_floor_mag`
uncal rescaling	injection-derived calibration	`limiting_magnitude.uncal_*`

21 Reproducibility and performance

Determinism. With `rng_seed: null` each image derives its seed from the file content, so identical inputs reproduce bit-identical photometry; an integer forces a shared seed for every image. MCMC chains use `emcee_random_state` and `zeropoint.random_state`; the pipeline pins BLAS/OpenMP to one thread before numerical imports so that worker processes cannot oversubscribe.

Parallelism. `nCPU` controls image-level workers (subprocesses with per-image logs); within an image, `photometry.aperture_n_jobs`, `limiting_magnitude.n_jobs`, SExtractor/SCAMP `nthreads`, and `emcee_threads` multiply it, and the driver warns when the product exceeds the core count. `nGPU` pins workers round-robin to visible devices (relevant to `sfft_backend: cupy`).

Restart and caching. `restart: true` reprocesses everything; `false` reuses intermediate outputs (except `photometry.redo_sources`). `wdir` holds the telescope database, catalogue caches (`gaia_cache_ttl_days`), template products, and the empirical-noise-factor ledger. External tool failures are logged and quality-flagged rather than swallowed by broad exception handlers; every stage writes to the per-image log regardless of console verbosity.

A Configuration parameter reference

The tables below list every parameter in `autophot/data/default_input.yml`, grouped by configuration section; the large `photometry` block is split into functional sub-tables. Each entry shows the YAML key, its default and declared type, and the description carried in the default file. Keys marked *(runtime)* are populated per image by the pipeline and exist in the schema for typing; user overrides are ignored or overwritten. Keys marked *(deprecated)* are kept for backwards

compatibility but no longer affect the reduction (the replacement, where one exists, is named in the description); keys marked *(unreferenced)* have no consumer anywhere in the package and are removal candidates (Section A.1). The tables are generated by `scripts/generate_parameter_appendix.py` so they track the file exactly; `autophot-main --config-help` prints the same map at the command line.

A.1 Parameter usage audit

Every leaf key in `default_input.yml` was traced to a code reference: `scripts/audit_config_usage.py` searches the package source for each key name, and misses are checked by hand against dynamic key construction (formatted names, `startswith` iteration, whole-dict pass-through to external tools). The audit found twelve keys that were either never read or read only as compatibility shims with no effect on the result (Table 5); they have been removed from the schema and registered in `functions.py::_CONFIG_REMOVED_KEYS`, so user configs that still carry them print a "no longer used" warning rather than an unknown-key error. All 900 remaining leaf parameters are consumed directly.

Key	Finding
`wcs.code_tolerance`	never read; no solver backend consumes it
`host_model.engine`	no engine dispatch; only galfit exists
`template_subtraction.alignment_clustered_span_frac`	coherent-cluster gate hardcodes a 0.25 span
`template_subtraction.alignment_clustered_quadrant_frac`	same gate hardcodes "<2 quadrants"
`photometry.psf_adaptive_smoothing_kernel`	deprecated no-op
`background.local_interp_order`	deprecated no-op (photutils ≥ 3)
`background.global_interp_order`	deprecated no-op (photutils ≥ 3)
`photometry.upper_limit_snr`	documented but never read; the `limiting_magnitude.snr_thresholds` list supplies the SNU convention
`photometry.apply_aperture_correction`	read only to log a note; `ZP_AP` already embeds the aperture loss
`photometry.oversample_psf_fwhm`	read into a variable that was never used; `undersampled_fwhm_threshold` decides
`background.global_interpolator`	logged and ignored (photutils ≥ 3 always uses cubic zoom)
`background.local_interpolator`	logged and ignored (photutils ≥ 3 always uses cubic zoom)

Table 5: Former parameters of `default_input.yml` with no consumer in the current code; removed from the schema and listed in `_CONFIG_REMOVED_KEYS` for backwards-compatible handling of old configurations.

The two `alignment_clustered_*` keys describe thresholds that were planned for the coherent-cluster gate in `templates.py::compute_alignment_rms` (a sparse alignment whose matched anchors bunch into one corner constrains translation but not rotation or scale); the gate is implemented with fixed thresholds (0.25 of a frame axis, or all anchors in a single quadrant) so the keys are documented but inert. `host_model.engine` anticipates alternative host-fitting backends; galfit is currently the only engine, so the key is never dispatched. `wcs.code_tolerance` predates the current solver verification path.

Renamed keys from older configs are still honoured with a deprecation warning via `functions.py::_CONFIG_DEPRECATED_KEYS` (`psf_contam_asymmetry_frac`→`psf_contam_asymmetry_sigma`, `psf_contam_radial_reversal_frac`→`psf_contam_radial_reversal_sigma`, `sfft_sky_subtract`/`zogy_sky_subtract`→`sky_subtract`, `sfft_forceconv`/`zogy_forceconv`→`forceconv`, `alignment_max_p95_px`→`alignment_max_p90_px`, `skip_resampling`→`resample_mode`, `use_lacosmic`→`cr_backend`).

A.2 General and run-control parameters

Run definition and control: input and output locations, the target specification, filter selection, image-level parallelism, and console verbosity. The tail of the table holds runtime placeholders that are filled internally per image; they exist in the schema so every `.get()` has a declared type (Section 2.4) – leave them alone. A typical run edits only this table plus whichever stage tables it is tuning.

```
autophot_input["fits_dir"] = "/data/SN2026abc/images"
autophot_input["wdir"] = "/data/SN2026abc/work"
autophot_input["target_name"] = "SN2026abc"
autophot_input["selected_filters"] = ["g", "r"]
autophot_input["nCPU"] = 8
autophot_input["rng_seed"] = 42
```

Table 6: General and run-control parameters — configuration reference.

Key	Default (type)	Description
`fits_dir`	*null* (str)	Directory containing FITS images (`.fits`, `.fts`, or `.fit`). Subdirectories are scanned recursively, so nested datasets (e.g. fits_dir/dataset1/images) work; a dataset may carry its own templates folder (fits_dir/dataset1/templates) which then overrides fits_dir/templates for that dataset's images.
`skip_file_check`	`False` (bool)	If True, skip FITS header consistency checks (use with care).
`validate_fits_headers`	`False` (bool)	If True, review all FITS files for required header keywords before processing. Files missing critical keywords (TELESCOP, INSTRUME, FILTER) are rejected; files missing important keywords (EXPTIME, GAIN, MJD-OBS/DATE-OBS) are flagged with warnings and the user is asked whether to continue.
`validate_fits_headers_non_interactive`	`False` (bool)	If True, the FITS header validation stage never calls input(): missing keywords are filled with defaults and the run continues (needed for batch/Slurm jobs where stdin is closed).
`outdir_name`	`REDUCED` (str)	Output directory suffix (e.g. `REDUCED`).
`select_filter`	`False` (bool)	If True, only run on selected_filters.
`selected_filters`	`[]` (list[str]|null)	Optional list of filters to run (e.g. `["g","r"]`). When empty/null, run all available filters.
`global_verbose_level`	`1` (int|str)	Console verbosity: 0/quiet=warnings, errors, and the final result only; 1/normal=stage progress and key measurements; 2/verbose=intermediate diagnostics; 3/debug=full internal diagnostics. The per-image LOG_*.log file always records everything (DEBUG). CLI flags -v/-q/–verbose-level override this at runtime.
`target_name`	`Transient` (str)	Target name used in outputs (e.g. IAU name). A real name is resolved via TNS/SIMBAD to obtain coordinates; setting a real name together with `target_ra`/`target_dec` is a config error. The default `Transient` is a placeholder, not a lookup target.
`target_ra`	*null* (float)	Target right ascension (degrees, FK5/J2000). Mutually exclusive with a real `target_name` (see above).
`target_dec`	*null* (float)	Target declination (degrees, FK5/J2000). Mutually exclusive with a real `target_name`.
`additional_targets`	*null* (list[str]|null)	Names of additional targets to resolve via TNS (then SIMBAD fallback), e.g. ['SN2021gmj'].

continued

Key	Default (type)	Description
additional_targets_ra_dec	*null* (list[list[float]]|null)	Explicit [RA, Dec] pairs (degrees) for additional targets, e.g. [[123.456, +45.678], [...]]. Used when names are not available or to skip name resolution.
nCPU	1 (int)	Number of CPU workers used for image-level parallelism (1=serial). >1 runs each image in its own subprocess with per-image logs (LOG_<base>.log); terminal shows only a [i/N] counter plus failures. Multiply by per-image n_jobs settings to avoid oversubscribing CPUs.
nGPU	1 (int)	Number of visible GPUs to spread image-level workers across (round-robin HIP_VISIBLE_DEVICES/ROCR_VISIBLE_DEVICES/CUDA_VISIBLE_DEVICES pinning per subprocess). 0 disables pinning (children inherit the ambient device mask). Only relevant when nCPU>1; honours the AUTOPHOT_NGPU env override.
restart	True (bool)	If True, reprocess all files (ignore existing outputs where applicable).
wdir	. (str)	Working directory for scratch files and caches (catalog queries, telescope DB, intermediate products).
rng_seed	*null* (int|null)	Random seed for injection/recovery experiments. null = deterministic per-image seed derived from the file content (identical inputs reproduce bit-identical photometry); set an int to force the same seed for every image.
use_header_radec	False (bool|list[str]|str)	If True, try header RA/Dec keys (CAT-RA,CAT-DEC). If a 2-item list or comma-separated string, treat as (RA_KEY, DEC_KEY) header keys and copy those into target_ra/target_dec (degrees, FK5/J2000; units auto-detected for sexagesimal vs degrees).
do_other_transients	False (bool)	If True, perform PSF+aperture photometry on other transient-like sources (SN*, No*, etc.) from the SIMBAD variable-source query, in addition to the primary target.
save_cutout_plot	True (bool)	Save diagnostic cutout plots where supported (e.g. star-shape checks).
plot_format	png (str)	File format for all diagnostic/output plots: "png" (default) or "svg".
fpath	'' (str)	*(runtime)* Runtime: current FITS file path (set internally).
write_dir	*null* (str|null)	*(runtime)* Runtime: output directory for current image (set internally).
imageFilter	'' (str)	*(runtime)* Runtime: band name for current image after mapping (set internally).
exposure_time	30.0 (float)	Fallback exposure time (s) when the FITS header lacks EXPTIME/EXPOSURE (e.g. survey coadds, reference templates); a warning is logged when assumed.
gain	1.0 (float)	Used only for workflows without a FITS header; science frames must supply GAIN (or the telescope.yml gain keyword) in the header (see main + aperture. resolve_gain_e_per_adu).
read_noise	0.0 (float)	*(runtime)* Runtime fallback read noise (e-) if header key is missing.
pixel_scale	*null* (float|null)	*(runtime)* Runtime fallback pixel scale (arcsec/pixel) if not in telescope DB/header.
fwhm	3.0 (float)	*(runtime)* Runtime fallback seeing FWHM (pixels) if not measured/available.
fwhm_max_sigma_px	30 (float)	Maximum Gaussian sigma (pixels) allowed in 2-D FWHM fitting; prevents blow-up on very extended sources or poor cutouts.

continued

Key	Default (type)	Description
`science_fwhm`	*null* (float|null)	*(runtime)* Runtime: measured science-image FWHM (pixels) (set internally).
`scale`	`0` (float|int)	*(runtime)* Runtime: plotting/cutout scale selector (set internally; pipeline-specific).
`saturate`	`.inf` (float)	*(runtime)* Runtime fallback saturation level (ADU) if header key is missing.
`aperture_correction`	`0.0` (float)	*(runtime)* Runtime: measured aperture correction (mag) for calibration (set internally).
`aperture_correction_err`	`0.0` (float)	*(runtime)* Runtime: aperture correction uncertainty (mag) (set internally).

A.3 Pre-processing

Frame cleanup applied before any measurement: trimming edge rows and columns, and dropping duplicate exposures so the same data are not reduced twice.

```
autophot_input["preprocessing"]["trim_image"] = 5  # px off every edge
autophot_input["preprocessing"]["deduplicate_input_files"] = True
```

Table 7: Pre-processing — configuration reference.

Key	Default (type)	Description
`trim_image`	`0` (float)	If >0, trim to a (2*trim_image) arcmin box centred on the target.
`deduplicate_input_files`	`True` (bool)	If True, skip input FITS files that are byte-identical copies of another queued file (same exposure under a different filename). Prevents doubled processing and duplicate lightcurve epochs.

A.4 Photometry

The largest section: it decides which sources get measured, how the aperture radius and local sky are chosen, how the empirical PSF is built and vetted, which fitter measures the target, and what counts as a detection at all. The sub-tables below follow that order; most runs touch only the detection and aperture groups.

A.4.1 Detection and forced-photometry decision

Whether the measured target flux counts as a detection: the S/N threshold, fit-quality and sign-consistency vetoes, the empirical false-alarm annulus test, the matched-filter and aperture-fallback rescue channels, and the SNU upper limit reported for non-detections.

```
autophot_input["photometry"]["detection_limit"] = 5.0
autophot_input["photometry"]["force_photometry"] = True  # pin to catalogue coords
autophot_input["limiting_magnitude"]["snr_thresholds"] = [3.0, 5.0]  # SNU = 2nd
    entry
```

Table 8: Photometry: Detection and forced-photometry decision — configuration reference.

Key	Default (type)	Description
detection_limit	3.0 (float)	S/N threshold for declaring a forced-photometry measurement a "detection" in the output (SNT in ZTF terminology). Forced photometry is always performed at the target position; this only controls the detection flag. ZTF automated source detection typically uses 5.0 sigma, but for forced photometry at known coordinates (ZTF forced-photometry service) SNT=3 is common.
detection_max_chi2	10.0 (float)	Maximum reduced chi-squared for a PSF fit to qualify as a detection. High chi2 indicates the PSF model failed and the flux measurement is unreliable. Default 10.0 is permissive; tighten to 5.0 for stricter quality control.
detection_flux_consistency	True (bool)	If True, require PSF and aperture flux to agree in sign for a detection. Disagreement indicates a subtraction artifact rather than a real transient.
detection_ap_fallback_ratio	1.5 (float)	AP SNR fallback ratio: when PSF SNR is below threshold but AP SNR >= threshold * this ratio, flag as detection. AP is less sensitive to PSF model mismatch on faint sources.
detection_empirical_trials	60 (int)	Number of blank annulus sites used to measure the empirical false-alarm probability (prob_detect evaluated at random positions near the target on the same image). Captures correlated difference-image noise that the analytic sigma model misses. Set 0 to disable.
detection_empirical_annulus_in_fwhm	3.0 (float)	Inner radius of the blank-site annulus (in image-FWHM units) around the target.
detection_empirical_annulus_out_fwhm	8.0 (float)	Outer radius of the blank-site annulus (in image-FWHM units).
detection_empirical_min_sites	20 (int)	Minimum valid blank sites before empirical_p_false is reported (below this the estimate is too noisy to use).
detection_empirical_max_pfalse	0.10 (float)	Reject a detection when the empirical false-alarm probability exceeds this value (fraction of blank sites reaching the target's prob_detect).
detection_empirical_snr_max	10.0 (float)	Only apply the empirical-FAP veto below this SNR; above it the finite trial count cannot resolve small false-alarm rates anyway.
detection_mf_snr_min	5.0 (float)	Matched-filter rescue floor: when the photometric SNR fails the detection gate but mf_snr (the PSF-matched detection statistic, insensitive to multiplicative scale systematics like FSCAL_DISC) reaches this value with prob_detect >= detection_mf_prob_min and positive flux, flag as detection anyway. The empirical-FAP and centroid vetoes still apply to rescued detections. Set very high to disable.
detection_mf_prob_min	0.9 (float)	Minimum prob_detect required for the matched-filter rescue (see detection_mf_snr_min).
detection_max_offset_arcsec	2.0 (float)	Reject a detection whose fitted centroid lies farther than this from the target position (arcsec). The PSF fit may recentre to absorb small WCS/centroid errors, but a large offset means it locked onto an offset source or subtraction artifact rather than the target. Set 0 to disable.

`force_photometry`	`False` (bool)	If True, hold every target X/Y position at the catalog coordinate during the PSF fit (forced photometry) regardless of detection S/N. The default adaptive mode only holds targets below fitting_xy_bounds_detect_snr.

A.4.2 Centroid window of the PSF fit

How far a fitted centroid may move from its seed position, and when neighbouring sources are solved as one blended group.

```
autophot_input["photometry"]["fitting_xy_bounds"] = 4.0  # px
autophot_input["photometry"]["psf_group_separation_fwhm"] = 2.5
autophot_input["photometry"]["centroid_max_wander_fwhm"] = 0.5
```

Table 9: Photometry: Centroid window of the PSF fit — configuration reference.

Key	Default (type)	Description
`fitting_xy_bounds`	`3.0` (float)	Allowed X/Y fitting offset bound in arcseconds; converted to pixels using image pixel_scale. The effective bound is additionally capped near half the PSF fit box, so very large values require a correspondingly larger fit shape.
`fitting_xy_bounds_target_fwhm`	`1.5` (float)	Minimum X/Y fitting offset bound for TARGET fits, in FWHM units. Ensures the target PSF fit can still re-center even when fitting_xy_bounds in pixels is smaller than the FWHM (e.g. coarse pixel scales like ZTF). The larger of this FWHM-relative floor and fitting_xy_bounds is used. Clamped to [0.5, 5.0] FWHM.
`fitting_xy_bounds_target_arcsec`	`1.5` (float)	Maximum X/Y fitting offset bound for TARGET fits, in arcseconds; converted to pixels using image pixel_scale. The target seed is a catalog position, so the fit may only drift as far as astrometric error and real displacement allow - this caps the window so a marginal detection cannot hop onto a brighter neighbour. Set to null to disable and use the full fitting_xy_bounds. The cap never undercuts the fitting_xy_bounds_target_fwhm floor.
`fitting_xy_bounds_detect_snr`	`4.0` (float)	Detection S/N required before the fitted X/Y position may move within xy_bounds. The peak is searched anywhere inside the bound window, so a source displaced by bad WCS or proper motion still counts as detected; on difference images either sign counts. Target sources are always window-checked; catalog sources skip the window check when their aperture S/N already clears this threshold. At or below 0 the check is disabled and every source may re-center.
`fitting_xy_bounds_full_snr`	`20.0` (float)	Detection S/N at which a single target is allowed the full free-fit bound. Between fitting_xy_bounds_detect_snr and this value the target bound grows linearly from the held bound to the free bound, so marginal detections get only a small recentering allowance.
`fitting_xy_bounds_hold_px`	`0.5` (float)	X/Y bound in pixels applied when the detection S/N falls below fitting_xy_bounds_detect_snr; the fit then becomes forced photometry pinned near the seed position instead of chasing a noise peak. Must be > 0 (photutils).

continued

Key	Default (type)	Description
`fitting_xy_bounds_astrometric`	`True` (bool)	If True, widen the held (forced-photometry) X/Y bound by the measured astrometric uncertainty: the residual dx/dy scatter from the WCS-vs-PSF and science-vs-template offset diagnostics plus the median matched-source position error, capped at the free-fit bound.
`fitting_xy_bounds_edge_frac`	`0.9` (float)	Single-target fits whose fitted centre lands at or beyond this fraction of the per-source X/Y bound are treated as edge-latched onto a source outside the window (the bound clamped the centroid mid-approach); the target is then refit held at the catalog position so the reported flux belongs to the target rather than the neighbour. Range (0,1]; 0 disables the guard.
`psf_group_separation_fwhm`	*null* (float or null)	Maximum source separation, in image FWHM units, at which targets are PSF-fitted as one blended group. Null uses 2.0 FWHM for normally sampled images and 3.0 FWHM for undersampled data; larger separations are fitted independently.
`centroid_max_wander_fwhm`	`0.5` (float)	Maximum drift (in FWHM units, floored at 2.5 px) accepted for a re-centroid during template-subtraction source matching. A centroid landing further from its seed position latched onto a neighbour or a mask edge rather than the source; it is treated as a failed fit (NaN) so the source is kept at its SExtractor position instead of being rejected as a centroid mismatch or moved off-source.

A.4.3 Aperture photometry

The radius (a fixed FWHM multiple, explicit pixels, or the per-image curve-of-growth optimum), the sky annulus, and the noise bookkeeping – including the empirical factor that rescales the propagated variance against measured sky-aperture scatter.

```
autophot_input["photometry"]["aperture_size"] = 1.7  # radius in FWHM
autophot_input["photometry"]["find_optimum_radius"] = True
autophot_input["photometry"]["annulus_min_pixels"] = 20
autophot_input["photometry"]["empirical_noise_factor"] = True
```

Table 10: Photometry: Aperture photometry — configuration reference.

Key	Default (type)	Description
`aperture_size`	`1.7` (float)	Aperture radius in units of FWHM (pixels = `aperture_size * fwhm`). Used directly when `find_optimum_radius` is False or the optimum-radius search cannot run or fails; in crowded fields `crowded_optimum_radius_fwhm` still takes precedence. Ignored when `aperture_radius` is set.
`enforce_nonnegative_local_background`	`False` (bool)	If True, clamp annulus-estimated local background to >=0 before subtraction (prevents flux inflation when local sky is negative from over-subtraction/ miscalibration).
`plot_aperture_correction`	`False` (bool)	If True, save the aperture-correction diagnostic plot (curve of growth).

continued

Key	Default (type)	Description
`aperture_radius`	*null* (float|null)	Explicit aperture radius (pixels). When set, the data-driven optimum-radius search is skipped entirely and this value is used as-is. If null, the radius is the optimum-radius result when `find_optimum_radius` succeeds, else `aperture_size * fwhm`.
`find_optimum_radius`	`True` (bool)	If True, derive the aperture radius from a curve-of-growth optimisation over isolated stars. If False, use the fixed `aperture_size` FWHM multiplier instead (or `aperture_radius` / `crowded_optimum_radius_fwhm` when those apply).
`crowded_field`	`False` (bool)	Photometry crowded-field toggle (affects source selection/cleaning heuristics where implemented).
`min_aperture_radius_fwhm`	`0.75` (float)	Minimum aperture radius as a fraction of FWHM. On undersampled data (FWHM ~2 px) a sub-FWHM aperture discards much of the PSF core and biases fluxes/colors; this floor guarantees the aperture covers a meaningful core fraction. Set 0 to disable.
`annulus_min_pixels`	`20` (int)	Minimum number of pixels required in the local-background annulus. On undersampled data the annulus is naturally small; if it holds too few pixels the background median is noisy/biased and the annulus is widened outward. Set 0 to disable.
`empirical_noise_factor`	`True` (bool)	If True, measure the ratio of empty-sky aperture-sum scatter to propagated pixel-variance prediction and inflate aperture and PSF flux/mag errors by it (resampling correlates pixel noise, so summed variance under-counts the real noise; a value >1.5 also warns about confusion noise). Only inflates errors, never shrinks them.
`systematic_error_floor_mag`	`0.0` (float)	Systematic error floor (mag) added in quadrature to all calibrated magnitude errors. Covers flat-fielding (0.5-2%), PSF model error (0.01-0.05 mag), and other untracked systematics. 0.0 = disabled. Typical value: 0.01-0.03 mag.
`aperture_n_jobs`	`1` (int)	Worker processes for aperture photometry loops (per-image). 1 = serial; >1 can speed up large source lists.
`optimum_radius_min_keep_abs`	`8` (int)	Minimum number of stars to retain in optimum-radius source selection (adaptive rescue if strict filters over-prune).
`optimum_radius_min_keep_frac`	`0.25` (float)	Minimum retained fraction (of prefiltered stars) in optimum-radius selection.
`optimum_radius_use_moffat_cog`	`False` (bool)	If True, derive per-source optimum radius from analytic Moffat encircled-energy inversion instead of empirical CoG interpolation.
`optimum_radius_core_radius_fwhm`	`1.7` (float)	Core concentration radius (in FWHM) used to reject very broad stars.
`optimum_radius_core_flux_min`	`0.50` (float)	Minimum enclosed normalized flux required within optimum_radius_core_radius_fwhm.
`optimum_radius_outer_start_fwhm`	`2.2` (float)	Radius (in FWHM) beyond which tail flatness/ negligibility is evaluated.
`optimum_radius_outer_std_max`	`0.05` (float)	Maximum allowed standard deviation of normalized CoG flux in outer tail.
`optimum_radius_outer_slope_abs_max`	`0.03` (float)	Maximum allowed absolute mean slope of normalized CoG in outer tail.
`optimum_radius_undersampled_tail_relax`	`2.5` (float)	On undersampled images (FWHM <= undersampled_fwhm_threshold), multiply the outer-tail std/ slope tolerances by this factor and add 0.5*(factor-1) to the tail-excess limit; the normalised CoG is derived from few-pixel radii so outer-tail statistics are intrinsically noisier.

continued

Key	Default (type)	Description
`optimum_radius_max_sources`	`250` (int)	Cap on sources fed to the optimum-radius CoG search (the result is a sigma-clipped median, so extra stars add runtime not information). When the pool exceeds this, keep the highest-SNR stars per grid cell so the median still samples the whole field.

A.4.4 Instrumental colour terms

An optional colour term between the instrumental and catalogue magnitudes, fitted as a polynomial or a piecewise line.

```
autophot_input["photometry"]["apply_colorterms"] = True
autophot_input["photometry"]["color_term_poly_order"] = 2  # 1 = linear
```

Table 11: Photometry: Instrumental colour terms — configuration reference.

Key	Default (type)	Description
`apply_colorterms`	`False` (bool)	If True, measure and apply color term corrections to catalog sources during calibration.
`color_term_poly_order`	`1` (int)	Polynomial order for color term fitting: 1 for linear, 2 for quadratic.
`color_term_n_segments`	`1` (int)	Number of segments for piecewise linear fitting: 1 for single line, 2 for two segments with breakpoint, etc. Overrides poly_order if > 1.

A.4.5 Local background infill (LPI)

(In development) An LPI-style local background infill behind the target, for structured backgrounds that defeat the global mesh.

```
autophot_input["photometry"]["lpi_background_for_target"] = True
autophot_input["photometry"]["lpi_n_samples"] = 250
```

Table 12: Photometry: Local background infill (LPI) — configuration reference.

Key	Default (type)	Description
`lpi_background_for_target`	`False` (bool)	(In development) If True, apply an LPI-style local background infill under the transient (tolerates structured background). Paper: `https://iopscience.iop.org/article/10.3847/1538-4357/ac6875`
`lpi_inner_radius_scale_fwhm`	`1.5` (float)	(In development) Hidden-region radius (pixels) = scale * FWHM for LPI-style infill (region treated as "behind the PSF core").
`lpi_outer_radius_scale_fwhm`	`4.5` (float)	(In development) Good-pixel outer radius (pixels) = scale * FWHM for LPI-style infill conditioning region.
`lpi_stamp_half_size_scale_fwhm`	`6.0` (float)	(In development) Stamp half-size (pixels) = scale * FWHM used for LPI-style infill.
`lpi_n_samples`	`250` (int)	(In development) Number of shifted local samples for LPI-style infill regression.
`lpi_sample_window_px`	`30` (int)	(In development) Random shift window (+/- pixels) for drawing local samples around the transient.

continued

Key	Default (type)	Description
`lpi_min_shift_px`	*null* (float|null)	(In development) Minimum shift (pixels) for LPI training samples to avoid learning/subtracting the transient itself. If null, defaults to outer_radius.
`lpi_ridge_lambda`	`0.01` (float)	(In development) Ridge regularization strength for LPI-style infill regression.
`lpi_save_diagnostic_plot`	`True` (bool)	(In development) If True and lpi_background_for_target=True, save an LPI diagnostic plot next to per-image outputs.

A.4.6 Sampling regime

The FWHM thresholds that classify each frame as undersampled, critically sampled, or oversampled; detection, aperture, and PSF behaviour adapt to the regime.

```
autophot_input["photometry"]["undersampled_fwhm_threshold"] = 2.5  # px
autophot_input["photometry"]["oversampled_fwhm_threshold"] = 5.0  # px
```

Table 13: Photometry: Sampling regime — configuration reference.

Key	Default (type)	Description
`undersampled_fwhm_threshold`	`2.5` (float)	Treat FWHM <= this value (pixels) as undersampled for PSF build/fit behavior.
`critical_fwhm_threshold`	`3.0` (float)	FWHM <= this (and > undersampled threshold) is critically sampled. Affects detection params and aperture sizing.
`oversampled_fwhm_threshold`	`5.0` (float)	FWHM > this value is oversampled (SNR inefficient); triggers a warning and larger apertures.

A.4.7 Field FWHM estimation

Robustness guards on the per-image seeing measurement itself: clipping iterations, how far the fitted FWHM may inflate past the robust estimate, and which sources may vote.

```
autophot_input["photometry"]["fwhm_inflation_max_factor"] = 1.5
autophot_input["photometry"]["fwhm_sigma_clip_maxiters"] = 15
```

Table 14: Photometry: Field FWHM estimation — configuration reference.

Key	Default (type)	Description
`fwhm_sigma_clip_maxiters`	`15` (int)	Higher = more stable FWHM on noisy outlier stars; slower is negligible.
`fwhm_inflation_max_factor`	`1.5` (float)	Maximum acceptable FWHM inflation ratio (post-alignment / pre-alignment). Beyond this, initial FWHM is preserved to prevent kernel sizing errors from galaxy contamination. SWarp LANCZOS3 broadens ~30-50%; 1.5x is generous.
`fwhm_flux_radius_max_frac`	`1.3` (float)	Maximum FLUX_RADIUS/FWHM ratio for sources used in the image-FWHM estimate. A point source's half-light radius is ~0.5-0.7x its FWHM; the generous bound removes only obvious galaxies and extended defects that would pull the median high. Sources with missing/NaN values are kept. Set 0 to disable.

A.4.8 ePSF construction

Everything about building the empirical PSF: the candidate pool and its vetoes, cutout size, recentering, the oversampling-factor pick, smoothing kernels, the iterative build loop, and the linearity/saturation screens applied to the pool.

```
autophot_input["photometry"]["oversample_psf"] = True  # force adaptive oversampling
autophot_input["photometry"]["sextractor_reject_constant_regions"] = True
```

Table 15: Photometry: ePSF construction — configuration reference.

Key	Default (type)	Description
`psf_oversample`	`1` (int)	Oversampling factor for PSF modelling/fitting grids.
`oversample_psf`	`False` (bool)	If True, enable adaptive PSF oversampling based on FWHM threshold. If False, use fixed psf_oversample value.
`psf_smoothing_kernel`	`quartic` (str)	ePSFBuilder smoothing kernel type: "quartic" (recommended – the Anderson & King 2000 polynomial kernel; its negative sidelobes preserve sharp ePSF structure), "quadratic" (same fit with a 2nd-order basis; used automatically for kernels smaller than 5x5 where a quartic is underdetermined), "auto" (mirrors the photutils 3.1 'auto' option: a quartic kernel sized to 0.7x the FWHM on the oversampled grid, skipped entirely when that footprint spans fewer than 5 grid points), "gaussian", or "none" to disable smoothing.
`psf_smoothing_kernel_size`	*null* (int|null)	Optional explicit smoothing-kernel size (pixels, on the oversampled grid). If null, auto-sized to ~psf_smoothing_kernel_size_scale_px native pixels.
`psf_smoothing_kernel_size_scale_px`	`1.0` (float)	Auto kernel footprint in NATIVE pixels: kernel size ~= scale * oversampling. ~1.0 reproduces the canonical Anderson & King kernel (osamp=4 -> 5x5); larger values over-smooth and erase ePSF structure (Godden & Blundell 2025). Kernels smaller than 5x5 use a quadratic basis (a 3x3 quartic fit is underdetermined and degenerates to an identity kernel); at oversampling=1 the kernel is ~1x1 i.e. effectively no smoothing.
`psf_smoothing_kernel_size_max`	`9` (int)	Maximum auto smoothing-kernel size (odd; oversampled grid).
`psf_smoothing_kernel_target_samples`	`10.0` (float)	Effective star samples per oversampled gridpoint (n_stars/oversample^2) at which the minimal ~1px kernel suffices. When samples/gridpoint falls below this, the kernel footprint is boosted by sqrt(target/samples) (variance-equalising), e.g. 45 stars at oversample=4 -> 2.8 samples/cell -> ~1.9px target, bounded by psf_smoothing_kernel_max_fwhm_frac * FWHM. 0 disables the boost.
`psf_smoothing_kernel_boost_max`	`2.5` (float)	Maximum footprint boost factor for thin sampling (native px multiplier). Caps the smoothing growth on very sparse fields so the kernel stays a reasonable fraction of the stamp.

continued

Key	Default (type)	Description
psf_smoothing_kernel_max_fwhm_frac	1.25 (float)	Footprint cap as a fraction of the build FWHM. Very wide kernels broaden the ePSF (synthetic undersampled builds: >=1.5xFWHM -> +10-25% FWHM), but under-smoothing is worse on real thin-sampled fields – residual noise walks the ePSF core ~10% narrow across iterations, and ~1.25xFWHM kernels (e.g. 9x9 at osamp=4, FWHM~1.8 px) recovered the true effective width. The realised kernel is the largest odd size within the cap. Applies to auto-sized and explicit psf_smoothing_kernel_size values; <=0 disables.
psf_smoothing_kernel_cap_min_px	1.75 (float)	Floor on the footprint cap in native pixels. For the most undersampled PSFs (FWHM ~1.5 px) a ~1.75 px kernel is still the empirical optimum, so the FWHM cap must not forbid it.
sextractor_reject_constant_regions	True (bool)	If False, do not reject sources in flat-variance regions.
psf_min_candidates	8 (int)	Minimum PSF candidates to preserve during quality cuts (adaptive guard against over-pruning).
psf_pool_preferred_min	25 (int)	If optimum-radius selection leaves fewer than this many sources and the broader pre-optimum pool (isolation + quality + linearity + border vetted) is larger, use the broader pool for PSF building. Optimum-radius vetting is tuned for aperture stability; PSF. build applies its own PSF-specific screening, so more candidates produce a better-constrained ePSF.
linearity_min_inlier_frac	0.35 (float)	Minimum fraction of quality-cut sources that must be linearity inliers for the RANSAC inlier test to vet sources. Below this, the fit is treated as unreliable (e. g. undersampled peak-flux scatter) and all quality-cut sources are kept; saturation_range is still reported from the fitted inliers.
psf_saturate_fraction	0.90 (float)	Nominal saturation cutoff for PSF stars (peak < fraction * saturate); auto-relaxed when candidate pool is small. Keep in sync with template_subtraction. subtraction_saturate_fraction.
psf_elongation_max	1.5 (float)	Maximum allowed ELONGATION (a/b axis ratio) for PSF stars; rejects trailed or blended stars. Uses ELONGATION or ELLIPTICITY column if present; skip if neither available. When the field median exceeds this (the image PSF itself is elongated, e.g. tracking drift), the cut relaxes to ~1.3x the field median so only outliers are struck; field-wide elongation also scales the flux-radius and minor-axis floors below.
psf_flags_max	1 (int)	Maximum SExtractor FLAGS for PSF stars. 0=clean, 1=nearby, 2=blended. Default 1 rejects blended sources (FLAGS >= 2) which corrupt ePSF shape. Auto-relaxed when too few candidates remain.
psf_isolation_radius_fwhm	3.0 (float)	Minimum neighbour-free radius (in FWHM units) for PSF star selection; rejects stars with a close companion that would contaminate the ePSF cutout.
psf_isolation_neighbour_snr_min	10.0 (float)	Minimum S/N a raw-catalog source needs to count as a contaminating neighbour in the isolation cut. Sub-threshold rows are noise peaks whose contribution the cutout-level contamination checks already mask; counting them vetoes good stars next to detection noise. Mirrors the SNR floor the supplement neighbour tree applies; set <= 0 to count every catalog row as a neighbour.

continued

Key	Default (type)	Description
psf_min_pair_separation_fwhm	1.0 (float)	Minimum pairwise separation (in FWHM units) between candidates on the final PSF pool. Two entries closer than this are dropped as an unresolved blend, except near-identical positions (< psf_dup_radius_px) which are treated as the same star listed twice and deduplicated (higher-ranked entry kept). Catches cross-list duplicates that per-list isolation checks cannot see. Set to 0 to disable.
psf_dup_radius_px	2.0 (float)	Separation below which two PSF-pool entries are treated as duplicate detections of the same star (better-ranked kept) rather than a blend. Matches the supplement-pool dedupe radius.
psf_defect_veto_min_fails	2 (int)	A candidate failing this many independent quality cuts (saturation, FLAGS, elongation, FWHM consistency, FLUX_RADIUS, peak/aperture, A/B size, sharpness, profile concentration, isolation) is removed unconditionally, even when every cut was individually skipped by the psf_min_candidates keep-floor. Prevents defect pools from reaching the ePSF when each cut is skipped one at a time. Set to 0 to disable.
psf_defect_exclusion_fwhm	2.0 (float)	Distance (in FWHM units) from cosmic ray / hardware defect pixels within which sources are excluded from the PSF candidate pool. Default 2.0 (was hardcoded 1.0); combined with the dilated CR mask (~2*FWHM radius), this gives ~4*FWHM total exclusion, reducing CR contamination in PSF stamps. The same radius also guards the image-template flux-matching source list.
psf_defect_dominance_frac	0.4 (float)	Defect-dominance warning threshold. A warning is logged when more than this fraction of surviving candidates sit near the lower FWHM bound (catalog stage), or have a measured cutout FWHM below 50% of the image FWHM (post-extraction stage): the signature of a hot-pixel/CR-dominated pool, which the ensemble-relative vetoes cannot fix. Diagnostic only; no candidates are removed.
psf_mask_core_min_dist_fwhm	0.75 (float)	PSF candidates within this distance (in FWHM units) of a masked pixel (chip gap, NaN, saturation, streak, cosmic ray) are always rejected: the defect sits inside the stellar core and cannot be repaired by per-pixel weights. Hard cut, no keep-floor. Set to 0 to disable.
psf_spike_flag_radius_fwhm	4.0 (float)	Radius (in FWHM units) of the stamp region counted for MaxiMask diffraction-spike (SP) pixels around each PSF candidate. The proximity cut above only rejects stars whose centroid lies near a flagged pixel; an arm that the network only detects away from the core still crosses the ePSF stamp, so flagged-pixel count inside this radius is the source-level check. Inactive when MaxiMask is off.
psf_spike_min_flagged_px	20 (int)	Minimum SP-flagged pixels within psf_spike_flag_radius_fwhm of the centroid before the candidate is rejected as spike-contaminated. Arms are 1-3 px wide and tens of px long, so ~20 px is a conservative arm-segment threshold. Set to 0 to disable.
psf_fwhm_min_frac	0.5 (float)	Minimum FWHM as fraction of image FWHM for PSF candidates; rejects cosmic rays (abnormally small FWHM). Set to 0 to disable.
psf_fwhm_max_frac	2.5 (float)	Maximum FWHM as fraction of image FWHM for PSF candidates; rejects extended defects/galaxies (abnormally large FWHM). Set to a large value to disable.

continued

Key	Default (type)	Description
psf_flux_radius_min_frac	0.3 (float)	Minimum SExtractor FLUX_RADIUS (half-light radius) as fraction of image FWHM for PSF candidates; rejects cosmic rays (abnormally small FLUX_RADIUS, typically <1 px regardless of image FWHM). Set to 0 to disable.
psf_peak_to_aperture_ratio_max	0.0 (float)	Maximum allowed peak_flux/flux_AP ratio for PSF candidates; rejects cosmic rays (sharp spikes with very high peak-to-total flux ratio). 0 = auto-compute from FWHM (5x expected Gaussian ratio, capped at 0.5). A hard ceiling of 0.80 applies even to configured values: ratios that high are all-flux-in-1-2-pixel hot-pixel clusters, not stars. Set to a large value to disable.
psf_min_ab_pixels	0.0 (float)	Fixed floor for minimum SExtractor A_IMAGE/B_IMAGE (semi-axes) in pixels for PSF candidates. Set to 0 to rely on psf_min_ab_frac only.
psf_min_ab_frac	0.3 (float)	Minimum A_IMAGE/B_IMAGE as fraction of image FWHM for PSF candidates; rejects cosmic rays (abnormally small source profiles). Combined with psf_min_ab_pixels via max(). Set to 0 to disable.
psf_flux_radius_max_frac	0.9 (float)	Maximum SExtractor FLUX_RADIUS (half-light radius) as fraction of the source's own FWHM for PSF candidates; rejects galaxies and extended defects. A point source's half-light radius is ~0.5-0.7x its FWHM, so ~0.9 admits stars while excluding extended profiles. This replaces the removed CLASS_STAR gate, whose neural classifier is uncalibrated on some cameras. Candidates with missing/NaN FLUX_RADIUS are kept (unmeasurable is not failing). Set to 0 to disable.
psf_sharpness_max	1.0 (float)	Maximum IRAFStarFinder sharpness for PSF candidates; rejects extended galaxies (sharpness > 1.0). Set to a large value to disable.
psf_fwhm_clip_sigma	2.5 (float)	Sigma threshold for the iterative FWHM sigma-clip on PSF candidates (rejects extended/CR-like outliers relative to the candidate distribution). Guarded by psf_fwhm_clip_min_keep.
psf_fwhm_clip_sigma_undersampled	3.5 (float)	Sigma threshold for the FWHM sigma-clip on undersampled images (FWHM <= undersampled_fwhm_threshold). Looser because per-source FWHM measurements are noisier on a few bright pixels.
psf_fwhm_clip_min_keep	4 (int)	Keep-floor for the FWHM sigma-clip (hard minimum 3). Deliberately NOT tied to psf_min_candidates: a smaller clean pool is recoverable via supplement while FWHM outliers poison the ePSF build.
psf_cutout_contamination_check	True (bool)	Enable pixel-level cutout contamination check (catches neighbouring stars/galaxies/trails/streaks in outer annulus that catalog-level isolation misses).
psf_threshold_limit	5.0 (float)	Minimum detection threshold for PSF-star candidates (lower is less restrictive).
psf_threshold_hard_min	3.0 (float)	Hard floor applied even when the keep-floor relaxes psf_threshold_limit: candidates below ~3 sigma are background peaks, not stars, and never help the ePSF build.
psf_snr_min	10.0 (float)	Minimum SNR for PSF-star candidates.

continued

Key	Default (type)	Description
psf_snr_hard_min	10.0 (float)	Hard floor applied unconditionally, even when the keep-floor relaxes psf_snr_min or undersampled mode disables the strict cut: no PSF star below this S/N ever reaches the ePSF build. Set equal to psf_snr_min to guarantee the minimum on every selected star; a pool that cannot meet it falls back to the analytic PSF.
psf_snr_rescue_min	5.0 (float)	Keep-floor rescue for the S/N hard floor: when applying psf_snr_hard_min would drop the pool below psf_min_candidates, the highest-S/N rejected candidates at or above this S/N are restored until the pool reaches psf_min_candidates. An S/N 5-10 star is still a usable ePSF sample on a starved field; cutout-level vetting gates the junk. Set <= 0 or >= psf_snr_hard_min to disable.
psf_supplement_max_add	800 (int)	Cap on sources added to the PSF pool by the catalog + detection-table supplement (highest SNR kept first); bounds pathological pools in dense fields.
psf_supplement_fwhm_min_frac	0.5 (float)	Supplement candidates with a measured SExtractor FWHM below this fraction of the image FWHM are rejected at supplement time (hot pixels/cosmic rays). Unlike the build's FWHM-consistency cut this gate is not keep-floor protected: supplements are optional additions, and junk admitted here drags the pool-median FWHM and fails cutout vetting anyway. Candidates with no measured FWHM (e.g. catalog frames) are kept and vetted by the cutout checks in PSF.build().
psf_supplement_fwhm_max_frac	2.5 (float)	Supplement candidates with a measured SExtractor FWHM above this multiple of the image FWHM are rejected at supplement time (resolved galaxies/diffuse sources).
psf_snr_max	1000000.0 (float)	Maximum SNR for PSF-star candidates.
psf_build_min_keep_frac_after_cut	0.60 (float)	Do not apply a threshold/SNR cut if it would keep fewer than this fraction of current candidates.
psf_preselect_factor	1.3 (float)	Candidate preselection factor before spatial downselection (closer to 1.0 is less restrictive).
psf_disable_build_quality_cuts_undersampled	True (bool)	In undersampled regime, disable strict threshold/SNR candidate cuts to avoid over-pruning.
psf_build_sampling_boost_max	1.6 (float)	Maximum adaptive build-window boost for normal sampling.
psf_build_sampling_boost_max_undersampled	2.2 (float)	Maximum adaptive build-window boost for undersampled images.
psf_build_from_aligned	True (bool)	If True, build PSF from the aligned image (same as target photometry); if False, build from original pre-alignment image to avoid resampling degradation.
psf_empirical_min_stars	6 (int)	Post-vetting floor applied INSIDE the ePSF build: when the ensemble that actually built the model is thinner than this, the fitted analytic composite is delivered instead - but only when the thin empirical model already failed a quality gate or demonstrably loses to the analytic composite on its own stars. A working thin ePSF is kept (flagged "understaffed"). The empirical attempt itself always runs down to psf_build_min_stars. Undersampled images never swap: an analytic profile cannot represent pixel-phase structure.

continued

Key	Default (type)	Description
psf_build_min_stars	2 (int)	Absolute floor on the PSF-pool size for an empirical ePSF attempt: the build always runs down to this count on every field. Below it the build is skipped for the analytic fallback.
psf_adaptive_iterations	True (bool)	Enable adaptive EPSFBuilder iteration strategy (start with fewer iterations, increase only if not converged).
psf_maxiters_initial	5 (int)	Initial EPSFBuilder max iterations when adaptive_iterations is True (lower = faster, but may need retry). Undersampled builds use at least min(psf_maxiters_max, 10) so the first pass can reach the relaxed center-accuracy convergence point (~7-9 iterations on thin-sampled fields) without a rebuild.
psf_maxiters_max	10 (int)	Maximum EPSFBuilder iterations for retry when adaptive_iterations is True and initial run didn't converge.
psf_epsf_fitter_maxiters	50 (int)	Cap on inner LM iterations of each per-star centre/flux fit inside EPSFBuilder (photutils >=3.0 fitter_maxiters; default there is 100). Well-posed star fits converge in ~10; the cap only bounds stars that cannot converge and will be excluded after 3 build iterations anyway.
psf_center_accuracy_undersampled	0.05 (float)	EPSFBuilder center_accuracy (px) used on undersampled images. The default 1e-3 px is unreachable at ~n_stars/oversample^2 samples per gridpoint (star centers jitter ~0.03-0.1 px/iter), so builds never converge and each extra iteration stacks noise-drawn residuals – observed to walk the ePSF core ~10-30% narrow on undersampled SN2022mop fields. ~0.05 px (~3% of FWHM) stops the iteration once centers settle; well-sampled builds keep the photutils default.
psf_epsf_fwhm_min_frac	0.85 (float)	Post-build sanity gate: if the measured ePSF FWHM is below this fraction of the image FWHM, the model is degraded (the effective PSF is pixel-integrated and can only be broader than the seeing estimate). A converged out-of-band model is kept with a warning; a NON-converged one is discarded (fall back to aperture-only) unless the residual gate rescues it (see psf_epsf_fwhm_discard_*).
psf_epsf_fwhm_max_frac	1.5 (float)	Post-build sanity gate: if the measured ePSF FWHM exceeds this multiple of the image FWHM, the model is treated as degraded (noise-broadened / centroid-smeared). Same keep/discard policy as psf_epsf_fwhm_min_frac.
psf_epsf_fwhm_discard_min_frac	0.6 (float)	Hard floor for the non-converged FWHM discard: below this fraction of the image FWHM the miss is gross (resampled-defect stacks read ~0.5x) and the model is discarded unconditionally. Between this and psf_epsf_fwhm_min_frac the miss is marginal – thin-but-clean pools land there too – so the build is kept when the residual gate (psf_epsf_residual_gate) shows the empirical model fits its own stars no worse than psf_epsf_residual_ratio_max times the analytic seed. Fields where the gate cannot run (disabled, no usable seed, unmeasurable residuals) keep the conservative discard.
psf_epsf_fwhm_discard_max_frac	2.0 (float)	Hard ceiling counterpart of psf_epsf_fwhm_discard_min_frac for non-converged builds.

continued

Key	Default (type)	Description
`psf_epsf_negfrac_max`	`0.4` (float)	Post-build wing-quality gate on the RAW empirical model at oversampling >= 2: if |sum of negative pixels| / sum exceeds this, the residual median is noise-drawn (too few samples per grid cell) and the build descends the oversampling ladder. FWHM and centre-accuracy gates cannot see wing pathology – fields can pass them with negative-flux fractions >1. A real PSF is positive; clean production builds sit below ~0.15. Skipped at osamp=1, where coherent resampling ringing legitimately dips below zero and no lower rung exists.
`psf_epsf_residual_gate`	`True` (bool)	Final accuracy diagnostic on the accepted empirical model: compare the median flux-normalised residual MAD of the PSF-star cutouts under the empirical ePSF vs the pixel-integrated analytic Moffat on the same grid (each star evaluated at its own fitted centre/flux, so the empirical model has every advantage). If the empirical model fits psf_epsf_residual_ratio_max times worse it is degraded; when psf_epsf_residual_swap is on, a sane fitted analytic composite is delivered instead of the degraded empirical model. Shape gates cannot see this failure mode – degraded production stamps fit the stars 3-16x worse than the Moffat while good builds sit at ~parity.
`psf_epsf_residual_ratio_max`	`1.3` (float)	Residual-MAD ratio above which the analytic model wins over the accepted empirical ePSF. Good empirical builds measure ~0.95-1.0 of the analytic MAD (they capture real resampling structure the Moffat lacks); only a noise/contamination-dominated build loses decisively.
`psf_epsf_residual_swap`	`False` (bool)	When the residual gate fires (empirical ePSF fits its own stars worse than psf_epsf_residual_ratio_max times the analytic composite), deliver the fitted analytic composite rather than the degraded empirical model. Default off: the analytic model is a last-resort deliverable, not a replacement for an empirical model that passed every shape gate - the gate still logs the ratio and marks the verdict non-converged. Set True to deliver analytic on decisive losses (degraded production stamps fit 3-16x worse; J2344 ePSF epochs median reduced-chi2 ~11 vs ~1.7 for analytic-composite epochs).
`psf_epsf_coarse_kernel_px`	`2.0` (float)	Coarse-to-fine schedule for undersampled osamp>=2 builds: the first ~4 iterations run with a smoothing kernel of this native-pixel footprint, producing a smooth model so the per-star shift refit cannot drag stars onto shared pixel-phase classes (measured: 0/9 empty classes and negfrac 0.08 at x3 vs 3/9 empty and 0.23 single-pass). The nominal kernel then refines detail. 0 disables the coarse phase.
`psf_epsf_max_star_shift_px`	`1.0` (float)	Bound on cumulative star-centre drift in the ePSF shift refit, in native pixels. Unbounded refits collapse the star ensemble onto one or two pixel-phase classes on undersampled data (the rest of the grid is then interpolated, not measured). Legitimate corrections are <0.3 px since the measured centroid is good to ~0.01 px, so larger values mainly help when the initial centroids are poor. 0 disables the bound.

continued

Key	Default (type)	Description
psf_prune_unsettled_retry	True (bool)	When an ePSF build fails to converge, drop the sources blocking convergence and rebuild: stars still drifting by >= center_accuracy on the final iteration (the build converges only when every fitted star's centre stops moving) plus any photutils excluded for repeated fit failures. A converged build runs no pruning at all, so good fields are untouched.
psf_prune_unsettled_rounds	3 (int)	Maximum prune-rebuild rounds per ePSF attempt. A single pass is often insufficient: the next cohort of unsettled stars only becomes identifiable once the worst movers are gone.
psf_prune_unsettled_max_frac	0.4 (float)	Maximum TOTAL fraction of the PSF-star pool the unsettled-star prune may remove across all rounds (worst movers first). Never drops below psf_shape_outlier_min_keep stars.
psf_center_accuracy_relaxed_px	0.01 (float)	Relaxed centre-movement bound (px) for the final non-convergence verdict. The strict photutils threshold (1e-3 px) can be unreachable when centroids are S/N-limited: well-posed builds plateau at a few millipixel jitter. If the final iteration's movement is already below this bound the model is accepted as effectively settled and the summary reports "converged: yes (relaxed)". <=0 disables.
psf_salvage_partial_epsf	True (bool)	When the ePSF build raises mid-iteration (one bad star's fit crash propagates out of photutils and discards the whole result), keep the newest completed iteration's empirical model and report it as non-converged. The salvaged model still passes through every post-build quality gate; only when nothing usable remains does the build return no model (aperture-only photometry downstream). False restores the pre-salvage behaviour.
psf_epsf_wing_smooth	False (bool)	OPT-IN post-build ePSF stamp cleanup for empirical models: blends the wings toward a Gaussian-smoothed copy between psf_epsf_wing_smooth_start_fwhm and psf_epsf_wing_smooth_end_fwhm (suppressing uncorrelated gridpoint speckle at ~n_stars/osamp^2 samples per cell) and tapers the outer psf_epsf_edge_taper_px native pixels to zero. The photometric core, the coarse resampling ringing, and the sum == prod(oversampling) flux convention are preserved – but the stamp is still modified after the fit converged, which can shift the encircled energy and symmetrise real wing ellipticity. Default off: the photometric model should be what the star ensemble converged to. Enable only after verifying on residuals that speckle noise, not signal, is being removed.
psf_epsf_wing_smooth_start_fwhm	1.5 (float)	Inner radius (x image FWHM) where wing smoothing blends in; inside this the raw ePSF is kept.
psf_epsf_wing_smooth_end_fwhm	3.5 (float)	Outer radius (x image FWHM) where the stamp is fully the smoothed copy.
psf_epsf_wing_smooth_sigma_px	1.25 (float)	Gaussian sigma in native px for the wing smoothing; kept small so ~2-3 px coherent ringing survives while gridpoint speckle is suppressed.
psf_epsf_edge_taper_px	2.0 (float)	Width in native pixels of the linear edge taper that forces the stamp border to zero.

continued

Key	Default (type)	Description
psf_epsf_wing_flatten_fwhm	3.5 (float)	Radius (x image FWHM) beyond which the wings blend to the azimuthal-median radial profile. Past ~3.5xFWHM the real wing flux is below the ePSF gridpoint-noise floor (star stacks carry ~2x less azimuthal variance than the raw stamp), so the radial profile is the better estimate of the true wing; only residual mottle is removed. 0 disables.
psf_epsf_wing_flatten_width_px	2.0 (float)	Width in native pixels of the blend zone between the smoothed stamp and the azimuthal profile.
psf_epsf_phase_uniformity_alpha	0.01 (float)	Chi-square p-value threshold for the post-build subpixel-phase uniformity diagnostic (photutils ePSF guidance: fitted star centres should be uniformly distributed over the oversample^2 phase cells, with ~10 stars per cell for randomly placed samples). A build with p below this marks the centres biased / the star sample heterogeneous (mixed PSFs, saturated or contaminated cutouts) and logs a warning. The test only runs when the pool is large enough to be meaningful (>= 4*oversample^2 stars); phase coverage and samples/cell are always reported.
psf_auto_oversample_undersampled	True (bool)	Automatically increase oversampling for undersampled images (FWHM <= 2.5 px). Factor is chosen adaptively based on PSF star count.
psf_oversample_min_samples_per_pixel	3.0 (float)	Minimum samples per oversampled gridpoint for adaptive oversampling. With N stars and oversampling k, each gridpoint gets ~N/k^2 samples (each star lands only on gridpoints of its own pixel-phase class). Below ~3 samples/cell the sigma-clipped median cannot function – most cells hold 0-1 noisy residuals and single-star contamination enters the ePSF at full amplitude (production fields at 1-3 samples/cell show negative-flux fractions >0.5). The factor is chosen to keep the median functional, not to maximise grid resolution.
psf_oversample_max	4 (int)	Maximum oversampling factor for adaptive oversampling.
psf_oversample_min_undersampled	1 (int)	Minimum oversampling factor for undersampled images with >= psf_oversample_hard_min_stars stars. Set to 1 so starved fields can reach the robust osamp=1 regime (every star constrains every grid cell); the former floor of 3 forced 1-3 samples/cell builds whose wings were noise-dominated.
psf_oversample_hard_min_stars	5 (int)	Below this star count, oversampling is disabled entirely (too few stars for a reliable oversampled ePSF).
psf_adaptive_sigma_clip	True (bool)	Adapt sigma clipping parameters based on number of PSF stars (less aggressive with few stars, more aggressive with many).
psf_cutout_ee_fraction	0.9995 (float)	Encircled-energy fraction of the analytic init PSF core profile used to size the star cutout (radius enclosing this fraction + psf_cutout_ee_margin_fwhm x FWHM sets the diameter). Values outside (0, 1) disable EE sizing and use the formula result.
psf_cutout_ee_margin_fwhm	1.0 (float)	Extra margin added to the measured EE radius, in FWHM units (covers near wings not captured by the core profile).
psf_cutout_size_scale_fwhm	10.0 (float)	Legacy EPSF star-cutout size in FWHM units; used as the fallback when EE-based sizing fails or is disabled.
psf_cutout_min_arcsec	9.0 (float)	Minimum PSF build cutout size in arcsec (converted to pixels via pixel_scale); also a floor for EE-based sizing.

continued

Key	Default (type)	Description
psf_build_sigma_clip_sigma	4.0 (float)	SigmaClip threshold used while stacking stars in EPSFBuilder (higher is less restrictive and preserves wings better).
psf_build_sigma_clip_maxiters	15 (int)	Maximum SigmaClip iterations for EPSFBuilder stacking.

A.4.9 ePSF star vetoes: cutout contamination

Pixel-level vetoes on the candidate cutouts: quadrant asymmetry, edge and border excess, radial reversals, secondary peaks, and streak tests that catch what catalogue-level isolation misses.

```
autophot_input["photometry"]["psf_cutout_contamination_check"] = True
autophot_input["photometry"]["psf_contam_asymmetry_sigma"] = 3.0
```

Table 16: Photometry: ePSF star vetoes: cutout contamination — configuration reference.

Key	Default (type)	Description
psf_contam_asymmetry_sigma	3.0 (float)	Max deviation of quadrant medians in the outer annulus, in units of the annulus MAD sigma, before rejecting the cutout (one-sided contamination from neighbour wings or trails). The former 0.3 default was a fraction of sigma, not of flux: real sky gradients and PSF ellipticity trip a 0.3-sigma quadrant test constantly, flagging most legitimate PSF stars. (Formerly misnamed psf_contam_asymmetry_frac; the old key is still honoured with a deprecation warning.)
psf_contam_edge_excess_sigma	3.0 (float)	Max sigma excess of edge-ring median over inner-ring median before rejecting cutout (neighbour wings entering cutout edge).
psf_contam_border_excess_sigma	3.0 (float)	Max sigma excess of cutout border pixel median over local background before rejecting cutout (neighbour flux at cutout boundary).
psf_contam_border_max_sigma	8.0 (float)	Max sigma excess of any single border pixel over local background before rejecting cutout (trail/streak crossing cutout boundary).
psf_contam_radial_reversal_sigma	3.0 (float)	Max increase (in annulus MAD sigma units) allowed between consecutive concentric ring medians before rejecting the cutout (contaminating source in the outer region). The former 0.3 default rejected stars on sub-sigma ring-to-ring noise. (Formerly misnamed psf_contam_radial_reversal_frac; the old key is still honoured with a deprecation warning.)
psf_contam_secondary_peak_sigma	5.0 (float)	Min sigma of a secondary peak in the core region (within 2*FWHM of center) above background before rejecting cutout (blended neighbour in core). Uses local maxima detection to find ALL off-center peaks, not just the brightest pixel.
psf_contam_streak_sigma	5.0 (float)	Min sigma excess of any row, column, or diagonal median in the annulus above the annulus median before rejecting cutout (satellite trail or cosmic-ray streak).
psf_contam_annulus_peak_sigma	5.0 (float)	Min sigma of any local maximum in the outer annulus above background before rejecting cutout (faint contaminating star that doesn't elevate median-based tests). Set to a large value to disable.

continued

Key	Default (type)	Description
`psf_contam_flux_excess_sigma`	`10.0` (float)	Max ratio of total positive flux excess in annulus to expected noise-only excess before rejecting cutout (catches overall flux contamination from neighbour wings). Set to a large value to disable.
`psf_contam_core_ellipticity_max`	`0.15` (float)	Max ellipticity (from second moments of core flux within 2*FWHM) before rejecting cutout (catches close blends within 1-2 FWHM that don't create separate local maxima). Clean stars have ~0.03; blended stars > 0.15. Set to 1.0 to disable.
`psf_contam_core_ellipticity_snr_min`	`8.0` (float)	Minimum peak SNR inside the core for the ellipticity test to run: below this the second moments are noise-dominated and cannot distinguish a close blend from sky fluctuations.
`psf_contam_mask_ring_sigma`	`3.0` (float)	When the keep-floor prevents flux-based rejection (too few clean stars), contaminating pixels are masked instead of dropping the star: annulus pixels exceeding their own concentric ring's median by this many ring-sigmas (MAD) are excluded from the residual stack while the star's core is retained.
`psf_contam_core_reject`	`True` (bool)	Always reject cutouts with a detected secondary peak closer than ~1.5*FWHM, even when the keep-floor binds: annulus masking cannot repair a neighbour inside the fit region, so these stars are dropped like hardware defects. Ellipticity flags stay keep-floor-gated (noisy on undersampled cores).
`psf_contam_hotpix_sigma`	`5.0` (float)	Gate for the hot-pixel-cluster test (peak-to-core-flux fraction within ~0.5*FWHM of centre). A defect cluster concentrates nearly all core flux into 1-2 pixels; a genuine PSF spreads it across the seeing disk. The annulus asymmetry/edge/reversal tests cannot see a symmetric central blob, and the ellipticity test cannot either. Set to 0 to disable.
`psf_contam_hotpix_max_peak_frac`	`0.65` (float)	Maximum allowed fraction of positive core flux carried by the single brightest pixel before the cutout is rejected as a hot-pixel cluster. A single-pixel defect approaches 1.0; a seeing-limited PSF core (FWHM >= ~2 px) stays well below 0.5.
`psf_contam_cog_check`	`True` (bool)	Enable the curve-of-growth test: enclosed flux in concentric apertures (normalised by the plateau flux, so brightness-independent) is compared against the clean-star ensemble and against a family of plausible Moffat encircled-energy profiles. Defect spikes saturate early and extended or wing-contaminated sources rise late; both look like stars to the peak/flag cuts.
`psf_contam_cog_sigma`	`5.0` (float)	Max deviation of a cutout's normalised growth curve from the clean-star ensemble median, in units of the per-radius ensemble MAD (floored at 0.04 EE fraction), before rejection.
`psf_contam_cog_max_dev`	`0.30` (float)	Absolute cap on EE-fraction deviation from the ensemble median; catches outliers when the ensemble MAD is inflated by many junk cutouts.
`psf_contam_cog_moffat_max_dev`	`0.15` (float)	Max EE-fraction distance outside the plausible Moffat encircled-energy envelope (beta 2.5-7, FWHM -25%/+40%). Ensemble-independent, so it still rejects spikes and diffuse sources when most of the pool is non-stellar.

continued

Key	Default (type)	Description
`psf_contam_cog_snr_min`	`15.0` (float)	Minimum core peak S/N (peak above background over pixel noise, taken as the quieter of the annulus and edge-ring MADs) for the curve-of-growth test to run: below this the plateau estimate is dominated by aperture-noise selection bias and noise looks like a growth-curve outlier.
`psf_contam_cog_always_reject`	`True` (bool)	Treat curve-of-growth failures as unrepairable (always rejected like hardware defects): the defect is the core profile itself, so annulus masking cannot fix it. Set False to let them fall back into the keep-floor flux class.

A.4.10 ePSF star vetoes: ensemble shape outliers

Rejects cutouts whose flux-normalised shape disagrees with the ensemble median stack – miscentred, contaminated, or deviant morphologies that would stop the build converging.

```
autophot_input["photometry"]["psf_shape_outlier_rejection"] = True
autophot_input["photometry"]["psf_shape_outlier_nsigma"] = 4.0
```

Table 17: Photometry: ePSF star vetoes: ensemble shape outliers — configuration reference.

Key	Default (type)	Description
`psf_shape_outlier_rejection`	`True` (bool)	Drop PSF-star cutouts whose flux-normalised shape is inconsistent with the ensemble median stack. One or two poor sources (miscentered, contaminated, deviant morphology) leave EPSFBuilder recentering chasing disagreement and the build never converges. Unlike the FFT metric this tests image-domain shape, so it still works on undersampled and elongated fields.
`psf_shape_outlier_nsigma`	`4.0` (float)	Rejection threshold in robust sigma units of the per-star median-stack residual MAD.
`psf_shape_outlier_max_frac`	`0.3` (float)	Maximum fraction of the PSF-star pool the shape-outlier check may drop (worst-scoring stars first).
`psf_shape_outlier_min_stars`	`6` (int)	Minimum PSF-star pool size before the shape-outlier check runs; below this every star is needed and pair-wise consistency cannot be estimated.
`psf_shape_outlier_min_keep`	`4` (int)	Never drop below this many PSF stars regardless of the outlier scores.
`psf_shape_outlier_abs_fwhm_floor`	`0.3` (float)	Absolute size floor on the measured per-cutout FWHM, as a fraction of the image FWHM. The ensemble-relative outlier test fails when the pool itself is defect-dominated (the median template becomes defect-shaped and real stars look like outliers); this floor drops hot-pixel/CR clusters regardless of the ensemble norm. Absolute-floor rejects bypass psf_shape_outlier_max_frac but still respect psf_shape_outlier_min_keep. Set to 0 to disable.

A.4.11 ePSF star vetoes: Fourier-domain outliers

Fourier-domain outlier rejection on the cutouts; it catches artifacts (hot pixels, cosmic-ray clusters) whose near-flat azimuthal power spectrum the image-domain tests miss.

```
autophot_input["photometry"]["psf_fft_rejection"] = True
autophot_input["photometry"]["psf_fft_n_sigma"] = 4.0
```

Table 18: Photometry: ePSF star vetoes: Fourier-domain outliers — configuration reference.

Key	Default (type)	Description
psf_fft_rejection	True (bool)	Enable FFT-based rejection during PSF star selection.
psf_fft_n_sigma	4.0 (float)	Rejection threshold (sigma units) for FFT outlier metric.
psf_fft_min_frac_keep	0.5 (float)	Minimum fraction of PSF stars to retain after rejection.
psf_fft_min_stars	10 (int)	Minimum PSF stars required to run FFT rejection; otherwise skip.
psf_fft_disable_undersampled	True (bool)	On undersampled images, relax the FFT rejection threshold to >=6 sigma instead of removing stars at psf_fft_n_sigma: high-frequency cutout power is dominated by subpixel-phase sampling jitter, not artifacts, so the metric rejects genuine stars at the normal threshold. The relaxed pass still catches extreme outliers (hot-pixel/CR clusters have a near-flat azimuthal power spectrum). Set False to run at the unmodified psf_fft_n_sigma.
psf_fft_use_mad	True (bool)	Use MAD-based scale estimate for FFT rejection statistics.
psf_fft_use_median_deviation	True (bool)	Use median absolute deviation around the median (outlier-resistant).

A.4.12 ePSF star vetoes: normalisation mask

How many masked or non-finite pixels may sit inside the flux-normalisation circle of a candidate before it is rejected.

```
autophot_input["photometry"]["psf_norm_radius_mask_frac_max"] = 0.02
```

Table 19: Photometry: ePSF star vetoes: normalisation mask — configuration reference.

Key	Default (type)	Description
psf_norm_radius_mask_frac_max	0.0 (float)	Maximum fraction of masked/non-finite pixels allowed inside the flux-normalisation circle (radius = aperture_radius) of a PSF-star cutout. The ePSF build normalises each star within that radius, so a masked pixel there silently shrinks the normalised area and biases the star's flux scale relative to the ensemble. 0.0 (default) rejects any star with a masked pixel inside the radius; raise slightly (e.g. 0.02) to tolerate isolated bad pixels on heavily masked fields.
psf_norm_radius_mask_frac_rescue_max	0.25 (float)	Keep-floor rescue for the normalisation-radius mask gate: when a strict cut would leave fewer than psf_min_candidates stars, the least-bad rejected cutouts are kept while their masked fraction inside the circle stays under this cap (masked pixels carry weight 0, so a partially masked star still contributes valid samples).

A.4.13 Analytic composite-Moffat PSF

The fitted composite-Moffat model that seeds the iterative ePSF build, serves as the residual diagnostic, and is delivered as a fallback when too few vetted stars survive.

```
autophot_input["photometry"]["psf_use_analytic_init"] = True
```

```
autophot_input["photometry"]["psf_analytic_components"] = 3
autophot_input["photometry"]["psf_analytic_fallback"] = True
```

Table 20: Photometry: Analytic composite-Moffat PSF — configuration reference.

Key	Default (type)	Description
`psf_init_moffat_beta`	`4.765` (float)	Moffat beta for initial ePSF kernel (4.765 ~ atmospheric-turbulence-like PSF; lower values add stronger wings).
`psf_analytic_fallback`	`True` (bool)	When no empirical ePSF is available (pool below psf_build_min_stars or the build failed), fit a composite-Moffat analytic PSF to the surviving pool cutouts and use it for PSF photometry, flagged "analytic-composite" (fitted) or "analytic-canonical" (no usable fit; profile built from the image FWHM alone) in the output psf_model column. A flagged analytic model recovers PSF-fit positions/fluxes and the AP-vs-PSF zeropoint cross-check on sparse fields. Skipped when the image FWHM is not finite. False restores strict aperture-only behavior.
`psf_use_analytic_init`	`True` (bool)	Seed the iterative ePSF build with the fitted composite-Moffat analytic model. The analytic model seeds the iteration and serves as the residual-accuracy diagnostic; when the empirical build is degraded or understaffed the fitted analytic composite can be delivered instead (psf_model "analytic-composite"). A good seed speeds convergence, but a wrong one can pull the empirical solution toward the analytic shape; set False for a from-scratch photutils build (the clean A/B test for seed bias).
`psf_analytic_components`	`3` (int)	Number of Moffat components the analytic PSF model targets, used as the ePSF seed and residual diagnostic. One component cannot capture both the PSF core and its extended wings; ~3 components (core, mid, halo) fit most seeing profiles. All star cutouts are fitted simultaneously: the component shapes are shared while each star varies only in position and amplitude. The target count is tried first (capped at stars-1) and smaller counts are used only when a fit fails or a component collapses to noise level; when no usable stars exist a canonical width spacing is used instead. Range 1-4.
`psf_analytic_component_offsets`	`True` (bool)	Allow each analytic-PSF component a small shared centre offset (bounded by psf_analytic_offset_max_fwhm x FWHM). Offsets let a sum of symmetric Moffats represent PSF asymmetries - coma, lopsided wings, trailing - that a concentric stack cannot produce. Both variants are fitted and BIC decides; False forces concentric components.
`psf_analytic_center_free_px`	`1.5` (float)	Per-star position freedom in the joint analytic fit (px). Star cutouts are centred on measured centroids that carry ~0.1-1 px error; letting each star shift within this bound stops centroid error smearing the stacked profile.
`psf_analytic_offset_max_fwhm`	`0.5` (float)	Maximum component centre offset in FWHM units for psf_analytic_component_offsets. Bounded so an offset term cannot wander onto a neighbour or duplicate the core at a displaced position.

continued

Key	Default (type)	Description
`psf_analytic_min_snr`	`5.0` (float)	Minimum peak detection significance (peak above sky / sky sigma, both measured inside the star cutout) for a PSF-star cutout to enter the joint analytic fit and its diagnostic plot. Noise clumps and faint contaminants that reach the pool drag the shared profile rather than constrain it. 0 disables the gate.

A.4.14 Spatially varying (gridded) ePSF

An optional spatially varying model: the field is split into detector grid cells, each running the full vet-and-build pipeline, and the per-cell ePSFs are combined into a `GriddedPSFModel`.

```
autophot_input["photometry"]["psf_spatial_grid"] = "auto"  # 3x3 then 2x2, else
    single ePSF
autophot_input["photometry"]["psf_grid_min_stars_per_cell"] = 25
```

Table 21: Photometry: Spatially varying (gridded) ePSF — configuration reference.

Key	Default (type)	Description
`psf_spatial_grid`	`False` (bool|int|str)	Spatially-varying PSF model: build one ePSF per detector grid cell and combine them into a photutils GriddedPSFModel instead of a single field-averaged ePSF. False (default) keeps the single-ePSF build. "auto" tries 3x3 then 2x2 (starved cells are grown into or merged with neighbours first, so the accepted grid may be rectangular, e.g. 2x3), and falls back to the single ePSF when no split gives >= psf_grid_min_stars_per_cell candidates per cell; an int N forces an NxN grid (clamped to 2-3). Safety gates: each cell runs the full ePSF vetting/build pipeline, a cell only counts as populated if >= psf_grid_min_stars_per_cell vetted candidates were assigned AND at least half of them (floor 3) survive the build, and at least half the cells must be real builds - any insufficiency falls back to the single field-averaged ePSF. Disabled automatically for build-from-original (psf_build_from_aligned: false) since grid fiducials would be in the wrong frame after alignment.
`psf_grid_min_stars_per_cell`	`25` (int)	Minimum vetted PSF candidates required in each grid cell for a spatially-varying build. Starved cells are first grown (the shared edge is shifted into the neighbour while the neighbour keeps >= this minimum and a minimum width), then merged with a neighbour by removing the shared edge - the grid may become rectangular (e.g. 2x3). Only cells that still cannot be rescued fall back to psf_spatial_fill_missing.
`psf_spatial_fill_missing`	`nearest` (str)	How to populate under-populated or failed grid cells in a spatially-varying PSF build: "nearest" copies the closest populated cell's ePSF, "fail" falls back to the single field-averaged ePSF.

continued

Key	Default (type)	Description
psf_grid_target_aware	True (bool)	When a spatial grid is built and the target position is known in the build image's detector frame, search non-uniform cell boundaries that place the target near the centre of its own populated cell (a fiducial then sits at the target rather than the target straddling a cell boundary). A candidate layout is only adopted when the target's cell has >= psf_grid_min_stars_per_cell candidates and the normal population rules pass; otherwise the uniform grid (and its fallback behaviour) is unchanged.
psf_grid_max_cells	6 (int)	Maximum number of grid cells in the final spatially-varying PSF model; larger layouts are merged down (whole row/column joins, least worst-cell-count cost first) until they fit. Floored at 4 since a spatial grid smaller than 2x2 is degenerate.
psf_grid_min_samples_per_gridpoint	4.5 (float)	Minimum samples per oversampled gridpoint used when picking the grid's shared oversampling factor (replaces psf_oversample_min_samples_per_pixel for the grid pick). Stricter than the field-wide floor (~3 is the bare minimum for the sigma-clipped median to function): cells hold only tens of stars, and a boundary-count cell otherwise lands one factor too high and collapses (observed: 27-star cell picked x3 at the 3.0 floor and every cell degenerated; the same field at x2 built 5/6 cells).
psf_grid_oversample_survival_frac	0.75 (float)	Fraction of a cell's candidates assumed to survive vetting when picking the shared oversampling factor. The pick uses min_cell_candidates * this so the factor is sized for the survivors, not the input list.
psf_grid_osamp_descent	True (bool)	If too few cells produce usable ePSFs at the picked oversampling, rebuild the whole grid at the next lower factor (down to 1x) rather than falling back to the single ePSF. Stamps stay shape-uniform within each attempt; thinner grids are more stable because every star then constrains every gridpoint.
psf_grid_cell_smoothing_target	15.0 (float)	Target samples per oversampled gridpoint for the thin-sampling smoothing boost in per-cell builds (replaces psf_smoothing_kernel_target_samples). Higher than the field value because ~25-50-star cells are closer to the undersampled regime; stronger smoothing damps the per-gridpoint noise that produces banding in x>=2 stamps.
psf_grid_cell_fwhm_min_frac	0.70 (float)	Lower bound on per-cell ePSF FWHM as a fraction of the image FWHM. Wider than the single-ePSF band (0.85): stamp FWHM measured from ~30-50 noisy star stacks scatters by ~0.1-0.2 px, and genuine spatial PSF variation is what the grid exists to capture - degeneracy is caught by the negfrac/consistency gates instead.
psf_grid_cell_fwhm_max_frac	1.70 (float)	Upper bound on per-cell ePSF FWHM as a fraction of the image FWHM (single-ePSF uses 1.5).
psf_grid_cell_negfrac_max	0.10 (float)	Maximum allowed fraction of negative pixels in a per-cell ePSF stamp. A starved stack can collapse to a sharp core with ringing wings while its measured FWHM stays inside the sanity band; the ringing appears as excess negative flux (healthy cells are typically <5%). Cells over the limit are demoted to unpopulated and filled from a neighbour.

continued

Key	Default (type)	Description
psf_grid_cell_nonfinite_max	0.10 (float)	Maximum allowed fraction of non-finite (NaN/inf) pixels in a per-cell ePSF stamp. GriddedPSFModel rejects any cube containing non-finite values, so a cell over the limit is demoted to unpopulated and filled from a neighbour; a cell under it has the stragglers zeroed at stack time (far-wing fill, same convention as clean_epsf_stamp).
psf_grid_cell_consistency_max	1.6 (float)	Maximum allowed ratio between a cell ePSF and the ensemble median, checked on sharpness (peak/sum - normalisation-invariant) and FWHM. Cells on one field share one PSF to within modest spatial variation; an outlier cell is a failed build that passed the per-cell gates (e.g. a collapsed-sharp stack) and is filled from a neighbour instead. Only applied when >= 3 cells built.
psf_grid_require_target_cell	True (bool)	When the target position is known in the build image's detector frame, require the grid cell containing the target to be a real ePSF build. The grid exists to put a local PSF at the target; if that cell failed its build it would be filled with a copy of a distant cell, giving no locality benefit at the position that matters most (observed: filled target cell gave chi2 4.3 and PSF flux 35% above the aperture flux where the single ePSF gave chi2 2.2 and flux parity). A rejected grid falls back to the single field-wide ePSF; set False to keep the grid anyway (a warning is still logged).

A.4.15 PSF fit and fitters

The fit itself: box sizes and shape priors, the non-negativity bound, and which of the three fitters runs – least-squares, Poisson likelihood, or MCMC below a chosen S/N.

```
autophot_input["photometry"]["use_poisson_likelihood_fitter"] = True
autophot_input["photometry"]["perform_emcee_fitting_s2n"] = 10  # MCMC below this S/N
autophot_input["photometry"]["check_inverted_image"] = True  # fading dips on diffs
```

Table 22: Photometry: PSF fit and fitters — configuration reference.

Key	Default (type)	Description
psf_fit_positive_flux	True (bool)	Bound the fitted PSF flux/amplitude at >= 0 during the fit itself (LSQ parameter bounds, MCMC prior, Poisson step projection). An unconstrained fitter can converge on a negative-flux solution that no later step recovers from; a source on a normal image cannot have negative flux. On difference images the bound pins a faded source at ~0 (a clean non-detection) - enable check_inverted_image to still measure it, or psf_diff_negative_flux to keep signed residuals. Set False to restore unconstrained amplitudes.
psf_diff_negative_flux	False (bool)	Allow signed (negative) fitted amplitudes on difference images (CONVD/FORCECON set). A fading source is legitimately negative on a diff, so True preserves the physical sign directly; the default False keeps the >=0 bound everywhere so primary-fit fluxes are never negative.

continued

Key	Default (type)	Description
psf_fit_boxsize_max_scale_fwhm	4.5 (float)	Undersampled only: cap on the ePSF-build fit box in FWHM units. fit_shape only bounds the per-star flux+shift fit each iteration (the residual stack always uses the full cutout), so an oversized box makes the fit sky-noise dominated – centers wander, the build never converges, and the ePSF can collapse (observed: 7xFWHM box on a 1.8 px PSF produced a 1.5 px ePSF). ~4.5xFWHM keeps the affine fit on the high-S/N core.
psf_fit_shape_bright_scale_fwhm	3.0 (float)	PSF photometry fit window scale in FWHM units (used for ALL sources: target and catalog).
psf_fit_shape_bright_min_arcsec	3.0 (float)	Minimum fit window size in arcsec (converted to pixels via pixel_scale). Used for ALL sources.
psf_fit_sampling_boost_max	1.6 (float)	Maximum adaptive boost for fit-window scales on coarse/undersampled images.
psf_target_fit_shape_boost	1.5 (float)	Multiplier applied to all fit-window scales for target-only PSF fits to better capture wings.
psf_fit_shape_ee_min	0.95 (float)	Grow the PSF fit window until the model encircled energy at the fit radius reaches this fraction (0-1). Truncated wings leak flux into the sky annulus and bias bright sources. Capped at the ePSF stamp size. Set to >=1 or 0 to disable.
psf_localbkg_ee_min	0.97 (float)	For catalog PSF fits, keep the local-background annulus inner radius at or beyond the model radius enclosing this EE fraction, so the median sky does not absorb wing flux (slope != 1 in the zeropoint). Target fits always use the aperture-matched annulus. Set to >=1 or 0 to disable.
psf_fwhm_guard_lo_frac	0.8 (float)	Lower guard for ePSF build FWHM re-estimation: clamped to >= this fraction of runtime FWHM. Prevents excessive FWHM shrinkage from PSF-star selection bias (point sources are naturally sharper than global median including galaxies). 0.8 allows 20% shrinkage.
psf_fwhm_guard_hi_frac	1.3 (float)	Upper guard for ePSF build FWHM re-estimation: clamped to <= this fraction of runtime FWHM. Prevents excessive FWHM inflation from blended/extended contaminants in the PSF star sample. 1.3 allows 30% inflation.
perform_emcee_fitting_s2n	10 (float)	If >0, run emcee PSF/MAG fitting when target PSF S/N is below this value. Uses PSF fitting S/N (not aperture S/N) for the trigger. Set to 0 to disable MCMC entirely.
check_inverted_image	False (bool)	If True, also fit target on image multiplied by -1 to detect negative PSF dips (fading sources in template subtraction).
use_poisson_likelihood_fitter	False (bool)	If True, fit PSFs by maximising the Poisson likelihood instead of least-squares. Best suited to low-count data; requires non-negative counts, so the fitter falls back to LSQ with a warning on difference images or data containing negative pixels.
poisson_maxiters	20 (int)	Maximum iterations for the Poisson likelihood fitter.
poisson_lnl_tolerance	0.0001 (float)	Log-likelihood convergence tolerance for the Poisson fitter.
poisson_max_step_cuts	7 (int)	Maximum step-halving cuts allowed during Poisson fitter line search.
emcee_nwalkers	32 (int)	Number of emcee walkers (minimum 2*ndim recommended; 32 balances quality and speed).
emcee_nsteps	*null* (int|null)	Total emcee steps (per walker). null enables adaptive convergence stopping (runs until autocorrelation time < 50*tau, capped at 50000 steps).

continued

Key	Default (type)	Description
emcee_delta	*null* (float|null)	Proposal scale / step size override (implementation-specific).
emcee_burnin_frac	0.3 (float)	Fraction of samples to discard as burn-in.
emcee_thin	10 (int)	Thinning factor for saved chain samples.
emcee_adaptive_tau_target	100 (int)	Convergence threshold for adaptive stopping: stop when chain length > this factor * autocorrelation time (100 gives a healthy effective sample size; the emcee floor is ~50).
emcee_early_stop	True (bool)	If True, a converged chain may stop before emcee_nsteps is reached (nsteps acts as a cap). Set False to always run the full fixed nsteps.
emcee_min_autocorr_N	100 (int)	Minimum chain length (in steps) before autocorr is estimated.
emcee_batch_steps	100 (int)	Steps per batch when running emcee in batches.
emcee_jitter_scale	0.01 (float)	Small jitter applied to initial walker positions (relative scale).
emcee_threads	1 (int)	Number of parallel threads for emcee (1=serial). Set to 2-4 for faster MCMC on multi-core machines.
emcee_random_state	42 (int|null)	Seed for the PSF MCMC fitter (walker init + moves). null keeps the previous unseeded behaviour; a fixed seed makes PSF fluxes reproducible run-to-run.
emcee_store_samples	False (bool)	If True, store full MCMC chains in fit_info for corner-plot diagnostics (uses ~2MB per source).

A.4.16 Diagnostics

Warning thresholds only – they flag suspicious conditions (astrometric offset, target on a defect) without changing any measurement.

```
autophot_input["photometry"]["wcs_offset_min_snr"] = 5.0
autophot_input["photometry"]["target_defect_warning_fwhm"] = 3.0
```

Table 23: Photometry: Diagnostics — configuration reference.

Key	Default (type)	Description
wcs_offset_min_snr	5.0 (float)	Minimum snr_psf for a catalog source to appear in the WCS vs PSF offset diagnostic. Lower-S/N centroids wander onto noise (held sources pile at the bound edge) and carry no astrometric information.
target_defect_warning_fwhm	3.0 (float)	When the target (or an additional target) sits within this distance (in FWHM units) of hardware_defects_mask pixels, a defect_flag is set on TargetPosition and a warning is logged that the epoch photometry may be biased. Wider than the calibrator cut because the local fit region plus sky annulus extends further than an aperture. Set to 0 to disable.

A.4.17 Parallelism

Per-image thread count for photutils calls; multiply by nCPU (Table 6) and keep the product under the core count.

```
autophot_input["photometry"]["photutils_n_threads"] = 4
```

Table 24: Photometry: Parallelism — configuration reference.

Key	Default (type)	Description
photutils_n_threads	1 (int)	Threads for photutils compiled paths (AperturePhotometry, LocalBackground, Background2D, SourceCatalog, deblend_sources; PSFPhotometry only benefits on free-threaded CPython). 1 = serial.

A.4.18 Other

Keys that fit nowhere else.

```
autophot_input["photometry"]["redo_sources"] = True  # rebuild the source list
```

Table 25: Photometry: Other — configuration reference.

Key	Default (type)	Description
redo_sources	False (bool)	If True, redo source detection/photometry even if intermediate outputs exist.

A.5 World coordinate system

Verify-first astrometry: the existing WCS is validated against Gaia and a full astrometry.net solve – guided by header hints, tweak orders, and CPU budgets – runs only when verification fails or redo_wcs is set.

```
autophot_input["wcs"]["redo_wcs"] = False  # trust the header WCS
autophot_input["wcs"]["verify_input_wcs"] = True
autophot_input["wcs"]["projection_type"] = "SIP"  # or "TPV"
```

Table 26: World coordinate system — configuration reference.

Key	Default (type)	Description
solver	astrometry (str)	WCS solver backend (astrometry or scamp). scamp requires projection_type: TPV; under SIP it is ignored and solve-field runs alone.
profile	default (str)	Named WCS-solver profile/tuning preset.
crowded_auto	True (bool)	Auto-switch to crowded-field WCS settings when heuristics trigger.
crowded_auto_n_sources_min	1500 (int)	Crowded-field trigger: minimum detected sources.
crowded_auto_sources_per_arcmin2_min	200.0 (float)	Crowded-field trigger: sources per arcmin^2 threshold.
crowded_auto_background_rms_min	*null* (float|null)	Crowded-field trigger: minimum background RMS (optional).
redo_wcs	True (bool)	If True, solve/refresh WCS even if a WCS exists in the header.
projection_type	SIP (str)	WCS projection/distortion mode: TPV (solve-field seed + SCAMP refinement) or SIP (solve-field only).
allow_fallback_to_existing_on_solve_fail	False (bool)	If False, require a fresh solve when redo_wcs=True (no silent fallback to original header WCS).
solve_field_exe_loc	solve-field (str)	solve-field executable name/path (defaults to solve-field on PATH).
solve_field_tweak_order	4 (int)	solve-field SIP tweak order to use (single-value mode; ignored if solve_field_tweak_orders is set).

continued

Key	Default (type)	Description
solve_field_tweak_orders	[4, 2, 1, 0] (list[int])	Fallback sequence: try order 4 SIP, then order 2, order 1, and finally order 0 (linear TAN) if too few stars are matched.
solve_field_nsigma	5 (int)	Sigma threshold for source detection in solve-field. Lower (e.g. 3) for sparse fields.
prefer_scamp_tpv_after_solve	True (bool)	projection_type=TPV only: after solve-field succeeds, refine with SCAMP (TPV/PV distortion). False keeps the solve-field TAN-SIP WCS and skips SCAMP; unused under projection_type: SIP (no SCAMP ever runs).
preserve_distortion_on_redo	False (bool)	If True, attempt to preserve existing non-linear distortion keywords (PV/SIP) when redoing WCS.
compare_input_vs_solved_wcs	True (bool)	Log input-vs-solved WCS quality/parameter deltas on solve-field matched stars.
keep_input_wcs_if_solved_not_better	True (bool)	Keep the original input WCS when the fresh solve is not demonstrably better: worse matched-point residuals, or a degraded solve that drops the input's distortion model (e.g. TAN-only result replacing SIP/TPV input when too few stars matched for the tweak order).
trust_input_distortion_model	False (bool)	If True, keep input distortion model (PV/SIP) and solve only linear WCS terms.
fit_wcs_from_points_max_points	1000 (int)	Max matched points to use in the refined SIP fit.
fit_wcs_from_points_max_match_lines	20000 (int)	Max solve-field .match file lines to scan for matched source pairs.
fit_wcs_from_points_max_sep_arcsec	3.0 (float)	Max catalog-vs-detection separation (arcsec) for a matched pair to enter the SIP refit.
solve_field_crpix_center	False (bool)	If True, pass astrometry.net `-crpix-center` (forces CRPIX at image center).
solve_field_no_fits2fits	False (bool)	If True, pass astrometry.net `-no-fits2fits` (skip the per-attempt input-image sanitization rewrite; safe for standard single-HDU FITS inputs).
refine_after_solve	False (bool)	If True, run an additional WCS refinement pass after a successful solve.
solved_scale_tolerance	2.0 (float)	Allowed ratio solved_scale / expected_scale. Reject the solve if outside [1/tol, tol]. Set to 0 to disable scale validation.
solved_position_tolerance_arcsec	*null* (float|null)	Max allowed offset (arcsec) of the solved CRVAL from the position hint actually used (header WCS or target_ra/target_dec). null uses wcs.search_radius. Set to 0 to disable position validation.
solve_field_downsample_max	8 (int)	If solve-field fails at the configured downsample, retry with progressively coarser downsampling (x2) up to this value to rescue large images.
solve_field_use_xylist	True (bool)	Feed detected positions to solve-field as a FITS xylist before image-based attempts; skips solver-side source extraction on sparse fields.
solve_field_xylist_min_sources	8 (int)	Minimum detected sources needed to try the xylist input path.
solve_field_sparse_detect_sigma	2.5 (float)	Relaxed DAO-style detection threshold (sigma) used to build the xylist when the detection stage found fewer than solve_field_xylist_min_sources.
solve_field_sparse_nsigma	3 (int)	–nsigma passed to solve-field when n_detected_sources is 0 (maximally sparse field).
solve_field_downsample_min_pixels	256 (int)	Never retry at a downsample that leaves the smaller image axis below this many pixels.
solve_field_fine_downsample_small_image	1024 (int)	Images whose larger axis is <= this also try finer downsamples (1 up to the configured level) before coarser retries.

continued

Key	Default (type)	Description
refit_wcs_from_points	True (bool)	If True and solve-field matched stars are available, refit the solved WCS linear terms from the matched points.
scamp_exe_loc	*null* (str|null)	Optional explicit path to scamp executable.
sextractor_exe_loc	*null* (str|null)	Optional explicit path to SExtractor executable.
scamp_astref_catalog	GAIA-DR3 (str)	Reference catalog name for SCAMP (if used).
scamp_ref_timeout	120 (int)	Vizier connection timeout (s) for SCAMP REF_TIMEOUT (built-in default 10 is often too low).
scamp_ref_server	vizier.unistra.fr (str)	SCAMP REF_SERVER (Vizier mirror). Override if needed (e.g. vizier.unistra.fr, vizier.ast.cam.ac.uk).
scamp_single_call	True (bool)	If True (default), run SCAMP only once per image (disables multi-trial optimization).
scamp_optimize_after_solve	False (bool)	(Advanced) If True, try multiple SCAMP parameter trials after solve-field.
scamp_optimize_max_trials	1 (int)	Maximum SCAMP optimization trials to evaluate (1=base only).
scamp_distort_degrees	3 (int)	SCAMP distortion polynomial degree (default 3; use 4 only for severe distortion).
scamp_nthreads	4 (int)	Number of SCAMP threads (within one SCAMP run).
sextractor_nthreads	4 (int)	SExtractor NTHREADS when run via run_sex wrapper.
scamp_crossid_radius	2.5 (float)	Cross-identification radius (arcsec) for SCAMP matching.
scamp_position_maxerr	1.0 (float)	Maximum positional error (arcsec) accepted by SCAMP.
scamp_fwhm_thresholds	"1.0,15.0" (str)	SCAMP FWHM acceptance range (pixels), "min,max".
scamp_astref_weight	1 (int)	Weight applied to astrometric reference catalog.
scamp_astrefmag_key	MAG_AUTO (str)	Magnitude column name used for astrometric reference weighting.
scamp_astrefmagerr_key	MAGERR_AUTO (str)	Magnitude-error column name for astrometric reference weighting.
scamp_astrefcent_keys	XWIN_WORLD, YWIN_WORLD (str)	World-coordinate centroid columns used by SCAMP.
scamp_astreferr_keys	ERRA_WORLD, ERRB_WORLD, ERRTHETA_WORLD (str)	World-coordinate centroid error columns used by SCAMP.
scamp_distort_keys	XWIN_IMAGE, YWIN_IMAGE (str)	Image-coordinate columns used by SCAMP distortion fit.
scamp_mosaic_type	UNCHANGED (str)	SCAMP mosaic handling mode.
scamp_stability_type	EXPOSURE (str)	SCAMP stability model type.
scamp_sn_thresholds	"3.0,100000.0" (str)	SCAMP S/N thresholds, "min,max". 2nd value controls high-SN sample for quad formation; 0=auto for both.
scamp_match_nmax	0 (int)	Max detections used for SCAMP pattern matching (quad matching). 0=auto (use all). Lower values speed up matching on dense fields.
scamp_pixscale_maxerr	1.2 (float)	Max scale-factor uncertainty for SCAMP quad matching (default 1.2=20%).
scamp_posangle_maxerr	5.0 (float)	Max position-angle uncertainty (degrees) for SCAMP quad matching.
correct_distortion	False (bool)	If True, undistort each science image via SCAMP+SWarp before source detection.
correct_distortion_resampling_type	LANCZOS3 (str)	SWarp RESAMPLING_TYPE for distortion correction (use LANCZOS2/BILINEAR for undersampled data).

continued

Key	Default (type)	Description
search_radius	0.5 (float)	Search radius (degrees) for blind/assisted WCS solving.
downsample	4 (int)	Downsampling factor used for source detection/ solving speed-up.
objs	1200 (int|null)	Optional target number of objects to use in solve-field.
verify_input_wcs	False (bool)	If True and redo_wcs is False, verify the header WCS by cross-matching detected sources to Gaia; skip solve-field when it verifies.
verify_max_median_arcsec	0.5 (float)	Maximum median detected-vs-Gaia residual (arcsec) for the input WCS to pass verification.
verify_max_p95_arcsec	1.5 (float)	Maximum 95th-percentile detected-vs-Gaia residual (arcsec) for the input WCS to pass verification.
verify_min_matches	8 (int)	Minimum matched source pairs for verification; fewer matches is inconclusive and the solve proceeds.
verify_match_radius_arcsec	2.0 (float)	Cross-match radius (arcsec) between detected sources and Gaia stars projected through the input WCS.
verify_gmag_limit	19.0 (float)	Gaia G-magnitude limit for the verification cone query.
prefer_header_position_hint	True (bool)	If True and the header WCS parses, use its reference position as the solve-field position hint before target_ra/target_dec.
wide_scale_low_arcsec	0.1 (float)	Lower bound (arcsec/pix) of the wide solve-field scale search.
wide_scale_high_arcsec	5.0 (float)	Upper bound (arcsec/pix) of the wide solve-field scale search.
sextractor_detect_thresh	1.5 (float)	SExtractor DETECT_THRESH for WCS source detection.
sextractor_detect_minarea	5 (int)	SExtractor DETECT_MINAREA for WCS source detection.
sextractor_detect_maxarea	0 (int)	SExtractor DETECT_MAXAREA for WCS source detection (0 disables max-area cut).
sextractor_analysis_thresh	1.2 (float)	SExtractor ANALYSIS_THRESH for WCS source detection.
sextractor_deblend_nthresh	32 (int)	SExtractor DEBLEND_NTHRESH for WCS source detection.
sextractor_deblend_mincont	0.005 (float)	SExtractor DEBLEND_MINCONT for WCS source detection.
sextractor_back_type	AUTO (str)	SExtractor BACK_TYPE for WCS source detection.
sextractor_back_value	0.0 (float)	SExtractor BACK_VALUE for WCS source detection.
sextractor_back_size	64 (int)	SExtractor BACK_SIZE for WCS source detection.
sextractor_back_filtersize	3 (int)	SExtractor BACK_FILTERSIZE for WCS source detection.
sextractor_backphoto_type	LOCAL (str)	SExtractor BACKPHOTO_TYPE for WCS source detection.
sextractor_backphoto_thick	24 (int)	SExtractor BACKPHOTO_THICK for WCS source detection.
sextractor_back_filtthresh	0.0 (float)	SExtractor BACK_FILTTHRESH for WCS source detection.
sextractor_phot_apertures	*null* (float|null)	Optional SExtractor PHOT_APERTURES override for WCS source detection; null uses 1.7*kernel FWHM.
sub-section crowded		
objs	800 (int)	Target number of objects in crowded-field solving mode.
sextractor_detect_thresh	2.5 (float)	Crowded-field SExtractor DETECT_THRESH.
sextractor_detect_minarea	8 (int)	Crowded-field SExtractor DETECT_MINAREA.
sextractor_analysis_thresh	2.0 (float)	Crowded-field SExtractor ANALYSIS_THRESH.
sextractor_deblend_nthresh	64 (int)	Crowded-field SExtractor DEBLEND_NTHRESH.
sextractor_deblend_mincont	0.001 (float)	Crowded-field SExtractor DEBLEND_MINCONT.

continued

Key	Default (type)	Description
`cpulimit`	`90` (int)	CPU time limit (seconds) for crowded-field solve attempts.
`search_radius`	`0.5` (float)	Search radius (degrees) for crowded-field solving.
`crpix_offset`	`0.0` (float)	Pixel offset applied to CRPIX when writing WCS (instrument-specific workaround).
`crpix_fits_convention`	`False` (bool)	Legacy toggle: when True and crpix_offset is unset (null), applies a +0.5 px CRPIX offset for solvers using a different pixel convention. Kept for backward compatibility with old configs; prefer setting crpix_offset directly.
`allow_crpix_offset_with_preserved_distortion`	`False` (bool)	Allow crpix_offset when the input distortion model is preserved. Offsets are skipped in that mode because the preserved PV/SIP terms are tied to the original reference pixel; True forces the offset anyway.
`fit_wcs_from_points_sip_degree`	*null* (int|null)	Retained for compatibility: a positive value keeps the solve-field .match/.corr/.axy artifacts on disk (used by the matched-points diagnostics and linear refit); null/0 removes them. The full SIP refit it once selected is no longer available under astropy >= 8.
`cpulimit`	`45` (int)	CPU time limit (seconds) for solve attempts.
`late_cpulimit`	`120` (int)	CPU time limit (seconds) for the later solve retries (wide scale bounds, no-SExtractor pass); they sweep more index tiles than the hinted tries.
`blind_cpulimit`	`300` (int)	CPU time limit (seconds) for the position-free blind solve; a sky-wide search needs minutes, the hinted cap gives it no chance.
`solve_field_total_budget`	`1800` (float|null)	Wall-clock budget (seconds) shared by the whole solve-field cascade (all retries, tweak orders, downsample levels); attempts are skipped once it is spent and SCAMP/header fallbacks still run. null or <=0 disables the cap.
`solve_field_input_wcs_budget`	`60` (float|null)	Same as solve_field_total_budget but applied when the input header already has a usable WCS; on failure the header WCS is kept anyway, so a short budget reaches the same result faster. null or <=0 disables the cap.
`solve_field_parity`	`auto` (auto|pos|neg|null)	–parity hint for solve-field: 'auto' derives sign(det(CD)) from the input header (image handedness rarely changes even when pointing is wrong), 'pos'/'neg' force it, null/anything else omits the flag.
`solve_field_depth`	*null* (int|range-str|null)	Optional –depth value (e.g. 30 or '10-50'); deeper index sweeps help sparse fields at higher CPU cost. null leaves astrometry.net's default.
`solve_field_logodds`	*null* (float|null)	Optional –logodds acceptance threshold; lower values accept weaker matches (guarded downstream by the solved-scale/position checks). null leaves astrometry.net's default.
`solve_field_extra_args`	*null* (list[str]|str|null)	Extra arguments appended verbatim to every solve-field attempt (escape hatch for flags without a dedicated key).
`TNS_BOT_ID`	*null* (str|null)	Optional TNS bot ID (if querying TNS).
`TNS_BOT_NAME`	*null* (str|null)	Optional TNS bot name (if querying TNS).
`TNS_BOT_API`	*null* (str|null)	Optional TNS API key/token (if querying TNS).

A.6 Reference catalogues

Which survey supplies the calibrator sequence and how the field is queried: backend selection (a single survey, a per-band mapping, or a user CSV), query geometry, retries, and published band

transformations for bands the survey does not natively carry.

```
autophot_input["catalog"]["use_catalog"] = "panstarrs"
autophot_input["catalog"]["allow_filter_transformations"] = True
autophot_input["catalog"]["include_IR_sequence_data"] = True  # add 2MASS JHK
```

Table 27: Reference catalogues — configuration reference.

Key	Default (type)	Description
`use_catalog`	*null* (str|dict|null)	Catalog backend for calibration (e.g. `"refcat"`), per-filter mapping (e.g. `{"ugriz":"sdss","UBVRI":"apass","default":"gaia"}`), or `"auto"` to pick the best catalog per band automatically.
`catalog_custom_fpath`	*null* (str|null)	Optional path to a user-supplied catalog file.
`transmission_curve_map`	*null* (dict[str,str]|null)	Optional band->throughput file map for Gaia curve-map custom catalog builds. Example: `{"g":"curves/g.dat","r":"curves/r.dat","i":"curves/i.dat"}`.
`curve_map_svo`	*null* (dict[str,str]|null)	Optional band->SVO filter-ID map to fetch curves. Example: `{"g":"SDSS/SDSS.g","r":"SDSS/SDSS.r","i":"SDSS/SDSS.i"}`.
`catalog_radius`	`0.25` (float)	Search radius (degrees) around target for catalog query.
`catalog_query_radius_arcmin`	*null* (float|null)	Cone radius (arcmin) for per-image catalog downloads. The driver sets this from the footprint union so the cone covers (and is limited to) the detector extent; when unset, main.py sizes it from each image's WCS, or its pixel scale when no WCS exists (clamped to region_min_arcmin-region_max_arcmin, default 10).
`catalog_query_regions`	*null* (dict|null)	Per-catalog query regions set by the driver: {"<catalog>": {"ra","dec","radius_arcmin","box_deg","width_deg","height_deg",...}, "default": {...}}. Each region is centred on the bounding box of the image footprints for the bands that catalog serves (not necessarily the target) and bounded by region_* below; box-capable backends (apass, 2mass, sdss via VizieR) query the rectangle, others the circumscribed cone. main.py resolves the entry for the selected catalog, falling back to "default" then catalog_query_radius_arcmin.
`region_margin`	`1.1` (float)	Padding factor applied to footprint bounding boxes when sizing catalog query regions (1.1 = 10% border beyond the image corners).
`region_min_arcmin`	`2.0` (float)	Lower bound (arcmin) on catalog query regions: the cone never shrinks below this even for tiny fields.
`region_max_arcmin`	`60.0` (float)	Upper bound (arcmin) on catalog query regions: fields larger than this get the truncated cone/box and the coverage diagnostic warns.
`max_distance`	`10` (float)	Max distance (arcmin) from the target a catalog source may sit before clean() drops it. Raised automatically to at least the resolved query radius so a widened cone is not re-trimmed.
`pan_starrs_max_retries`	`3` (int)	Attempts for the MAST Pan-STARRS mean-object API before the query is declared failed (read timeouts are common on wide cones). An exhausted query raises a distinct "query failed" error so it is not conflated with a genuine 0-source (no coverage) result.
`pan_starrs_retry_base_delay_sec`	`5.0` (float)	Initial backoff (seconds) between Pan-STARRS query attempts; doubles each retry.

continued

Key	Default (type)	Description
optimizer_mag_err_max	0.2 (float)	Max catalog magnitude error for a source to count as "usable" in the auto-catalog optimizer (scoring and coverage map). Sources whose band error column is missing/NaN or above this limit are not counted; set <= 0 or null to lift the cap (a finite error is still required where the catalog carries one).
coverage_recenter_min	0.8 (float)	Coverage fraction below which the auto-catalog optimizer re-centres a cone query on covered sky. A survey boundary through the field leaves the footprint-centred query half-empty; distance-ordered (refcat, legacy) and radius-clamped cones then spend their row budget on unobserved sky, so the query is re-issued centred on the covered portion and kept only when the required box gains coverage or rows. The adopted centre propagates into catalog_query_ regions for the per-image downloads. Box backends (apass, 2mass, sdss) cover their region exactly and are never recentred - a boundary cannot gain sources by moving. Set <= 0 or null to disable.
refcat_nsources	*null* (int|null)	Max RefCat2 rows kept per query (spatially subsampled from a wider fetch so coverage stays uniform). When null the cap scales with the query area (~3 sources/arcmin^2, floored at 500, capped at 10000) so the footprint-anchored optimizer regions are not thinned to the old flat 500; the cap rides in the cache filename so older truncated caches are not reused.
blend_neighbor_radius_arcsec	6.0 (float)	Drop catalog sources within this separation of a much brighter catalog neighbour (see blend_neighbor_dmag). Targets spurious faint entries near bright stars (e.g. Pan-STARRS artifacts) and genuinely contaminated faint companions. Set <= 0 to disable.
blend_neighbor_dmag	3.0 (float)	Minimum magnitude difference (image band) that marks the fainter member of a close pair as blend-contaminated. Set <= 0 to disable.
gaia_xp_radius_deg	0.1667 (float)	Search radius (degrees) for Gaia XP query (if used).
gaia_xp_min_sources	25 (int)	Desired minimum XP-spectra sources; the cone grows when a query returns fewer.
gaia_xp_max_radius_deg	1.0 (float)	Hard cap (degrees) on cone growth.
gaia_xp_grow_factor	1.5 (float)	Radius multiplier per retry while under gaia_xp_min_sources.
gaia_xp_max_sources	500 (int)	Max Gaia sources for XP spectra/synthetic photometry (speed vs completeness).
gaia_xp_order_by	brightness (str)	brightness (SQL order) or distance (prefetch + Python sort by on-sky sep; Gaia ADQL has no ORDER BY distance).
gaia_xp_show_progress	False (bool)	tqdm progress for GaiaXPy batched calibrate/generate.
gaia_curve_map_max_sources	100 (int)	Max sources for curve-map Gaia CSV (nearest stars by default).
gaia_curve_map_order_by	distance (str)	distance (Python sort after prefetch) or brightness (SQL order only).
gaia_nearest_prefetch_factor	50 (int)	SQL TOP >= max_sources * factor (capped below) before distance sort.
gaia_nearest_prefetch_min	200 (int)	Minimum prefetch TOP when using distance ordering.
gaia_nearest_prefetch_max	10000 (int)	Maximum prefetch TOP (archive load vs completeness tradeoff).
gaia_archive_query_pause_before_sec	0.25 (float)	Sleep before each Gaia ADQL job (seconds).
gaia_archive_query_pause_after_sec	0.25 (float)	Sleep after each Gaia ADQL job (seconds).

continued

Key	Default (type)	Description
gaia_xp_batch_size	200 (int)	Sources per GaiaXPy calibrate/generate call; use <=0 for one call (heavy). Effectively capped at 50: anonymous GaiaXPy archive requests allow at most 50 source IDs per call, so larger values are split internally.
gaia_xp_batch_pause_sec	0.5 (float)	Sleep between GaiaXPy batches (seconds).
gaia_archive_max_retries	3 (int)	Retries per ADQL batch / GaiaXPy batch on failure.
gaia_archive_retry_base_delay_sec	2.0 (float)	Initial backoff (seconds); doubles each retry.
gaia_xp_generate_error_correction	True (bool)	If True, ask GaiaXPy generate() to apply empirical flux-error corrections (see Montegriffo+ 2022; GaiaXPy generator tutorial).
gaia_xp_generate_truncation	False (bool)	If True, enable GaiaXPy generate(..., truncation=True) (GaiaXPy >= 2.1.4) to reduce high-order basis noise for faint sources.
gaia_xp_photometric_systems	["SDSS_Std", "JKC_Std"] (list[str]|null)	Which GaiaXPy photometric systems to generate.
include_IR_sequence_data	False (bool)	If True, augment catalog with IR sequence data (when available).
allow_filter_transformations	False (bool)	If True, synthesize catalog magnitudes in bands the survey does not measure natively using published transformations (Lupton 2005 and Jester+2005 for sdss; Tonry+2012 for panstarrs; LS DR10 PS1 color terms plus Toptun+2024 fallback for legacy). Lets e.g. Johnson-Cousins BVRI images be calibrated against ugriz/grizy/grz catalogs. Propagated errors include the published fit scatter. Even when False, a missing image-filter band is still synthesized as a last resort so the zeropoint can be computed; a warning is logged in that case.
MASTcasjobs_wsid	*null* (str|null)	Optional MAST CasJobs workspace ID for refcat/casjobs-backed catalog queries.
MASTcasjobs_pwd	*null* (str|null)	Optional MAST CasJobs password/token for refcat/casjobs-backed catalog queries.

A.7 Photometric zero-point

The calibrator vetoes and the fit that turns them into a zero point: magnitude range, saturation and linearity gates, blend and defect rejection, the fit method, and the colour/spatial correction terms.

```
autophot_input["zeropoint"]["fit_method"] = "odr"  # mcmc | odr | ransac
autophot_input["zeropoint"]["faint_mag_limit"] = 21.0
autophot_input["zeropoint"]["min_source_no"] = 5
```

Table 28: Photometric zero-point — configuration reference.

Key	Default (type)	Description
faint_mag_limit	22.0 (float)	Discard catalog calibrators fainter than this in Zeropoint.clean(). The old hard-coded limit of 100 allowed very noisy faint stars that inflated ZP scatter. Increase for deep calibration catalogs.
bright_mag_limit	11.0 (float)	Discard catalog calibrators brighter than this in Zeropoint.clean() (saturated/non-linear regime).
reject_nonlinear_sources	True (bool)	Reject sources near the non-linear/saturation regime during ZP clean.

continued

Key	Default (type)	Description
nonlinear_peak_frac	0.85 (float)	Sources with peak_flux >= this fraction of the saturation level are rejected as non-linear.
saturation_peak_frac	0.99 (float)	Sources with peak_flux >= this fraction of saturation level are flagged as saturated (subset of non-linear reject; kept for diagnostics).
max_flags	2 (int)	Maximum SExtractor FLAGS value allowed for ZP calibrators (0=clean, 1=nearby source, 2=blended, etc.).
fwhm_reject_sigma	3.0 (float)	MAD sigma-clip band on calibrator FWHM in Zeropoint.clean(): rejects extended sources (galaxies, blends) above and CR-like narrow detections below the stellar locus. Set <= 0 to disable.
fwhm_reject_min_sources	4 (int)	Minimum calibrators with valid FWHM before the FWHM clip engages. Below this the median/MAD locus is too unstable to trust.
fwhm_reject_min_keep	3 (int)	Keep-floor for the FWHM clip: skipped if it would leave fewer calibrators than this. The ZP fit needs only a handful of clean points, so a sparse inlier set is still usable.
saturation_check_min_keep	3 (int)	Keep-floor for the detection-S/N gate in check_saturation_range: if removing low-S/N calibrators would leave fewer than this, the highest-S/N sources are kept instead. Prevents the gate from starving the ZP fit on sparse fields.
variable_source_match_radius_pix	5.0 (float)	Matching radius (pixels) for removing known variable sources (from SIMBAD) from the ZP calibration catalog. Sources within this distance of a known variable are excluded from the zeropoint fit.
defect_exclusion_fwhm	2.0 (float)	Calibrators within this distance (in FWHM units) of hardware_defects_mask pixels (trails, bad columns, saturation, MaxiMask classes) get a defect_flag and are rejected from the ZP fit: masked pixels in the aperture/PSF footprint bias measured flux even when the fit converged. The flag is kept on the CALIB output catalog. Set to 0 to disable flagging.
reject_defect_sources	True (bool)	Drop defect_flagged calibrators in Zeropoint.clean() (see defect_exclusion_fwhm). When False the flag is still written to the CALIB catalog but not used as a rejection.
defect_reject_min_keep	3 (int)	Keep-floor for reject_defect_sources: if dropping defect-flagged calibrators would leave fewer than this, the highest-S/N flagged sources are kept instead. Prevents the defect veto from emptying the ZP calibrator pool on sparse fields; rescued sources still produce a ZP flagged unreliable.
clean_min_keep	3 (int)	Keep-floor for the combined quality mask in Zeropoint.clean(): if the union of all cuts would leave fewer than this, the highest-S/N rejected sources with a finite catalog magnitude are rescued (saturated rows stay excluded). Prevents sparse fields from losing all calibrators to a pile-up of soft cuts.
min_source_no	3 (int)	Minimum number of sources required for color-term fitting.
mcmc_n_walkers	32 (int)	Number of emcee walkers for ZP MCMC fit. 32 is sufficient for the 2-parameter (ZP + f_out) model. Increase for better posterior sampling.
mcmc_n_burn	500 (int)	Burn-in steps for ZP MCMC. 500 is sufficient for the simple 2-parameter model.
mcmc_n_steps	1000 (int)	Initial production steps per batch for ZP MCMC. The sampler runs in adaptive batches until convergence.

continued

Key	Default (type)	Description
mcmc_max_steps	10000 (int)	Hard cap on total MCMC production steps. Prevents runaway convergence attempts on pathological data.
mcmc_tau_factor	10.0 (float)	Convergence criterion: stop when chain length > tau_factor * autocorrelation_time. 10 is sufficient for a 2-parameter model (emcee recommends ~10).
fit_method	mcmc (str)	ZP fitting method: "mcmc" (Bayesian, slow), "odr" (orthogonal distance regression, fast), or "ransac" (outlier-tolerant, medium). MCMC is default but ODR is ~50x faster and gives equivalent results for clean calibrator sets.
sigma_clip_max	0.5 (float)	Upper cap on the clip width used by the slope-1 ODR zeropoint fit (sigmaMax-style): an unbounded MAD on contaminated data inflates the 3-sigma acceptance window until no outlier is ever rejected. The clip gate uses min(inlier MAD, this value). Set <= 0 to disable the cap.
fast_path_max_mad	0.3 (float)	Maximum allowed calibrator scatter (mag, MAD of catalog - instrumental delta) for which the ZP fast-path weighted mean may replace the MCMC mixture fit. Above this the data are not clean regardless of the formal per-source errors.
psf_fit_reject_sigma	4.0 (float)	Upper-tail MAD sigma-clip on PSF fit-quality columns (qfit, |cfit|, reduced_chi2) in Zeropoint.clean(): rejects blended, contaminated, or non-converged fits that bias the zeropoint. Set <= 0 to disable.
min_inlier_fraction	0.5 (float)	Reliability gate: if the ZP fit's inlier set is smaller than this fraction of the finite calibrators, the fit did not find a dominant 1:1 locus (mis-stacked image or contaminated catalog) and is flagged unreliable in fit_params + a warning.
max_inlier_mad	0.3 (float)	Reliability gate: inlier delta MAD (mag) above this means no tight 1:1 locus exists; the ZP is flagged unreliable. Larger values tolerate noisier calibrator sets but risk accepting bogus-locus fits.
spatial_gradient_min_mag	0.1 (float)	Reliability gate: minimum ZP residual tilt (mag) across the detector that flags a spatial gradient (signature of mis-stacking or mis-registration) when the fitted slope is >3 sigma significant.
ap_psf_disagree_min_mag	0.05 (float)	Reliability gate: absolute floor (mag) for the AP-vs-PSF zeropoint disagreement flag; the >3-sigma test alone fires on honest ~0.03-0.1 mag method systematics when statistical errors are ~0.01 mag. Set 0 to use sigma alone.
spatial_corr_min_neff_frac	0.25 (float)	Reliability gate: when the spatial block-bootstrap shows calibrator residuals are correlated enough that the effective calibrator count drops below this fraction of the inlier count, the fit is flagged unreliable - calibrators sharing a smooth field-dependent deviation do not provide N independent measurements. 0.25 corresponds to a block-bootstrap SE > 2x the iid SE, roughly the 99% significance region for 4-9 blocks; looser values false-flag clean fields. The correlated error always floors the statistical error when significant (>1.5x), regardless of this flag threshold.
bootstrap_resamples	512 (int)	Bootstrap resamples of the inlier locus used to check the reported ZP error nonparametrically (SE of the bootstrap medians floors the statistical error). Seeded by zeropoint.random_state. Set 0 to disable.

continued

Key	Default (type)	Description
`random_state`	`42` (int)	Seed for the MCMC zeropoint fit's walker initialisation and sampler draws. The same image then yields a bit-identical zeropoint and error across runs; set to null for the old non-deterministic behaviour.
`systematic_err`	`0.02` (float)	Field-level zeropoint systematic (mag) folded in quadrature into the reported zeropoint_error. The fit statistical error (scatter/sqrt(N)) only sees per-source dispersion; calibration systematics that correlate across the field - reference-catalog ties, bandpass mismatch, spatially correlated flat-field residuals - do not average down with N and would otherwise be invisible. Typical single-epoch ground-based value: 0.01-0.03 mag. Set 0 to disable.
sub-section `spatial_zp_surface`		
`enabled`	`False` (bool)	Fit and apply a spatial ZP surface to inlier calibrator residuals. Opt-in: it changes calibrated magnitudes on fields where the residual pattern clears the gates below.
`order`	`1` (int)	Surface polynomial order: 1 = plane (tilt only), 2 = full quadratic. Order 2 needs well-distributed inliers (>=18 recommended) or it overfits noise.
`min_inliers`	`12` (int)	Minimum surviving inlier calibrators for the surface to apply (also requires >=3x the coefficient count). Below this the field-mean ZP stands alone.
`sigma_clip`	`4.0` (float)	Iterative clip width (MAD units) on the weighted surface fit - keeps a few discrepant calibrators from pulling the surface.
`min_improvement_frac`	`0.10` (float)	Minimum fractional reduction in weighted inlier RMS the surface must achieve to be applied. Below this the correction is not worth the extra degrees of freedom.
`min_f_stat`	`4.0` (float)	Minimum F-statistic for the added surface terms (weighted chi2 reduction per coefficient over residual chi2/dof). Guards against fitting noise-level wiggles; ~4 approximates a 2-sigma improvement.
`max_correction_mag`	`0.3` (float)	Maximum |s(x,y)| over the calibrator footprint (mag). Larger fits are rejected: a genuine field correction is rarely more than a few tenths of a magnitude, and a bigger surface signals a broken calibration the reliability gates should catch, not smooth over.
sub-section `magnitude_dependent_zp`		
`enabled`	`True` (bool)	Run the magnitude-dependent ZP trend check.
`n_bins`	`5` (int)	Number of magnitude bins (equal-count). Fewer bins
`min_per_bin`	`3` (int)	Minimum calibrators per bin to use it.
`max_zp_trend_mag`	`0.05` (float)	Maximum allowed |ZP trend| per

A.8 Cosmic rays and defects

Cosmic-ray rejection and defect masking: the LA.Cosmic-style clip or the astroscrappy backend, mask dilation, and the guards that stop a cosmic ray being carved out of the target itself.

```
autophot_input["cosmic_rays"]["remove_cmrays"] = True
autophot_input["cosmic_rays"]["cr_backend"] = "lacosmic"  # ccdproc instead of
    astroscrappy
autophot_input["cosmic_rays"]["cr_sigclip"] = 4.5
```

Table 29: Cosmic rays and defects — configuration reference.

Key	Default (type)	Description
`remove_cmrays`	`False` (bool)	Enable cosmic-ray removal (astroscrappy) in preprocessing. Astroscrappy is the single CR detector: it interpolates CR pixels and its detection mask (plus cr_defect_margin_px) is unioned into the defect mask (an interpolated pixel is not real data). MaxiMask's default classes exclude CR for this reason.
`cr_backend`	*null* (str|null)	CR removal backend: 'astroscrappy' (fast Cython L.A.Cosmic variant), 'lacosmic' (ccdproc.cosmicray_lacosmic), or 'dfcosmic' (Dragonfly's faithful PyTorch port of the original van Dokkum 2001 algorithm - true non-separable median filter, better CR/star-core separation). null selects astroscrappy. dfcosmic is an optional CPU dependency: lazy-imported only on this path; if it is not installed the run logs a warning and falls back to astroscrappy.
`cr_sigclip`	`4.5` (float)	Detection threshold (sigma) for cosmic ray pixels in astroscrappy/LACosmic.
`cr_sigfrac`	`0.3` (float)	Fraction of neighboring pixels required for detection (LACosmic/astroscrappy).
`cr_objlim`	`10.0` (float)	Object detection limit - rejects CRs that look like real objects (higher = more aggressive CR removal, may clip star cores).
`cr_dilate_factor`	`1.0` (float)	Dilation radius as fraction of FWHM for the CR source-exclusion halo. Sets how close a detection may sit to a defect before it is dropped from source lists; does NOT control which pixels are masked as invalid.
`cr_dilate_iterations`	`2` (int)	Dilation iterations for the source-exclusion halo.
`cr_defect_margin_px`	`2.0` (float)	Fixed pixel margin around flagged CR pixels for the defect mask propagated to photometry masks and the subtraction universal mask. Repaired pixels are interpolated, not real data, but the unrepaired neighbourhood is valid flux - keep this small (raw flags are typically 1-3 px).
`cr_protect_target`	`True` (bool)	Protect the configured target (and resolved additional fit targets) from CR flagging/cleaning. An isolated, undersampled point source reads as a CR cluster to astroscrappy; a flag erases real flux via median interpolation and nulls the target in the difference image. Pixels within cr_protect_target_radius_fwhm of each target position are added to the detector input mask so they can never be flagged.
`cr_protect_target_radius_fwhm`	`2.0` (float)	Radius of the protective disk around each target position, in units of the image FWHM. ~2 FWHM covers the full PSF; larger values risk shielding a genuine CR that lands on the source. The effective radius is floored at 3 px (an undersampled <1.5 px FWHM still covers its flaggable core) and capped at 25 px (a runaway FWHM estimate cannot shield a large patch from CR detection).
`cr_defect_target_extent_fwhm`	`2.5` (float)	Compactness limit for the target-rescue pass, in units of the image FWHM (+ the defect margin on the CR-flag path). A defect component containing a target position is dropped only when every one of its pixels lies within this extent of the position - a false flag hugs the source, while a real elongated defect (dead column, satellite trail, saturation streak) merely crosses it and is always kept.

continued

Key	Default (type)	Description
`cr_defect_target_component_max_px`	`150` (int)	Floor of the area cap (pixels) for the target-rescue pass. After CR detection, and again when the defects mask enters the subtraction universal mask, a component containing a target position is dropped when it is within both the extent limit and the area cap AND the science pixel under the target is finite - a suspect flag over real data, not a hole. The effective cap scales up with FWHM as pi*extent^2 since a mis-flagged source footprint grows with the PSF; this value is only the small-FWHM floor.
`cr_plot`	`False` (bool)	Save side-by-side before/after CR removal plot.

A.9 Source detection and FWHM

How the source list is found and measured: the finder and its threshold, the FWHM estimation set, and the shape cuts (elongation, χ^2, segment size) that separate stars from artifacts.

```
autophot_input["source_detection"]["dao_nsigma"] = 4.0
autophot_input["source_detection"]["elongation_max"] = 1.6
autophot_input["source_detection"]["min_segment_npixels"] = 3
```

Table 30: Source detection and FWHM — configuration reference.

Key	Default (type)	Description
`finder`	`dao` (str)	Point-source finder for image-FWHM measurement: 'dao' (DAOStarFinder; Gaussian matched filter on the unsmoothed image, best faint-star sensitivity for well-sampled PSFs) or 'iraf' (IRAFStarFinder; connected-pixel moments on the pre-smoothed image, more tolerant of undersampled/non-Gaussian profiles). Both paths feed the same sharpness/roundness cleaning and per-source Gaussian FWHM fits, so the choice mainly shifts which marginal detections enter the FWHM median.
`dao_nsigma`	`4.0` (float)	Detection significance (sigma) for the DAO finder, measured on the kernel-convolved noise scale. DAOFIND convention is ~4 sigma because convolution turns noise spikes into stellar-looking blobs; matched stars still gain sensitivity (a raw S/N ~5 star passes). Only used when finder='dao'.
`fwhm_est_s2n_min`	`8.0` (float)	Minimum detection S/N for the subset used to estimate the image FWHM. The near-threshold detection population is dominated by sub-stellar junk (noise excursions, compact defects) which drags a flat median low, so the estimate uses only high-S/N, kernel-consistent sources (falls back to all detections if <3 qualify).
`fwhm_est_band_lo`	`0.5` (float)	Lower bound of the FWHM-consistency band (x detection-kernel FWHM) for the FWHM-estimate subset.
`fwhm_est_band_hi`	`2.0` (float)	Upper bound of the FWHM-consistency band (x detection-kernel FWHM) for the FWHM-estimate subset.

continued

Key	Default (type)	Description
`fwhm_abs_floor_px`	`0.8` (float)	Absolute lower bound (pixels) on the per-source fitted FWHM. A detection narrower than ~1 native px cannot be a resolved point source - it is a hot pixel or CR regardless of the population median (the sigma-clips are relative and cannot catch a defect-dominated pool).
`defect_dominance_frac`	`0.4` (float)	Warn when more than this fraction of the surviving FWHM pool sits within 20% of fwhm_abs_floor_px - a sign the "stars" may be compact defects rather than a genuinely small PSF.
`elongation_max`	`1.6` (float)	Maximum axis ratio (a/b, from the anisotropic Gaussian diagnostic fit) for FWHM-pool detections; rejects galaxies, trails, and blends the azimuthally-averaged Moffat FWHM cannot see. Relaxed to 1.3x the pool median when the field PSF itself is elongated.
`elongation_s2n_min`	`5.0` (float)	Minimum detection S/N for the elongation cut; below this the anisotropic fit is noise-dominated and a large axis ratio is not trusted as a rejection reason.
`chi2_red_max`	`5.0` (float)	Maximum reduced chi-square of the per-source 2D Gaussian shape fit; a galaxy profile or blended pair can converge to a finite FWHM with large residuals. NaN (unmeasurable) is kept.
`min_segment_npixels`	`3` (int)	Minimum connected area (pixels) above a 1.5-sigma threshold a detection must sit on; removes isolated single-pixel hot pixels/CR hits that pass the finder shape statistics. 1 disables.
`segment_isolation`	`true` (bool)	Reject detections lying within segment_isolation_min_sep of another deblended segment's centre of mass (catches sources sitting on extended galaxy light, which point-distance crowding cannot see). Disabled on failure or if it would leave <3 detections.
`segment_isolation_min_sep`	`0.0` (float)	Minimum separation (pixels) from another segment's COM for segment_isolation. 0 = auto: max(10, 2 x detection-kernel FWHM).
`scale_multipler`	`3` (float)	FWHM->scale factor (see utils/run_sex.scale_multiplier_from_config).
`scale_min_px`	`11` (int)	Minimum allowed runtime `scale` (pixels) after FWHM-based scaling/clamping.
`scale_max_px`	`300` (int)	Maximum allowed runtime `scale` (pixels) after FWHM-based scaling/clamping.
`use_catalog`	*null* (str|null)	Optional catalog to seed/guide source detection (if supported).

A.10 Limiting magnitudes

Injection/recovery limiting magnitudes: how many artificial sources are drawn and where they are placed, the S/N threshold(s) each limit is reported against, and the completeness-fit machinery that turns recoveries into m_{50}.

```
autophot_input["limiting_magnitude"]["snr_thresholds"] = [3.0, 5.0]
autophot_input["limiting_magnitude"]["inject_final_n_sites"] = 30
autophot_input["limiting_magnitude"]["completeness_solver"] = "bisect"  # or the
    emcee logistic fit
```

Table 31: Limiting magnitudes — configuration reference.

Key	Default (type)	Description
`beta_limit`	`0.5` (float)	Legacy beta diagnostic threshold. Injected-source recovery is now gated by S/N only; beta is computed for diagnostics and completeness plots but does not affect the detection decision.
`detection_limit`	*null* (float|null)	S/N threshold for declaring an injected source recovered in limiting-magnitude experiments. If null (default), falls back to photometry.detection_limit so the reported limit means "the magnitude at which this pipeline flags a detection". Higher values (e.g., 5.0) match the detection thresholds used by ZTF and other automated survey pipelines.
`completeness_target`	`0.5` (float)	Target recovery fraction at the limiting magnitude (Harris 1990 completeness definition; 0.5 = 50% completeness).
`recovery_method`	`auto` (str)	"auto" matches the transient: aperture-only -> AP, PSF path -> PSF. Or set explicitly: PSF, AP, EMCEE/MCMC.
`recovery_use_absolute_snr`	`false` (bool)	If false (default), a source is "recovered" only when the SIGNED S/N >= detection_limit. This is correct for positive-source injection-recovery: a negative flux with high |S/N| is a noise dip, not a detection. Set true only for exotic negative-injection experiments. Independent of recovery_require_positive_flux.
`recovery_require_positive_flux`	`true` (bool)	If true (default and recommended), a source is "recovered" only when the recovered flux is POSITIVE. Genuine detections of positive injected sources are always positive-flux. This should remain true for all normal science and difference-image runs (the injection experiment always injects positive flux). Only set false for exotic negative-source injection experiments.
`recovery_max_flux_ratio`	`0.0` (float)	Optional sanity gate: if > 0, reject trials where |recovered_flux| exceeds this multiple of the injected flux (catches cosmic rays or unmasked sources dominating the aperture). 0 = disabled (default; near the detection limit, flux scatter is large and this would bias results).
`completeness_solver`	`bisect` (str)	How to solve for the limiting magnitude at completeness_target: "bisect" (default; bracket+bisection on recovery fraction), "logistic_mle" (fast deterministic logistic fit over ALL evaluated magnitudes using cached per-site flags – no extra trials; tolerant of non-monotone wiggles), or "logistic_emcee" (emcee posterior on the logistic curve; slowest, gives full posterior error).
`m50_bootstrap_n`	`200` (int)	Site-resample bootstrap repetitions for the limiting-magnitude uncertainty: resample per-site detection flags at each evaluated magnitude and refit the logistic curve. Captures the dominant Monte-Carlo error (which sites were drawn). Runs on cached flags; no extra photometry. 0 = disable.
`recovery_fit_centroid`	`false` (bool)	PSF recovery only: re-fit flux on a small centroid grid around the injected position and keep the max signed S/N. This approximates a blind DETECTION limit rather than forced photometry at the exact known position: it recovers marginally off-peak sources (deeper, like a real finder) but also admits noise-peak detections within the search radius. Default false = forced-photometry/upper-limit semantics.
`recovery_centroid_search_px`	`1.0` (float)	Half-width (pixels) of the centroid search grid when recovery_fit_centroid=true (clamped 0-5).

continued

Key	Default (type)	Description
recovery_centroid_search_step	0.5 (float)	Grid step (pixels) for the centroid search when recovery_fit_centroid=true.
logistic_emcee_nmags	9 (int)	For completeness_solver=logistic_emcee: number of magnitudes to sample around the transition.
logistic_emcee_span_mag	1.5 (float)	For completeness_solver=logistic_emcee: half-span (mag) around the initial m50 guess to sample.
logistic_emcee_nwalkers	32 (int)	For completeness_solver=logistic_emcee: emcee walkers for the logistic fit.
logistic_emcee_nsteps	2000 (int)	For completeness_solver=logistic_emcee: emcee steps for the logistic fit.
logistic_emcee_burnin_frac	0.3 (float)	For completeness_solver=logistic_emcee: burn-in fraction for the logistic fit.
emcee_save_diagnostic_plot	True (bool)	If True and recovery_method is EMCEE/MCMC, save a small emcee diagnostic plot for one representative injected trial near the final limiting magnitude.
initial_guess	*null* (float|null)	Initial instrumental-magnitude seed for injected-limit search. Null enables automatic annulus-based estimation around the target.
inject_source_location	3 (int)	Injection position strategy for limiting-mag trials (pipeline-specific).
inject_min_radius_fwhm	2.0 (float)	Minimum injection radius from the transient (in FWHM units). Prevents injecting on top of the transient.
inject_max_radius_fwhm	6.0 (float)	Maximum injection radius from the transient (in FWHM units) when searching for quiet injection sites.
inject_use_quiet_sites	false (bool)	If false (default), use a spatially uniform farthest-point draw from valid sites for representative limiting magnitude. If true, select only "quiet" (low-|S/N|) sites, which biases the limit spuriously faint by injecting only into the cleanest sky patches.
inject_max_variance_ratio	3.0 (float)	Pixel-statistics site filter: reject sites whose aperture-pixel variance exceeds this multiple of the expected background variance (from RMS map or local annulus MAD). Catches bright-star template subtraction residuals whose noise is elevated even when their mean is near zero. Set to 0 to disable.
inject_max_abs_mean_sigma	2.5 (float)	Pixel-statistics site filter: reject sites whose aperture mean deviates from the local annulus mean by more than this many sigma. Catches positive/negative subtraction pedestals. Set to 0 to disable.
inject_avoid_zero_pixels	true (bool)	Treat exact-zero pixels as invalid during injection-site filtering (common SWarp no-coverage fill). Keep true for coadded/reprojected images.
inject_min_finite_annulus_frac	0.5 (float)	Minimum fraction of annulus pixels that must be finite (and nonzero if inject_avoid_zero_pixels) for a site to be usable for local background/noise estimation. Same 0.5 convention as aperture.py.
inject_min_finite_annulus_pix	10 (int)	Minimum absolute count of valid annulus pixels required for a site to be usable (in addition to inject_min_finite_annulus_frac).
inject_bright_wing_fwhm	3.0 (float)	Bright-neighbour veto: reject injection sites whose aperture edge lies within this many FWHM of a masked source/defect footprint. A bright star's PSF wings extend past its detected footprint; the wing profile is higher inside the aperture than in the background annulus, so local background subtraction leaves a positive residual that biases every recovered flux high at that site. 0 = disable.

continued

Key	Default (type)	Description
`inject_bright_min_cluster_pix`	`4` (int)	Minimum connected size (pixels) of a masked footprint counted as a source for the wing veto. Smaller components are isolated defects; sites already avoid them via the NaN/aperture checks so they should not veto whole neighbourhoods.
`inject_neighbour_peak_sigma`	`6.0` (float)	Compact-neighbour veto: reject sites whose aperture + inject_neighbour_guard_fwhm guard ring contains a pixel exceeding local_annulus_median + this * annulus_sigma. Catches bright compact sources the source mask missed (faint detections excluded from the mask, blend edges). 0 = disable.
`inject_neighbour_guard_fwhm`	`1.0` (float)	Guard-ring width (FWHM) beyond the aperture disk used by the compact-peak veto, so a source sitting just outside the aperture that still leaks flux inside is rejected.
`inject_bright_min_keep`	*null* (int|null)	Starvation floor for the bright-neighbour veto: if it would leave fewer than this many candidates, restore the least-contaminated sites first (null = floor at inject_final_n_sites; 0 = strict veto, allowed to starve the pool).
`initial_guess_n_samples`	`24` (int)	Number of annulus probe positions used to estimate the injected-limit initial instrumental magnitude.
`initial_guess_sigma_mult`	`5.0` (float)	Initial injected-limit guess uses flux ~= (this * sigma_sky * sqrt(Npix)) from annulus probe measurements; converted to instrumental magnitude.
`scale_growth_factor`	`1.5` (float)	If no quiet injection sites are found, multiply runtime `scale` by this factor and retry (up to scale_growth_max_px).
`scale_growth_max_px`	`600` (int)	Maximum runtime `scale` (pixels) allowed during limiting-magnitude quiet-site retries.
`injection_n_sites`	`25` (int)	Legacy: Number of quiet injection sites around the target used for injection/recovery trials. Prefer inject_final_n_sites.
`inject_final_n_sites`	`6` (int)	Target number of final injection sites after filtering and spatial selection (default 6, clamped 6-500). Sites are distributed uniformly around the target via farthest-point sampling.
`inject_candidate_n_sites`	`0` (int)	Candidate pool size before filtering (0 = auto-size to 3x final count, min 150, max 5000). Larger pools improve quiet-site ranking but increase measurement cost.
`injection_jitter_repetitions`	`3` (int)	Sub-pixel jitter repetitions per site per magnitude (more = smoother completeness curve; slower).
`injection_jitter_pix`	`0.5` (float)	Radius (pixels) of the uniform sub-pixel offset applied to jitter repetitions 2..N (rep 1 uses the nominal site position).
`inject_source_poisson_noise`	`true` (bool)	Add source shot noise (Poisson in e-, converted back to ADU) to injected PSF flux so the completeness transition includes bright-source scatter.
`plot_injection_recovery`	`false` (bool)	If true, generate the InjectionRecovery plot showing injected vs recovered magnitudes. If false, skip this plot.
`n_jobs`	`1` (int)	Worker processes for injection/recovery and background-limit trials (per-image). 1 = serial; >1 can speed up Monte Carlo trials.
`snr_thresholds`	`[3.0, 5.0]` (list[float])	S/N thresholds to calculate detection limits at (e.g., [3.0, 5.0]). The code generates lightcurve columns like Limit_3p0S2N, Limit_5p0S2N. Can be customized (e.g., [3, 5, 10]).
`adaptive_snr_selection`	`true` (bool)	Use adaptive S/N threshold selection in plots (use 5sigma limit when source S/N<3).

continued

Key	Default (type)	Description
`report_all_limits`	`true` (bool)	Include all S/N limits in output tables and CALIB files.
`uncal_from_injections`	`True` (bool)	Collect per-trial (true, measured, err) samples during the limiting-magnitude search and compute the uncertainty calibration. True fluxes live on the recovery method's own scale (model amplitude for PSF/EMCEE, aperture rate for AP), so no aperture correction is needed.
`uncal_apply`	`True` (bool)	Apply the calibration factor to the target's reported error columns (flux_err, inst_mag_err, calibrated mag_err quadrature term). False = diagnostics only.
`uncal_min_sources`	`30` (int)	Minimum finite injection trials required before the calibration factor may be applied.
`uncal_factor_clip`	`[0.5, 2.0]` (list[float])	Allowed range for the applied calibration factor; factors outside the range indicate a pathological injection sample and are not applied.
`recovery_calibrate_snr`	`True` (bool)	If true (and uncal_apply is on), re-evaluate the injection detection flags under errors scaled by the empirical calibration factor and re-solve the limiting magnitude. The real detection flag is decided on calibrated S/N, so an uncalibrated gate would report a limit ~2.5*log10(factor) mag deeper than the pipeline's own detection threshold. The pre-calibration limit is kept in result details as inject_lmag_raw; a failed re-solve reverts to the raw gate.

A.11 Host-galaxy modelling

Optional parametric host-galaxy subtraction: an external GALFIT call fits Sérsic (and optional point-source) components around the target before photometry.

```
autophot_input["host_model"]["enabled"] = True
autophot_input["host_model"]["galfit_path"] = "/usr/local/bin/galfit"
autophot_input["host_model"]["n_sersic"] = 1
```

Table 32: Host-galaxy modelling — configuration reference.

Key	Default (type)	Description
`enabled`	`false` (bool)	Master switch for the optional GALFIT host-subtraction stage. Default off: the pipeline is unchanged when the binary is absent.
`galfit_path`	*null* (str|null)	Explicit path to the galfit executable. Null searches PATH, then ~/bin/galfit.
`fit_radius_fwhm`	`10.0` (float)	Radius (FWHM units) around the target for the galfit fit region. The fit runs on a full-frame box of this radius (larger than the pipeline cutout) so the host body is not clipped at the frame edge; only the detected-source component(s) near the target are unmasked, everything else detected stays masked. Should comfortably cover the host plus the measurement annulus.
`n_sersic`	`1` (int)	Number of free Sersic components (1-4). Extra components can absorb a bright off-centre nucleus, bulge, or close companion inside the host footprint; seeded at the brightest reopened host pixels at least 2 FWHM apart.

continued

Key	Default (type)	Description
n_psf	0 (int)	Number of galfit psf (point-source) components (0-3) seeded at the brightest reopened host pixels at least 2 FWHM from the Sersic seeds. Use for unresolved host nuclei or compact neighbours: a Sersic collapses on a point source and crashes galfit, a psf component models it exactly. Requires the PSF stamp (analytic Moffat fallback).
fit_point_source	false (bool)	If true, add a psf component at the target position; its fitted flux is logged (psf_target_mag) and added back to the residual so the pipeline still measures the target. Useful as a galfit cross-check flux.
hold_target_position	true (bool)	With fit_point_source: fix the PSF component position at the catalog target (true, consistent with force_photometry) or let it move (false).
seed_exclude_target_fwhm	1.5 (float)	Radius (FWHM units) around the target excluded when seeding the host centroid, so target flux cannot steer the Sersic centre.
timeout_s	120 (float)	Wall-clock limit for the galfit subprocess.

A.12 Image reprojection

Reprojection settings used whenever one frame is resampled onto another (template to science, diagnostic checks): method, interpolation order, and flux conservation.

```
autophot_input["alignment"]["reproject_method"] = "adaptive"  # exact / adaptive /
    interp
autophot_input["alignment"]["reproject_interp_order"] = "bicubic"
autophot_input["alignment"]["reproject_adaptive_conserve_flux"] = True
```

Table 33: Image reprojection — configuration reference.

Key	Default (type)	Description
reproject_method	exact (str)	Reprojection strategy: "exact" (most accurate), "adaptive" (fast), "interp" (fastest).
reproject_roundtrip_coords	True (bool)	Coordinate round-trip check (matches reproject default; False can mis-register).
reproject_interp_order	bicubic (str|int)	Interpolation order for interp fallback: bicubic (sharp), bilinear (smooth), nearest (fast).
reproject_parallel	True (bool)	Enable parallel reprojection (recommended for exact method on large images).
reproject_adaptive_conserve_flux	False (bool)	Adaptive reprojection: conserve flux by rescaling pixels (can blur slightly when scales match).
reproject_adaptive_center_jacobian	False (bool)	More accurate Jacobian at grid points; auto-enabled for large rotations (>30 deg).
plot_source_check_distortion_vectors	True (bool)	Overlay catalog->detected residual vectors on Sourcecheck plot.
plot_source_check_distortion_grid_map	False (bool)	Overlay gridded distortion-magnitude heatmap (pixels) on Sourcecheck plot.
plot_source_check_distortion_grid_colorbar	True (bool)	Show colorbar for distortion grid-map magnitude overlay.
plot_source_check_max_sep_pix	6.0 (float)	Max nearest-neighbor separation (pixels) for residual-vector pairing.
plot_source_check_max_vectors	300 (int)	Max vectors to draw on Sourcecheck plot (for readability).
plot_source_check_min_vectors	10 (int)	Minimum matched vectors required before drawing overlay.

A.13 Template subtraction

Reference-template discovery, the alignment cascade, flux matching, and the three subtraction backends with their vetoes and quality gates. Disabled by default; enabling the stage is one flag, the rest is tuning.

```
autophot_input["template_subtraction"]["do_subtraction"] = True
autophot_input["template_subtraction"]["method"] = "sfft"  # sfft | hotpants | zogy
    | auto
autophot_input["template_subtraction"]["alignment_method"] = "reproject"
autophot_input["template_subtraction"]["forceconv"] = "REF"
```

Table 34: Template subtraction — configuration reference.

Key	Default (type)	Description
do_subtraction	False (bool)	Enable template subtraction step.
get_simbad_sources	True (bool)	Query SIMBAD for variable/extended sources near the target and store them in input.yaml as variable_sources (used for galaxy masking, other-transient photometry, and plot context). The query is timeout-bounded and degrades gracefully; set False to skip it entirely (e.g. offline runs or SIMBAD outages).
alignment_method	spalipy (str)	Science-template registration method. Options: spalipy=spline-warp with RA/DEC source pre-matching [recommended, most reliable], swarp=SCAMP+SWarp [best subpixel], reproject=WCS-only, astroalign=source matching, tweakwcs=STScI WCS tweaking+reproject [requires: pip install tweakwcs], chi2_shift=cross-correlation for extended-source fields [requires: pip install image-registration]. Each method falls back through the remaining source-based methods and finally to chi2_shift (translation-only) if they fail.
alignment_use_extended_sources	True (bool)	If True, allow large/extended sources (galaxies) in SCAMP matching catalogs. Extended sources provide valuable alignment anchors in sparse fields.
alignment_max_fwhm_pix	50.0 (float)	Max FWHM (pixels) kept when alignment_use_extended_sources=False.
alignment_min_matched_sources	6 (int)	Minimum matched sources for a reliable SCAMP WCS solution. Below this, retry with larger radius or fall back to reproject/AstroAlign.
alignment_max_match_radius_arcsec	30.0 (float)	Maximum cross-match radius (arcsec) when retrying with fewer sources. Prevents false matches at extreme radii.
alignment_max_scamp_residual_arcsec	2.0 (float)	SCAMP astrometric residual RMS (arcsec) above which a warning is logged. SCAMP+SWarp still proceeds; post-SWarp verification gate makes the final accept/reject decision.
alignment_max_offset_px	0.5 (float)	Maximum allowed systematic post-SWarp centroid offset (pixels).
alignment_max_rms_px	0.75 (float)	Maximum post-SWarp centroid RMS (pixels) when enough matches are available.
alignment_max_p90_px	1.5 (float)	Maximum post-alignment centroid 90th-percentile offset (pixels) when enough matches are available. (Formerly misnamed alignment_max_p95_px; the old key is still read with a deprecation warning.)
alignment_max_local_offset_px	0.5	
alignment_min_sources_for_field_gate	20 (int)	Minimum post-alignment source matches required before RMS and P90 gates are applied. P90 is unreliable with fewer than 20 samples.

continued

Key	Default (type)	Description
alignment_min_overlap_fraction	0.25 (float)	Minimum aligned-footprint coverage accepted from reprojection. Measured over the whole science frame; when below the floor the gate re-checks coverage inside a target-centred window (capped to the template footprint extent), so a survey cutout smaller than the science FOV still subtracts where it matters.
alignment_min_finite_fraction	0.25 (float)	Minimum finite-pixel fraction an aligned template must retain (rejects mostly-empty resampled products). When the template footprint can only cover less of the science frame than this floor (e.g. a PS1 cutout stamp), the gate instead requires >=80% of that attainable coverage.
alignment_min_valid_pixels	1000 (int)	Minimum finite pixel count for an aligned template to be usable.
alignment_min_sources_translation	3 (int)	Matched sources needed before a residual check counts as translation_verified.
alignment_min_sources_affine	6 (int)	Matched sources needed for affine_verified.
alignment_min_sources_spatial	20 (int)	Matched sources needed for spatially_verified (spline/high-order warps).
alignment_require_post_swarp_verification	True (bool)	Reject common-grid SWarp output when subpixel centroid alignment cannot be measured.
alignment_verify_independent	False (bool)	If True, run the independent post-alignment verifier (utils/alignment_verification.py: WCS consistency, pixel-grid alignment, coordinate accuracy and resampling quality) on each successful aligned product. Its 0-1 score and quality label are stamped as ALIGSCOR/ALIGLABL and recorded in the alignment report. For WCS-derived products (swarp/reproject/tweakwcs) a "poor"/"failed" score demotes a "pass" alignment to "warn"; for source-matched backends the score is recorded but never downgrades (its WCS checks are not authoritative when the template WCS was not resampled).
alignment_allow_non_scamp_fallback	False	
post_swarp_verify	True (bool)	If True, run SExtractor cross-match on SWarp output images to measure post-alignment offsets. When False, SCAMP+SWarp is trusted without independent verification. Enable for diagnostics or when alignment quality is uncertain.
alignment_sparse_field_retry_count	15 (int)	If SExtractor detects fewer than this many sources in either image, re-run with maximum sensitivity (DETECT_THRESH=0.5, smaller BACK_SIZE, CLEAN=N). Set to 0 to disable sparse-field retry.
spalipy_match_radius_arcsec	*null* (float|null)	RA/DEC cross-match radius (arcsec) for source pre-matching before spalipy quad matching. null=adaptive (3x FWHM, floored at 10").
spalipy_detect_thresh	2.0 (float)	SExtractor detection threshold (sigma) for spalipy source detection. Detections always use ellipticity<0.9 so extended sources serve as alignment anchors; when the pool has fewer than spalipy_detect_min_sources sources a second pass at thresh*0.5 (floor 0.5) is run and merged into the pool. Deep-pass detections are tagged (det_tier=1) as lower-confidence candidates.
spalipy_detect_min_sources	20 (int)	Detection-pool size below which the deeper second SExtractor pass is run for spalipy alignment.
spalipy_deep_match_frac_warn	0.5 (float)	When more than this fraction of the matched source pool comes from the deep low-threshold pass, a "pass" alignment is demoted to "warn": low-significance anchors are weaker evidence than their count suggests.

continued

Key	Default (type)	Description
`spalipy_n_quad_det`	*null* (int|null)	Number of detections used for quad matching. null=auto (all sources if <=25, else 25).
`spalipy_min_n_match`	*null* (int|null)	Minimum matched sources for spalipy alignment. null=auto (max(6, min(n//3, 20))).
`spalipy_max_quad_hash_dist`	*null* (float|null)	Max hash distance for quad matching. null=auto (scaled by FWHM).
`spalipy_min_sep`	*null* (float|null)	Minimum detection separation (pixels). null=auto (max(FWHM, 2*max_match_dist)). Removes crowded/blended sources.
`spalipy_interp_order`	*null* (int|null)	Spline interpolation order (1-3). null=auto (2 for undersampled, 3 otherwise).
`spalipy_sub_tile`	*null* (int|null)	Sub-tiling factor for affine fitting. null=auto (2 for >=200 sources, 1 otherwise).
`spalipy_spline_order`	*null* (int|null)	Spline order for distortion correction (0-3). null=auto (based on source count).
`spalipy_max_match_dist`	*null* (float|null)	Max matching distance in template pixel frame after affine transform. null=auto (clip(0.5*FWHM, 2, 5) px).
`spalipy_min_quad_sep`	*null* (float|null)	Minimum distance between detections in a quad (pixels). null=auto (clip(min_img_dim/4, 10, 50)).
`spalipy_quad_edge_buffer`	*null* (int|null)	Exclude detections within N pixels of template edge from quad construction. null=auto (FWHM if partial overlap, 0 otherwise).
`spalipy_max_quad_cand`	*null* (int|null)	Maximum quad candidates tried for the affine transform. null=auto (20).
`spalipy_patience_quad_cand`	*null* (int|null)	Stop the affine-seed search after this many consecutive non-improving quad candidates. null=auto (4; spalipy default 2 can abort before ambiguous hash-order lists are explored).
`spalipy_pair_match_dist_cap`	`30.0` (float)	Upper bound (px) on spalipy_max_match_dist when the detection lists are mutual RA/DEC pairs. With a trusted correspondence the inlier radius is sized to 1.5x the measured pair-residual P90 (floored at 0.5xFWHM) so real WCS/SIP disagreement does not veto every affine seed; false matches cannot sneak in because the pairing is already fixed.
`spalipy_pair_min_match_frac`	`0.7` (float)	Fraction of the matched-pair pool spalipy must reproduce as inliers when the lists come from the RA/DEC mutual match (floor 4). Lets one distorted anchor fail without vetoing the seed; ignored for unmatched lists.
`spalipy_use_matched_pairs`	`true` (bool)	When mutual RA/DEC pairs are available, fit the spalipy affine/spline directly to that one-to-one correspondence instead of letting proximity rematching replace it; quad matching remains the fallback for unpaired lists.
`spalipy_pair_clip_px`	*null* (float|null)	Maximum residual under the paired affine transform for a mutual RA/DEC pair to remain an alignment anchor. null=auto (max(1 px, median FWHM)).
`spalipy_quad_target_frac`	`0.35` (float)	Total fraction of the quad-anchor pool reserved for sources near requested targets when no paired list is available; the quota is split across targets and remaining anchors are chosen globally by flux. Set 0 to disable target-aware quad ranking.
`spalipy_quad_target_exclude_fwhm`	`2.0` (float)	Radius around the transient excluded from target-near quad anchor selection, in units of median detection FWHM, so the variable source itself cannot become an alignment anchor.

continued

Key	Default (type)	Description
alignment_prioritize_target	True (bool)	When True, source-based alignment methods (spalipy, tweakwcs, scamp+swarp) prefer sources closest to the target position for the match pool. With few sources this prevents a distant anchor from dominating the fit and degrading local alignment at the transient. Set False to use all detected sources regardless of distance.
alignment_target_radius_px	*null* (float|null)	Radius (pixels) around the target within which sources are preferred for alignment matching. null = auto (max(3*FWHM, min(frame_dim)/4)). Sources outside this radius are still eligible if the pool inside is too small; they are simply ranked after target-near sources.
alignment_target_verify	true (bool)	After a source-matched alignment, measure post-fit residuals around every requested target and apply target-region quality gates when enough anchors exist. A thin local sample is recorded as unverified rather than failed.
alignment_target_verify_radius_fwhm	10.0 (float)	Radius, in units of science FWHM, used to select matched anchors for target-region alignment residuals.
alignment_target_min_anchors	4 (int)	Minimum matched anchors inside the target verification radius before the target-region gate can demote or reject an alignment.
alignment_target_max_offset_px	*null* (float|null)	Maximum target-region median offset (px). null uses the FWHM-scaled global alignment_max_offset_px gate.
alignment_target_max_rms_px	*null* (float|null)	Maximum target-region RMS (px). null uses the FWHM-scaled global alignment_max_rms_px gate.
alignment_target_max_p90_px	*null* (float|null)	Maximum target-region P90 residual (px). null uses the FWHM-scaled global alignment_max_p90_px gate.
alignment_sparse_field_threshold	12 (int)	Source count below which sparse-field mode activates: lower min_match, wider detection ladder, relaxed spline-order requirements, and target-weighted source selection take effect. Below this count the pipeline cannot verify field-wide rotation/scale/distortion and the alignment tier is demoted accordingly.
tweakwcs_searchrad_arcsec	15.0 (float)	tweakwcs cross-match search radius (arcsec). Widened in sparse-field mode (to max(this, 3*FWHM*scale)) so poorly plate-solved fields still find matches.
tweakwcs_separation_px	3.0 (float)	tweakwcs minimum source separation (pixels) for matching. Lowered in sparse-field mode so close pairs are not rejected.
tweakwcs_tolerance_px	2.5 (float)	tweakwcs matching tolerance (pixels). Relaxed in sparse-field mode to tolerate subpixel centroid noise.
reproject_target_validate	True (bool)	For WCS-based reproject alignment, validate that the WCS agreement is good near the target position (not just globally). Rejects reproject when the target-region offset exceeds alignment_max_offset_px, preventing subtraction on a globally-good but locally-wrong WCS.
reproject_target_radius_px	*null* (float|null)	Radius (pixels) around the target for reproject target-region validation. null = auto (max(3*FWHM, min(frame_dim)/4)).
resample_mode	common_grid (str)	SWarp resampling mode: "common_grid" (both to science grid), "native_scale" (each keeps own pixel scale), "wcs_only" (no resampling).

continued

Key	Default (type)	Description
`science_first_astrometry`	`False` (bool)	If True, distortion-correct the science image against GAIA-DR3 first via SCAMP+SWarp, then use the corrected catalog as astrometric reference for the reference image. Disabled: science-first astrometry adds an extra SCAMP+SWarp step that can introduce overfitting on sparse fields. Direct reference-to-science alignment is more reliable.
`gaia_cache_dir`	*null* (str|null)	Directory to cache downloaded GAIA-DR3 reference catalogs for SCAMP. If null, uses ~/.autophot/scamp_gaia_cache. Caching avoids repeated Vizier downloads for the same field.
`gaia_cache_ttl_days`	`30` (int)	Time-to-live for cached GAIA catalogs (days). GAIA-DR3 is static, so long TTLs are safe.
`preserve_scamp_logs`	`False` (bool)	If True, keep SCAMP log files on failure for debugging. Useful when diagnosing astrometric alignment issues.
`method`	`sfft` (str)	Subtraction backend (sfft default; hotpants or zogy optional where available; auto selects from image quality). "auto" picks ZOGY for well-sampled frames (broad FWHM >= 2.5 px) with PSF stamps available when the science/template PSF is matched (FWHM ratio < 1.5) OR when the vetted matching-source count cannot constrain SFFT's delta-basis kernel ((2*hw+1)^2 pixels > sfft_kernel_pix_per_source per source – sparse fields leave the SFFT solve noise-dominated with broken flux scaling, while ZOGY's PSF-derived kernel needs almost no sources), and SFFT otherwise. The in-subtract cascade (zogy -> sfft -> hotpants) still falls through on failure. The ZOGY backend is a self-contained numpy implementation (no external package needed).
`sky_subtract`	`True` (bool)	Subtract the sigma-clipped median sky from both images before subtraction. SFFT ONLY: the sky-subtracted copies are written to temporary prepared FITS files inside the SFFT branch, so a SFFT->HOTPANTS fallback always retries on the original images (HOTPANTS models the background itself via -bgo; ZOGY subtracts its own median internally). Without this the sky pedestal creates step edges at SFFT mask boundaries that contaminate the kernel solution. Invalid/sentinel pixels are preserved as NaN. Backward compat: sfft_sky_subtract and zogy_sky_subtract are still read as fallbacks.
`common_grid_tol_px`	`0.5` (float)	Maximum allowed science-vs-template WCS disagreement (pixels, probed at frame corners and centre) before subtraction refuses to run. The backends subtract pixel-by-pixel, so both inputs must share a common pixel grid after alignment.
`zogy_per_pixel_noise`	`True` (bool)	Compute per-pixel noise maps for ZOGY variance estimation. When True, estimates local noise on a coarse grid (zogy_noise_map_box_size) and upsamples. Captures spatially varying noise (chip gaps, bright galaxy backgrounds). A supplied pipeline noise map (the folded <base>.weight.fits product, when present and grid-valid) always takes precedence over this internal estimate. NOTE: the maps feed only the S/Scorr detection maps (ZOGY Eqs. 25-31) – the difference image D itself always uses the scalar noise estimate, so this option has no effect unless zogy_write_score_images is also enabled.

continued

Key	Default (type)	Description
zogy_noise_map_box_size	64 (int)	Tile size (pixels) for local noise estimation when zogy_per_pixel_noise=True. Smaller boxes capture finer spatial variations but are noisier; larger boxes are smoother but may miss small-scale structure. Values <16 are rejected and 64 is used instead.
zogy_write_score_images	True (bool)	If True, write the ZOGY score image S and corrected significance Scorr (the matched-filter detection maps, ZOGY Eqs. 2 and 13) as zogy_score_<diff> and zogy_scorr_<diff> FITS files next to the difference image, with their paths recorded in the DIFFS/DIFFSCOR header cards. This is the only way zogy_per_pixel_noise becomes visible in pipeline outputs.
zogy_nan_fill	median (str)	NaN fill method for ZOGY: "median" (fill with image median, recommended) or "zero" (fill with 0). Zero-filling creates step functions at NaN boundaries causing FFT ringing; median-filling provides smoother transitions.
zogy_lsq_kernel	True (bool)	If True (default), probe a Bramich-style pixel-domain kernel alongside the Wiener PSF-ratio kernel for ZOGY pre-convolution: every kernel pixel is a free parameter solved by least squares on stamps around the (variable/target/mask-filtered) matched sources, with a Laplacian Tikhonov penalty damping unconstrained high-frequency noise (kernel centre exempt; Hu & Wang 2024, Becker et al. 2012). The pixel kernel absorbs sub-pixel misregistration that a PSF-ratio kernel cannot represent, and its raw kernel sum is a free photometric factor that is folded into the effective flux scale (SFFT ConstPhotRatio equivalent). The LSQ kernel is preferred when its sidelobe mass stays under zogy_lsq_neg_cap and the Wiener probe does not flag the direction as deconvolution; a single MAD-clipped re-solve pass drops stamps with outlier residuals. Sanity gates (finite, kernel sum in [0.25,4], enough stamps/pixels, residual RMS under zogy_lsq_resid_cap) fall back to the Wiener kernel when the solve is unusable.
zogy_lsq_lambda	1.0e-3 (float)	Relative Tikhonov strength for zogy_lsq_kernel: the absolute lambda is lam_rel * mean(diag) of the kernel normal-equation block. Larger values oversmooth the kernel (monopole residuals); smaller values leave unconstrained pixel noise in the wings.
zogy_lsq_min_sources	3 (int)	Minimum number of source stamps for the LSQ kernel solve; below this the Wiener kernel is used.
zogy_lsq_kernel_hw_max	12 (int)	Cap on the LSQ kernel stencil half-width (pixels). Larger stencils add unknowns quadratically and are rarely needed for PSFs under ~6 px FWHM.
zogy_lsq_neg_cap	0.45 (float)	Maximum negative-mass fraction for the LSQ kernel to be preferred over the Wiener candidate. Legitimate shift-absorbing smoothing kernels carry ~0.05-0.3 sidelobe mass; a sharpening fit measures ~0.9 or worse on real data. The fixed-eps Wiener probe gates the choice independently: LSQ is never applied for a direction whose PSF-ratio probe exceeds the hard deconvolution limit.
zogy_lsq_resid_cap	0.5 (float)	Maximum median stamp-residual RMS (in units of the target-image std) for the LSQ kernel to be applied; a poorly-fitting solve falls back to the Wiener kernel.

continued

Key	Default (type)	Description
`zogy_lsq_flip_neg_cap`	*null* (float or null)	Negative-mass ceiling for an LSQ kernel adopted through a veto override (the configured direction was ringy, the opposite direction is adopted instead). null (default) inherits zogy_lsq_neg_cap: the residual-ratio and width-fidelity gates bound the sombrero risk, so an override candidate is held to the same side-lobe budget it would need as the configured direction. Set lower to re-impose the pre-gate 0.30 ceiling.
`zogy_lsq_ksum_tol`	`0.03` (float)	Kernel-sum noise tolerance. The LSQ solve leaves the kernel sum free as a photometric factor, but on sparse fields it wanders a few percent solve-to-solve on identical data; folding that wander into the applied flux scale over/under-subtracts every bright source. Deviations within this tolerance are treated as solve noise: the kernel is renormalized to unit sum and the measured aperture/exposure flux scale stands. Larger deviations fold into the effective scale as a real photometric correction (and warn past 15%).
`zogy_residual_warp`	`true` (bool)	Residual registration refinement for ZOGY. The global spalipy alignment leaves a smooth ~0.2-0.4 px spatially-varying displacement field – the dipole residuals at bright stars that an order-0 matching kernel cannot represent. When enabled, per-anchor lags are measured on the vetted stamp-pool positions and a degree-1 polynomial displacement field is fit (MAD-clipped) and applied to the flux-scaled reference before stamp/kernel work. Skipped automatically when too few anchors survive, the raw lag RMS is already small, or the field fit does not beat the anchor scatter.
`zogy_warp_min_sources`	`6` (int)	Minimum vetted anchors with a measurable sub-pixel lag for zogy_residual_warp; below this the field fit is untrustworthy and the warp is skipped.
`zogy_warp_min_rms`	`0.08` (float)	Minimum raw anchor-lag RMS (pixels) for zogy_residual_warp; smaller residuals gain nothing and the warp is skipped.
`zogy_astrometric_noise`	`true` (bool)	Propagate the measured residual registration error into the Scorr variance (ZOGY 2016 section III.3, Eq. 13). Misregistration leaves residuals proportional to the local image gradient, so a sigma_ast^2 * |grad I|^2 term is added to each image's variance before the matching-kernel convolution. The extra weight lands on bright-source edges where dipoles live, suppressing them as false positives in the zogy_scorr_* detection map while leaving blank-sky significance untouched. sigma_ast per axis comes from the residual-warp anchor lags (post-fit residuals when the warp applied, raw lag scatter otherwise), falling back to the ALIGRMS alignment card, then to zogy_astrometric_noise_floor_px. Per-axis values are stamped as ASTN_X/ASTN_Y. Set False to restore the old constant-variance Scorr.
`zogy_astrometric_noise_floor_px`	`0.02` (float)	Floor for the Scorr astrometric-noise sigma (pixels). Even perfect registration leaves ~10-30 milli-pixel astrometric-scintillation residuals at bright sources (ZOGY 2016 section VIII.5), so sigma_ast is never allowed to reach zero while the term is enabled.

continued

Key	Default (type)	Description
`zogy_astrometric_shift`	`true` (bool)	ZOGY residual astrometric correction (Zackay, Ofek & Gal-Yam 2016 section III.4). A bulk translation left by the global registration is measured by masked phase correlation (upsampled-DFT refinement, ~0.01 px resolution) on the cleaned science/reference pair and applied to the reference by an exact Fourier shift before the kernel solve and subtraction. Without it the matching kernel absorbs the shift, coupling it into the flux-scale gates and kernel-noise audit. Complements zogy_residual_warp: the warp removes the spatially-varying field, this removes the remaining global translation.
`zogy_shift_max_px`	`3.0` (float)	Maximum measured translation accepted by zogy_astrometric_shift (pixels). A larger residual means the global alignment failed rather than left a fine offset, so the raw pair is kept.
`zogy_wiener_adaptive_eps`	`True` (bool)	If True (default), build the Wiener PSF-ratio kernel at the weakest regularization that keeps negative mass below the deconvolution tolerance, stepping eps over 1e-3..1.0; if False, use the fixed eps=1e-2. Adaptive damping rescues borderline broadening directions from the kernel veto without over-smoothing clean directions. Candidates are also screened by zogy_kernel_fidelity_tol, so the ladder cannot climb to a damping level that no longer reproduces the target PSF.
`zogy_kernel_fidelity_tol`	`0.15` (float)	Maximum fractional FWHM error between the Wiener kernel's realised convolution (source PSF convolved with the windowed kernel) and the target PSF. Damping strong enough to clean a mismatched PSF pair (eps -> 1, the correlation-kernel limit) overbroadens the result by ~20% while looking clean on negative mass alone; a kernel that misses the target width leaves positive-core/negative-ring residuals at every source and is refused so the direction falls back to the LSQ candidate or canonical ZOGY.
`zogy_kernel_applied_neg_cap`	`0.30` (float)	Maximum negative-mass fraction within the applied (windowed) Wiener kernel for directions that require real broadening (stamp FWHMs differing by more than sfft_forceconv_auto_tol). A faithful kernel can still carry deep negative near-field lobes when the PSF mismatch demands genuine wing deconvolution; applied, that rings every bright source, so the direction falls back to the LSQ candidate or canonical ZOGY instead. The cap is waived for width-matched stamp pairs, where the kernel does no real convolution work and its sidelobe mass is regularisation ripple; that case is guarded by the direction-level ringy-kernel veto instead.
`zogy_template_psf_independent`	`True` (bool)	If True (default), select PSF stars for the reference image independently on the reference image itself (best-quality reference PSF; falls back to the science-matched stars when fewer than 5 isolated stars are found). If False, reuse the stars matched between science and template for both PSFs.

continued

Key	Default (type)	Description
`zogy_min_psf_stars`	`3` (int)	Minimum stars for the dedicated ZOGY PSF builds. The template PSF prefers >=5 independently selected reference stars, then >=zogy_min_psf_stars matched stars, then >=zogy_min_psf_stars isolated reference stars as a last resort. For the science PSF, fewer than 5 matched sources means the photometry ePSF written earlier (built from the full vetted pool) is reused; a >=zogy_min_psf_stars matched-star build runs only when no photometry ePSF file exists. This floor exists so a sparse matched pool cannot disable ZOGY entirely.
`zogy_psf`	`auto` (str)	PSF source for the ZOGY matched filter: "auto" (default) reuses the photometry ePSF stamp for science when one exists on disk (a matched-subset rebuild cannot improve a vetted model and is only built as a last resort when no ePSF file exists), and falls back to a fitted composite-Moffat stamp only when no empirical model materialises; "analytic" skips the ePSF builds entirely and always uses the fitted analytic composite-Moffat PSF on both sides (ZOGY_PSF_<sci|template>_<base>.fits, fitted to each image's own star pool; for science the photometry analytic model is serialized verbatim when it exists). ZOGY only needs a plausible normalised PSF for its Fourier matched filter and pre-convolution probes, so the smooth noise-free analytic model is a sound choice on fields where the ePSF builds do not converge. In "auto" the analytic fallback stamps are also fitted to the image's star cutouts rather than the canonical profile. Whichever stamp is delivered, the subtraction still validates it against the measured star stack: an analytic stamp loses to a vetted stack of >=zogy_min_psf_stars stars, while an empirical (PSFBUILD=epsf) stamp is kept unless its width disagrees with the stack by >15%.

continued

Key	Default (type)	Description
`forceconv`	`REF` (str)	Convolution direction for SFFT, ZOGY, and HOTPANTS: "REF" (default, convolve reference to science PSF – difference keeps the native science PSF so the science ePSF applies directly and no PSF-shape correction can bias the measured flux; Bramich 2008, Hu et al. 2022), "SCI" (convolve science to reference PSF – difference has the reference PSF; photometry uses the DIFFPSF stamp or convolves the science ePSF with the solved kernel via SOLPATH), "AUTO" (resolved by the pipeline from the measured post-alignment FWHMs – convolve the sharper image up to the broader PSF when the FWHM difference exceeds sfft_forceconv_auto_tol, otherwise keep REF; AUTO is never passed to SFFT itself, whose internal header-FWHM AUTO can be flipped by SWarp resampling, see BUG 122). Backward compat: sfft_forceconv and zogy_forceconv are still read as fallbacks. WARNING: an explicit REF when the template is *broader* than the science image (the common case for stacked templates) requires deconvolution – a ringy kernel with strong negative sidelobes that leaves dipole residuals at bright sources and amplifies noise. For ZOGY such a direction is vetoed rather than applied: the pipeline probes both candidate Wiener kernels (built from wing-tapered stamps, windowed to compact support) and flips to the cleaner direction with a warning, or skips pre-convolution entirely (canonical ZOGY, geometric-mean diff PSF) when neither direction is clean. For SFFT the measured-FWHM veto flips the impossible direction by default (sfft_veto_deconvolution). The veto only fires in that impossible case; well-posed REF requests are always honoured.
`sfft_forceconv_auto_tol`	`0.05` (float)	Fractional FWHM tolerance for forceconv=AUTO and for the deconvolution warning (default 0.05 = 5%). AUTO switches direction only when one FWHM exceeds the other by more than this fraction (FWHM measurement noise floor); within tolerance REF is kept.
`sfft_veto_deconvolution`	`True` (bool)	If True (default), an explicit SFFT direction that must sharpen the convolved image is vetoed to the opposite broadening direction, matching the ZOGY kernel-probe guard. This prevents negative-sidelobe kernels when a stacked template is broader than the science image. Set False to restore warning-only behavior.
`kernel_order`	`0` (int|"auto")	Spatial polynomial order for the SFFT kernel. 0=constant (low RAM, default), 1=linear, 2=quadratic, 3=cubic. An explicit integer is authoritative and is never escalated. "auto" starts at 0 and adds spatial freedom only with evidence: 1 for large PSF mismatch or last-resort alignment when >=30 sources are available, 2 only for severe mismatch with >=80 sources and a manageable kernel size. Orders >=2 can require >7 GB RAM. Higher orders add freedom, not guaranteed accuracy: they can fit edge noise or masked-source systematics when no real spatial variation exists.

continued

Key	Default (type)	Description
`kernel_hw_fwhm_multiplier`	`2.5` (float|null)	Multiplier applied to the PSF-mismatch FWHM when sizing the subtraction kernel half-width. The kernel FWHM is computed as sqrt(FWHM_broad^2 - FWHM_narrow^2); kernel_hw ~= multiplier * FWHM_conv. If null, the templates.py sizing uses 2.5 and run_sfft.py auto-sizing uses 2.0. Lower values (e.g. 1.5) give a tighter kernel; higher (e.g. 3.0) allow more fitting freedom at the cost of computation. For undersampled images (FWHM < 2 px), the multiplier is automatically boosted by 1.5x to capture extended PSF wings.
`kernel_hw_override`	*null* (int|null)	If set, directly overrides the auto-computed kernel half-width in pixels. Use for debugging or when automatic sizing is wrong.
`sfft_auto_kernel_size`	`True` (bool)	If True (default), let run_sfft.py auto-size the SFFT kernel half-width using its own formula (max of ceil(mult*FWHM_broad) and ceil(3*FWHM_conv/2.355), per SFFT Hu et al. 2022 and ip_diffim conventions). The templates.py-computed value is kept for logging/diagnostics. If False, the templates.py-computed kernel_half_width is passed directly. Setting to False restores pre-fix behavior (which used FWHM_conv as the primary term and could produce kernels too small to contain the full PSF, causing dipole residuals and flux scaling mismatch).
`fwhm_crosscheck_max_ratio`	`4.0` (float)	Sanity ceiling on the science/template FWHM ratio. A larger ratio means one FWHM measurement is probably contaminated (saturation/crowding/few stars); the pipeline logs a warning and keeps kernel sizing conservative. Set higher to silence the warning for genuinely very different PSFs.
`sfft_sources_cap_threshold`	`30` (int)	Source count below which the templates.py kernel half-width (including the PSF-mismatch convolution term) is capped by the constraint budget (2*hw+1)^2 <= 100*n_eff, and never reduced below the physical minimum. That minimum is ceil(1.5*FWHM_broad) while the field can afford it (broad-floor kernel pixels <= sfft_kernel_pix_per_source * n_eff); otherwise it is re-anchored to ceil(1.5*FWHM_conv), since the matching kernel only needs to cover the PSF-difference core. Prevents under-constrained kernels that fit noise/ringing in small-source fields; the alignment-RMS boost cannot push the kernel back past this cap. Set to 0 to disable the cap (only sfft_sparse_field_threshold would then apply).
`sfft_cap_order_on_weak_alignment`	`True` (bool)	If True (default), force the SFFT/HOTPANTS kernel spatial order to 0 when the aligned template's verification tier (ALIGVER) is known-weak ("unverified" or "translation_verified"): a spatially-varying kernel would fit alignment systematics rather than a real kernel gradient. Absent tiers are left untouched. Set False to allow higher orders regardless of alignment verification.

continued

Key	Default (type)	Description
`sfft_sparse_field_threshold`	`10` (int)	When the number of vetted matching sources is below this threshold, the templates.py-computed kernel half-width is passed directly to SFFT instead of letting run_sfft.py auto-size. The templates.py formula uses FWHM_conv as the primary term with an adaptive floor and source-count cap, producing a smaller, better-constrained kernel for sparse fields. run_sfft.py's auto-size uses FWHM_broad as the primary term, which can produce an under-constrained kernel when few sources are available. Set to 0 to always auto-size.
`sfft_const_phot_retry_pct`	`10.0` (float)	Flux scaling discrepancy percentage threshold for retrying SFFT with ConstPhotRatio=True in sparse fields. When the convolution-based flux scaling differs from the photometric estimate by more than this percentage and the field is too sparse for the standard kernel_order+1 retry (fewer than ~20 sources), retry with a spatially-constant flux-scaling model to remove under-constrained polynomial degrees of freedom. NOTE: ConstPhotRatio does NOT pin the kernel sum to the SExtractor photometric ratio – the retry is adopted only if it does not increase the median absolute residual at matched-source positions. Also note FSCAL_PHOT is biased by PSF/aperture differences between the images, so moderate discrepancies are a kernel-quality flag, not proof of wrong normalisation. Set to 0 to disable.
`download_templates`	`False` (bool|str)	If set, download templates into `fits_dir/templates/` before subtraction. Use one of: "panstarrs", "legacy", "sdss" (or False to disable). The run stops with an error when user-supplied template files already exist in any `templates` directory under `fits_dir` (flat `fits_dir/templates/` or per-dataset `fits_dir/<dataset>/templates/`) and this is set - remove them or set False to use them.
`templates_size`	`10` (float)	Minimum template cutout size (arcmin). When image WCS is available each band's cutout is centred on the bounding box of that band's image footprints (not the target, so offset/dithered pointings stay covered) and sized to cover it times templates_margin; templates_size is the floor and the target-centred fallback when no footprint resolves.
`templates_margin`	`1.1` (float)	Padding factor on the science-image footprint bounding box when sizing downloaded templates (1.1 = 10% border beyond the image corners). Border sky gives source-based alignment extra anchors.
`templates_max_arcmin`	`60.0` (float)	Cap (arcmin) on each side of the footprint-union box used for downloaded-template sizing; larger FOV unions are truncated and the region is flagged [clamped]. The PS1 cutout service additionally limits each side to 6000 px (~25 arcmin at 0.25"/px).
`hotpants_exe_loc`	`hotpants` (str)	HOTPANTS executable name/path (defaults to `hotpants` on PATH).
`hotpants_timeout`	`100` (int)	Timeout (seconds) for HOTPANTS subtraction call.
`image_shape`	*null* (list[int]|null)	Optional expected image shape (ny, nx) for validation; null disables check.
`use_catalog_for_matching`	`True` (bool)	If True, use catalog sources (when available) for matching/registration heuristics.

continued

Key	Default (type)	Description
match_max_sources	2000 (int)	Cap on matching priors passed to the subtraction stage. Candidates are ranked brightest-first with an isolation constraint (>= 2xFWHM separation), filling the remainder with the next-brightest sources, so the subset spreads across the field. use_for_matching detections carry no NMAX cap, so deep/crowded fields can produce tens of thousands of candidates; fclusterdata deduplication and per-source aperture photometry are O(N^2)/O(N) in that count. 0 disables the cap.
sfft_crowded_auto	False (bool)	Auto-enable SFFT crowded-field tuning based on source density heuristics.
sfft_crowded_min_sources	300 (int)	Crowded-field trigger: minimum number of detected sources for SFFT crowded-mode auto-trigger.
sfft_crowded_min_density	1.5 (float)	Crowded-field trigger: minimum sources per sq arcmin for SFFT crowded-mode auto-trigger.
sfft_bg_order	0 (int)	SFFT background spatial polynomial order (0=constant). Images are always background-subtracted (constant median removed by pipeline before SFFT), so bg_order=0 is sufficient to absorb any residual constant offset. Higher orders can overfit and introduce spatial structure that mimics point-source residuals. Set to 1-2 only for fields with strong residual spatial background gradients after median subtraction.
sfft_star_ext_iter	4 (int|null)	SFFT StarExt_iter (source extension iterations). An explicit value is used for both sparse and crowded modes; set to null for adaptive (4 for sparse, 2 for crowded). Higher values (6-8) improve deblending for complex PSFs but are slower.
sfft_back_size	128 (int)	SExtractor BACK_SIZE used by SFFT source selection/background estimation.
sfft_back_filtersize	6 (int)	SExtractor BACK_FILTERSIZE used by SFFT background smoothing.
sfft_backphototype	LOCAL (str)	SExtractor BACKPHOTO_TYPE for source photometry background: LOCAL or GLOBAL.
sfft_detect_thresh	*null* (float|null)	SExtractor DETECT_THRESH for SFFT source selection (sigma). If null, uses 2.0 for sparse fields, 3.0 for crowded fields. Lower values detect fainter sources but may include noise peaks that bias the kernel fit.
crowded_field	False (bool)	Global crowded-field flag for subtraction/source selection heuristics.
sextractor_crowded	False (bool)	Use crowded-field SExtractor settings for subtraction/source selection (if supported).
sfft_crowded_method	False (bool)	Force SFFT crowded-field configuration (overrides auto logic). Default False (sparse) for better performance.
sfft_const_phot_ratio	False (bool)	SFFT ConstPhotRatio; False fits a spatially-varying flux-scaling polynomial (like HOTPANTS). True uses a single spatially-constant solved flux scale (fewer degrees of freedom; can oversubtract where the true flux ratio varies across the image). This does NOT constrain the kernel sum to the SExtractor photometric ratio – the realised scaling comes from the least-squares kernel fit in both modes. When False (default), auto-enabled for very sparse fields (< sfft_const_phot_ratio_sparse_threshold matching sources) where the spatial polynomial is underconstrained. Set sfft_const_phot_ratio_sparse_threshold to 0 to disable auto-enable.

continued

Key	Default (type)	Description
`sfft_const_phot_ratio_sparse_threshold`	`15` (int)	Auto-enable ConstPhotRatio=True when the number of matching sources is below this threshold (default 15). A spatially-varying scaling polynomial needs far more constraints than a sparse field provides – with ~10 sources it absorbs source noise as fake structure and drives the flux-scale discrepancy; the constant model is the safer choice. Set to 0 to disable auto-enable.
`sfft_only_flags`	`[0, 1, 16, 17]` (list[int]| null)	SExtractor FLAGS values kept by SFFT source selection (null disables flag filtering). Excludes blended sources (FLAGS & 2) which have unreliable photometry and bias the SFFT flux scaling. FLAGS=1 (near edge) and FLAGS=16 (near NaN) are kept since they are common in masked/resampled images and do not bias photometry. If SFFT fails with these restrictive flags, the pipeline automatically retries once with permissive flags [0,1,2,3,16,17,18,19].
`sfft_allow_crowded_bg_order_override`	`False` (bool)	If True, crowded SFFT may override bg_order=0 to 2 for stability.
`sfft_cvrej_magd_thresh`	`0.12` (float)	Coarse variable-rejection magnitude-difference threshold (lower is stricter).
`sfft_coarse_var_rejection`	`False` (bool)	Enable SFFT coarse variable-star rejection. When True, sources with magnitude differences exceeding sfft_cvrej_magd_thresh are excluded from the kernel fit. Disabled by default - in sparse fields with PSF differences, this rejects most sources (e.g. 14/15 killed with 0.12 mag threshold) because aperture mag differences are caused by PSF mismatch, not variability.
`sfft_evrej_ratio_thresh`	`6.0` (float)	Elaborate variable-rejection ratio threshold (lower is stricter).
`sfft_elabo_var_rejection`	`False` (bool)	Enable SFFT elaborate variable-star rejection. When True, SFFT performs a second-pass ratio-based rejection using sfft_evrej_ratio_thresh and sfft_evrej_safe_magdev to further filter variable sources from the kernel fit. Disabled by default - same problem as coarse variable rejection: in sparse fields with PSF differences, the 0.04 mag threshold rejects 60% of matched sources (9/15) because aperture mag differences are caused by PSF mismatch, not variability. With only 6 sources left, the kernel is poorly constrained -> flux scaling mismatch -> dipoles.
`sfft_evrej_safe_magdev`	`0.04` (float)	Safe magnitude-deviation threshold for elaborate rejection.
`sfft_pac_ratio_thresh`	`4.0` (float)	Post-anomaly-check ratio threshold for SFFT source filtering.
`sfft_use_post_anomaly_feedback`	`True` (bool)	If True, run one extra SFFT pass using MeLOn post-anomaly sources as additional banned priors (also removed from the prior matching list). The retry result is adopted only if it produces a valid difference image; otherwise the first-pass result is kept.
`sfft_post_anomaly_min_count`	`1` (int)	Minimum number of post-anomaly sources required before feedback retry is triggered. 1=retry on any anomaly (aggressive). 3=only retry when multiple sources flag anomalies (conservative).
`sfft_post_anomaly_max_fraction`	`0.80` (float)	Warning threshold for the post-anomaly fraction (fraction of prior sources flagged). When exceeded, the feedback retry still proceeds with a boosted kernel half-width (+~50%, capped) to accommodate the likely poor kernel fit, and a warning is logged.

continued

Key	Default (type)	Description
`sfft_post_anomaly_match_radius_px`	`1.5` (float)	Radius (pixels) for removing prior-selected matches near post-anomaly sources on retry.
`sfft_flux_discrepancy_retry_pct`	`3.0` (float)	Flux scaling discrepancy percentage threshold that triggers a retry with higher kernel_order. When convolution vs photometric flux scaling mismatch exceeds this, SFFT retries with kernel_order+1 (capped at 2) if enough sources are available. Set to a large value (e.g. 100) to disable.
`sfft_fscal_phot_sanity_frac`	`0.15` (float)	Fractional deviation at which SFFT's SExtractor-based FSCAL_PHOT is repaired to the matched-aperture scale (FSCAL_AP, written to the diff header with FSCAL_NAP ratio count). FSCAL_PHOT is the correction target and gain basis for CONVD=SCI photometry, so a badly-biased value (it has produced ~5-8x on same-instrument pairs) corrupts the correction and the discrepancy metric; moderate deviations within this fraction are kept because a few-percent aperture bias on mismatched PSFs is real. Set to 0 to repair on any deviation, or a huge value to disable.
`sfft_fscal_ap_min_sources`	`5` (int)	Minimum matched-aperture ratios required before FSCAL_AP can referee (and repair) FSCAL_PHOT. Below this the aperture estimate is too noisy to trust and the SExtractor value stands.
`sfft_diff_rescale_min_frac`	`0.10` (float)	CONVD=REF difference-image photometric rescale. When the solved convolution flux scale (FSCAL_CONV) deviates from the trusted matched-aperture scale (FSCAL_AP, requiring sfft_fscal_ap_min_sources ratios) by more than this fraction, the least-squares scale is the outlier and every source keeps a fractional positive residual – the diff is rewritten as SCI - (AP/CONV)*(SCI - DIFF), which is algebraically SCI - AP*Conv(REF) at the correct flux scale while the transient keeps native science units (a plain multiplicative rescale would corrupt them). The solved background constant is sub-percent of sky noise and is rescaled with it. Original scale is kept in FSCAL_CV2; FSCAL_CONV/FSCAL_DISC are updated so the error budget sees the residual FSCAL_AP-vs-FSCAL_PHOT spread only. Fires once per product inside the FSCAL stamping pass, so retries re-evaluate fresh cards; an already-corrected product reads conv==ap and is untouched. Set to 0 to disable (raw SFFT product stands, discrepancy folded into the error budget).
`sfft_pass_masked_sources`	`False` (bool)	If True, pass excluded/masked sources to SFFT via -masked_sources argument. If False, SFFT will not receive source bans from the pipeline.
`sfft_manual_matching_sources`	*null* (list[list[float]]|null)	Optional list of [x, y] pixel coordinates to use as prior matching sources for SFFT. If provided, overrides automatic source matching. Example: [[100.5, 200.3], [150.2, 300.1]]
`sfft_backend`	`numpy` (str)	SFFT compute backend passed to run_sfft.py: "numpy" (CPU, default) or "cupy" (NVIDIA GPU; requires cupy installed in the environment). run_sfft.py warns and falls back to numpy when cupy is unavailable.
`sfft_cuda_device`	`"0"` (str)	CUDA device index used when sfft_backend=cupy.

continued

Key	Default (type)	Description
`sfft_use_bspline_kernel`	`False` (bool)	Run a B-spline/polynomial spatial-kernel refinement pass (BSpline_Packet.BSP) after the main SFFT solve (SFFT v1.5.0+). Requires the Cupy/GPU backend in sfft 1.7.x (the Numpy path is unimplemented upstream); on CPU runs it is skipped with a warning and the standard kernel result is kept. Better for non-Gaussian PSFs.
`sfft_regularize_kernel`	`auto` (str)	B-Spline kernel regularization (Hu et al. 2024): Laplacian penalty on the matching kernel's second derivatives to suppress delta-basis overfitting. Cupy backend only (requires sfft_use_bspline_kernel too). "auto" enables it when matched sources < sfft_regularize_sparse_threshold; "true"/"false" force on/off.
`sfft_regularize_lambda`	`1e-6` (float)	Regularization strength (SFFT LAMBDA_REGULARIZE). Larger values penalize kernel roughness more strongly; too large can underfit genuine PSF mismatch.
`sfft_regularize_sparse_threshold`	`30` (int)	With sfft_regularize_kernel=auto, enable regularization only when the number of matched sources falls below this count.
`sfft_kernel_noise_retry`	`0.12` (float)	Under-constrained-kernel retry trigger. run_sfft.py writes the realized kernel's wing noise (outer-annulus MAD / peak) to the KERNOISE header card; when it exceeds this threshold the delta-basis solve is noise-dominated (few matched sources vs kernel pixels) and the pipeline retries once with a smaller kernel sized by sfft_kernel_pix_per_source. The retry is adopted only if it does not increase matched-source residuals by >10%. Set to 0 to disable.
`sfft_kernel_negfrac_retry`	`0.40` (float)	Second under-constrained-kernel retry trigger: when the realized kernel's negative-mass fraction (KERNEG header card) exceeds this, the solve is putting mass into cancellation sidelobes rather than the transform even if KERNOISE stayed low (J2344 TDP6: KERNEG=0.46 at KERNOISE=0.079), so the source-budgeted retry fires. A solved kernel can legitimately carry ~0.2-0.3 negative mass when matching unlike PSF shapes; 0.4 errs toward triggering only on clearly unconstrained solves. Set to 0 to disable.
`sfft_kernel_pix_per_source`	`35.0` (float)	Kernel-pixel budget per matched source, used in three places: the retry kernel size (hw_retry = (sqrt(budget * n_sources) - 1) / 2, floored at ceil(FWHM_conv) so the retry kernel still holds the PSF-difference core), the over-budget retry trigger (a realized kernel larger than budget * n_constraints retries even when KERNOISE/KERNEG look acceptable), and the physical-minimum anchor in primary kernel sizing. ~35 px/source keeps the solved kernel's wing noise near 10% of its peak on sparse fields.
`sfft_decorrelate_noise`	`False` (bool)	Apply noise decorrelation to difference image (SFFT v1.5.0+). Whitens correlated noise using the DMTN-021 method (Reiss & Lupton 2016). Disabled by default: the decorrelated image is only used for a cosmetic plot panel, not for detection or photometry. The main diff image is always non-decorrelated regardless.
`sfft_save_decorrelated`	`False` (bool)	Save decorrelated difference image separately as *_decorr.fits. Disabled by default since decorrelation is off.

continued

Key	Default (type)	Description
`sfft_kernel_hw_min`	`3` (int)	Minimum kernel half-width in pixels (ensures at least 7x7 kernel for stability).
`sfft_kernel_hw_max`	`50` (int)	Maximum kernel half-width in pixels (prevents excessive computation for very large FWHM differences).
`sfft_min_prior_sources`	`3` (int)	Minimum number of prior sources required to use them for kernel fitting. If fewer sources are provided, SFFT performs its own matching. Default 3 (lowered from 10): pipeline-vetted sources are higher quality than SFFT's own matching (which allows blended sources). With >=3 vetted sources, SFFT uses them as preferred priors.
`sfft_self_match_min_sources`	`20` (int)	If the prior-selected SFFT run keeps fewer matched sources than this, try one prior-free SFFT self-match for more kernel constraints. The self-match result is adopted only when it has enough sources and does not increase median source-aperture residuals (or, when still below the threshold, clearly improves both count and residuals). 20 matches the sparse-prior target: 10-19 realized constraints can still leave the kernel under-constrained. Set equal to sfft_min_prior_sources to disable the extra pass.
`sfft_self_match_est_sources`	`10` (int)	Provisional constraint count assumed for kernel sizing when SFFT will self-match (fewer than sfft_min_prior_sources priors). Replaces a hard-coded estimate of 20, which over-provisioned the kernel on genuinely sparse fields. The realized count is measured post-run from the NKERSRC header card; when SFFT retained fewer sources than assumed, the solve is treated as under-constrained and the kernel-size retry applies.
`residual_artifact_mask`	`True` (bool)	Mask compact subtraction dipoles at bright static cores after validating that a subpixel local-translation model explains the paired positive/negative lobes. Prevents local registration/nonlinear-core artifacts from contaminating photometry. Protected positions (primary target plus variable/additional-target prior bans) are never masked.
`residual_artifact_min_static_snr`	`50.0` (float)	Minimum smoothed static-core S/N required before a residual can be treated as a subtraction artifact.
`residual_artifact_min_core_radius_px`	`1.4` (float)	Minimum half-light radius of the static core. Point-source stars are left to the existing saturation/source masks; this pass targets broader galaxy-like cores that can carry a local-registration dipole without a saturation flag.
`residual_artifact_min_peak_sigma`	`5.0` (float)	Minimum absolute residual peak relative to the robust whole-image difference-image sigma.
`residual_artifact_min_rms_sigma`	`0.5` (float)	Minimum residual RMS within the fit region relative to the robust whole-image difference-image sigma.
`residual_artifact_min_rms_drop`	`0.20` (float)	Minimum fractional RMS reduction required from the local translation model before masking.
`residual_artifact_min_shift_px`	`0.02` (float)	Minimum fitted local translation; smaller values are treated as noise, not registration artifacts.
`residual_artifact_max_shift_px`	`1.0` (float)	Maximum fitted local translation accepted as a compact subtraction artifact.
`residual_artifact_lobe_sigma`	`3.0` (float)	Signed residual threshold used to require both positive and negative dipole lobes.
`residual_artifact_min_lobe_pixels`	`2` (int)	Minimum number of pixels required in each signed dipole lobe.
`residual_artifact_fit_radius_fwhm`	`4.0` (float)	Radius, in science FWHM units, used to fit the local translation model.

continued

Key	Default (type)	Description
residual_artifact_mask_radius_fwhm	2.5 (float)	Radius, in science FWHM units, masked around each accepted artifact core.
residual_artifact_protect_radius_fwhm	4.0 (float)	Radius around protected transient/variable positions that can never be artifact-masked.
residual_artifact_candidate_sep_fwhm	4.0 (float)	Minimum separation between evaluated bright-core candidates.
residual_artifact_max_regions	4 (int)	Maximum number of artifact regions masked per difference image.
sfft_max_flux_radius_frac	0.9 (float)	Maximum FLUX_RADIUS/FWHM ratio for SFFT kernel sources. A point source's half-light radius is ~0.5-0.7x its FWHM; larger ratios indicate galaxies or extended defects, whose different profiles in science and reference images bias the SFFT kernel solution. This replaces the removed CLASS_STAR gate, whose neural classifier is uncalibrated on some cameras. Sources with missing/NaN values are kept. Set 0 to disable.
sfft_profile_rescue_min	10 (int)	Sparse-field keep-floor for the profile-concentration cut. When the threshold would leave >=5 but fewer than this many sources, the cut is relaxed to keep the most concentrated candidates instead of a hard rejection. Default 10 matches the SFFT kernel-quality warning threshold.
sfft_max_ellipticity	0.5 (float)	Maximum SExtractor ellipticity for SFFT kernel sources. 0.5=permissive (default), >=1.0=disabled. SFFT vets its own sources; this is only a coarse prefilter for obviously elongated objects.
sfft_hard_max_elongation	1.5 (float)	HARD maximum A/B elongation ratio for SFFT kernel sources. Sources with ELONGATION > 1.5 are rejected unless doing so would leave fewer than sfft_elongation_rescue_min sources, in which case the roundest candidates are kept. 1.0=disabled.
sfft_elongation_rescue_min	3 (int)	Sparse-field keep-floor for the hard elongation cut. When removing elongated sources would leave fewer than this many candidates, the roundest sources are kept so the kernel fit is not starved. Defaults to sfft_min_prior_sources when unset.
sfft_cr_exclusion_fwhm	1.0 (float)	Distance (in FWHM units) from hardware_defects_mask pixels (cosmic rays, trails, bad columns, saturation, MaxiMask classes when enabled, plus non-finite pixels) within which SFFT matching sources are excluded. Prevents defects from entering the SFFT kernel-fitting pool and corrupting the subtraction. Set to 0 to disable.
sfft_target_exclusion_fwhm	2.0 (float)	Exclusion radius around the configured target (in FWHM units of the larger of science/template FWHM, floored at 3 px) applied to the matched-source pool before subtraction priors are built. A matched pair at the target position is transient-plus-host flux; keeping it lets the transient bias the kernel fit and flux scale via the consistency inliers, SNR fallback, NaN-flux add-back, sparse passthrough, or the ZOGY LSQ stamp list. The same position is banned from SFFT's XY_PriorBan list and the sparse-prior supplement independently. Set to 0 to disable the matched-pool ban (not recommended).

continued

Key	Default (type)	Description
`sfft_fft_rejection`	`True` (bool)	If True, reject SFFT kernel sources whose 2D FFT power spectrum deviates > sfft_fft_n_sigma from the median across all sources. Same technique as PSF build (psf.py detect_fft_outliers). Catches extended/blended/spike-contaminated sources that pass catalog-level cuts.
`sfft_fft_n_sigma`	`4.0` (float)	FFT power-spectrum outlier threshold in sigma (MAD-based). Sources whose median per-frequency deviation exceeds this are rejected. 4.0 = strict (same as PSF build). A relaxed retry at 1.5x this value is attempted if the strict cut leaves too few sources.
`sfft_fwhm_ratio_nsigma`	`0.0` (float)	Cross-image FWHM consistency: reject sources whose FWHM ratio (sci/ref) deviates more than this many sigma (MAD) from the median log-ratio. Catches sources blended in one image but isolated in the other. 0=disabled (default). Enable (e.g. 3.0) only if kernel fits are poor and blended sources are suspected.
`sfft_fwhm_ratio_min`	`0.5` (float)	Cross-image FWHM consistency: hard minimum on FWHM ratio (sci/ref). A source whose catalogued width differs by more than a factor of ~2 between frames is extended or blended in one of them; always active when both FWHM columns exist (unlike the MAD clip, it does not require sfft_fwhm_ratio_nsigma).
`sfft_fwhm_ratio_max`	`2.0` (float)	Cross-image FWHM consistency: hard maximum on FWHM ratio (sci/ref); always active (see sfft_fwhm_ratio_min).
`sfft_stamp_fwhm_max_ratio`	`1.6` (float)	Direct stamp-morphology check on SFFT/HOTPANTS kernel priors: each prior's own radial-profile FWHM is measured on a peak-recentred stamp in BOTH the science and template images, and sources wider than this ratio x the frame PSF FWHM in either one are rejected. Unlike the catalog morphology cuts this does not depend on SExtractor columns and catches sources extended in only one frame (flux-consistency matching cannot remove galaxies because their aperture flux is stable between epochs). Bounded by the sfft_psf_quality_min_keep floor, worst-ratio first; unmeasurable stamps (edge, masked) are kept. Set <=1.0 to disable.
`sfft_flux_ratio_max_deviation`	`0.0` (float)	Cross-image flux-ratio sanity: reject sources whose flux ratio (sci/ref) deviates from the median by more than this factor. Catches mismatched sources, variables, and contamination. 0=disabled (default). RANSAC in find_flux_consistent_sources already does fine filtering; enable only if mismatched sources are suspected.
`sfft_flux_mag_residual_threshold`	`0.0` (float)	Maximum magnitude residual for flux-consistent source selection (mag). 0=use built-in default (0.3 mag). Tighter values (e.g. 0.15) reject variable/mismatched sources more aggressively before passing to SFFT. Recommended when SFFT PostAnomaly rejects many sources.
`sfft_flux_linearity_min_corr`	`0.90` (float)	Minimum Pearson correlation between science and template magnitudes for flux-consistent sources. Below this, the relationship is non-linear (variables/mismatches present). Warning only; does not reject sources by default.

continued

Key	Default (type)	Description
`sfft_flux_linearity_max_resid_corr`	`0.50` (float)	Maximum absolute Spearman correlation between magnitude and fit residuals. Above this, residuals trend with magnitude (non-linear/curved relationship), indicating flux-dependent bias or variables. Sources following the trend are rejected.
`sfft_flux_linearity_min_trend_sources`	`20` (int)	Minimum inlier count before the residual-magnitude (non-linearity) trend correction may remove sources. Below this, a rank correlation is easily inflated by a few faint, high-error points and removing priors costs SFFT more than a marginal non-linearity; the warning is still logged. The removal itself is error-normalised: a source is only rejected where the quadratic trend amplitude exceeds both 2x the detrended scatter and 2x its own magnitude error.
`sfft_snr_model_relerr`	`0.10` (float)	Assumed fractional uncertainty of the flux-error (noise) estimate, used to build S/N error bars in the flux-comparison S/N-consistency stage: sigma_SNR = sqrt(1 + (rho*SNR)^2). The '+1' term is the flux-measurement contribution (exact); rho accounts for background-estimation uncertainty dominating at high S/N. 0.10 is conservative for aperture photometry; set ~0.03-0.05 for very well-characterised noise models, 0 to use numerator noise only.
`sfft_psf_vetting`	`True` (bool)	Require flux-comparison sources to be well fitted by the PSF model in BOTH images (per-source stamp fit, reduced chi2 cut). Rejects blends, extended sources and artifacts that aperture-flux consistency alone cannot catch. Sources whose stamp cannot be measured (edge/fully masked) are kept. Set False to restore flux-consistency-only selection.
`sfft_psf_chi2_max`	`5.0` (float)	Maximum reduced chi2 of the per-source PSF-stamp fit for a source to be retained. The noise model includes a model-mismatch floor (sfft_psf_model_floor_frac), so well-fit stars sit near chi2_red ~1 even at high S/N; 5.0 is a permissive cut aimed at blends/extended/artifact sources. Acts as a floor on the effective per-side cut, which can rise above it via sfft_psf_chi2_recal_nsigma when a side's chi2 distribution is systematically inflated. 0 disables the veto.
`sfft_psf_model_floor_frac`	`0.05` (float)	Fractional PSF-model-mismatch floor added in quadrature to the stamp pixel noise before the chi2 test. Prevents bright, well-fit stars from being rejected for sub-percent ePSF inaccuracies. Applies to sides vetted with an empirical ePSF. 0 disables the floor.
`sfft_psf_model_floor_frac_analytic`	`0.15` (float)	Mismatch floor used on a side whose vetting model is analytic (the template always uses a Moffat at the image FWHM; the science side falls back to one when the ePSF build fails). An analytic model cannot capture real PSF structure, so its expected mismatch is much larger (~15% measured on resampled stacks): with the empirical floor, bright unsaturated stars - the best kernel anchors - are vetoed for model error rather than morphology. Only gross pathologies (blends/artifacts with >~30% mismatch) still fail.

continued

Key	Default (type)	Description
`sfft_psf_min_keep`	`8` (int)	Adaptive keep-floor for the PSF-quality veto: if the veto would leave fewer than this many surviving flux-consistent inliers, the best-fitting vetoed sources (lowest worst-image chi2_red, each side normalised by its own effective cut) are restored until the floor is met. Guarantees the SFFT prior pool is never starved by the veto while still rejecting the worst fits; the relaxed effective threshold is logged. Capped by the number of inlier sources available.
`sfft_psf_min_keep_frac`	`0.5` (float)	Fractional keep-floor for the PSF-quality veto: restores best-fitting vetoed sources until at least this fraction of the robust inlier pool survives. Guards against systematic model-limited over-rejection (e.g. a crude side model vetoing most bright stars) starving the kernel fit, where the absolute sfft_psf_min_keep floor alone would leave too few priors. 0 disables.
`sfft_psf_chi2_recal_nsigma`	`4.0` (float)	Per-side robust recalibration of the PSF-stamp chi2 veto. Each image's measured chi2 distribution (over the pair-residual-robust pool) is tracked independently: the effective cut is max(sfft_psf_chi2_max, median + this many MAD-sigma). A systematically inflated side - resampled/correlated template pixels, model-limited analytic vetting PSF, structured background - is therefore cut at its own upper tail, rejecting true outliers rather than typical field stars an absolute cut could never pass. Never tightens below sfft_psf_chi2_max. 0 disables recalibration (strict absolute cut).
`sfft_psf_chi2_recal_min_sources`	`8` (int)	Minimum finite stamp-chi2 measurements on one image side before its recalibrated threshold is trusted; below this the absolute sfft_psf_chi2_max cut applies and sparse fields rely on the keep-floor restore instead.
`sfft_min_consistent_for_nan_add`	`10` (int)	Minimum number of flux-consistent sources required before NaN-flux (position-only) sources are added back for SFFT priors. Prevents unvetted sources from dominating the kernel fit when few consistent sources exist. Below this gate a sparse-field rescue tier still admits them when >= sfft_nan_rescue_min_consistent, but caps the additions at the vetted count so they can never outnumber consistent priors; the downstream refinement gauntlet vets every added position.
`sfft_nan_rescue_min_consistent`	`3` (int)	Minimum flux-consistent sources before the capped sparse-field rescue tier may admit NaN-flux position-only priors (see sfft_min_consistent_for_nan_add).
`sfft_sparse_prior_supplement`	`true` (bool)	When the refined prior pool is smaller than sfft_sparse_prior_target, pull additional stellar positions from the cleaned photometric catalog (snapshot taken before ZP cleaning). Only positions not covered by any science detection are offered (a catalog row matching a vetoed detection would re-admit a variable), each must measure a non-extended star in BOTH image stamps via sfft_stamp_fwhm_max_ratio, and the target plus known variables are banned. Also rescues fields where the cross-match starves entirely. Requires sfft_stamp_fwhm_max_ratio > 1.0.

continued

Key	Default (type)	Description
`sfft_sparse_prior_target`	`20` (int)	Pool size the sparse-field supplement tries to reach. Additions stop once this many priors are present. A ~10-prior pool under-constrains the delta-basis kernel (J2344: median 11 priors, ~46% negative kernel mass, dominant dipole/bright-star residuals); ~20 gives the solve a workable constraint budget.
`sfft_sparse_prior_max_add`	`30` (int)	Hard cap on catalog positions added per field, regardless of target.
`sfft_flux_fallback_max_sources`	`30` (int)	Maximum number of sources used in the fallback when flux consistency returns < 5 sources. Sources are ranked by SNR/threshold and the top N are kept. Prevents all sources (including non-linear/variable) from being passed to SFFT.
`sfft_stamp_contam_sigma`	`5.0` (float)	Pixel-level stamp contamination: minimum sigma above background for a secondary peak within the kernel stamp to reject a source. Prevents uncatalogued neighbours from contaminating the SFFT kernel fit.
`sfft_stamp_contam_frac`	`0.3` (float)	Pixel-level stamp contamination: minimum fraction of central source flux for a secondary peak to reject the source. Catches bright uncatalogued sources within the stamp that would bias the kernel.
`sfft_defects_stamp_radius`	`0` (int)	Radius (px) of stamp to check around each matching source for diffraction spike/trail contamination. 0 = auto (uses kernel half-width). Sources with > sfft_defects_max_frac of the stamp masked by hardware defects are rejected.
`sfft_defects_max_frac`	`0.15` (float)	Maximum fraction of the stamp that can be covered by hardware defects (saturation streaks, satellite trails, NaN) before the source is rejected. Lower = stricter; 0.15 means >15% contaminated stamps are rejected.
`sfft_bright_star_reject`	`True` (bool)	If True, reject matching sources within sfft_bright_star_radius_fwhm * FWHM of any bright star (peak > sfft_bright_star_n_sigma * sigma above background). Prevents contamination from diffraction spikes and PSF wings of bright stars.
`sfft_bright_star_n_sigma`	`50.0` (float)	Bright star detection threshold in sigma above background. 50 sigma catches saturated/near-saturated stars with diffraction spikes. Lower (e.g., 30) catches more stars but may reject good sources.
`sfft_bright_star_radius_fwhm`	`5.0` (float)	Exclusion radius around bright stars in units of FWHM. 5*FWHM covers the typical extent of diffraction spikes. Increase for telescopes with strong spike patterns.
`sfft_max_cfit`	`5.0` (float)	Maximum PSF fit concentration (cfit) for SFFT kernel sources. High cfit = extended/blended. Default 5.0 rejects obviously extended sources. 0=disabled.
`sfft_max_qfit`	`1.0` (float)	Maximum PSF fit goodness-of-fit (qfit) for SFFT kernel sources. High qfit = bad PSF fit = extended/blended. Default 1.0 rejects sources where residuals are as large as the source. 0=disabled.
`sfft_max_reduced_chi2`	`10.0` (float)	Maximum PSF fit reduced chi-squared for SFFT kernel sources. High chi2 = poor model fit = extended or variable. Default 10.0 is permissive. 0=disabled.
`sfft_max_psf_flags`	`-1` (int)	Maximum PSF fit flags value for SFFT kernel sources. Non-zero flags indicate fit problems (blend, edge, etc). -1=disabled (default). Set 0 to reject any flagged source.

continued

Key	Default (type)	Description
`sfft_psf_quality_min_keep`	`5` (int)	Shared keep-floor for the PSF fit-quality cuts (cfit/qfit/reduced_chi2/flags). Each cut applies in full while at least this many priors survive; once the floor is reached only the worst-ranked flagged sources are removed. Prevents independent cuts from combining to starve the SFFT kernel fit in sparse fields.
`sfft_max_fwhm_ratio`	`2.0` (float)	Maximum ratio of source FWHM to image PSF FWHM. Sources with FWHM > 2x the image PSF are extended (galaxies, nebulae) and rejected from kernel fitting. 0=disabled.
`sfft_point_source_min_ellip`	`0.3` (float)	SFFT PointSource_MINELLIP: minimum ellipticity for point-source classification in SFFT's own source vetting. Sources with ellipticity below this are considered extended. Default 0.3 (SFFT default). Lower = more permissive.
`sfft_allow_unvetted_source_retry`	`True` (bool)	If True (default), when SFFT fails or yields too few matched sources while using the pipeline-vetted priors, retry once letting SFFT select its own sources (SExtractor + cross-correlation; keeps the prior-ban list). If False, such failures abort directly to the HOTPANTS fallback.
`sfft_timeout`	`600` (int)	SFFT subprocess timeout in seconds. 600=10min (default). Large images or crowded fields with kernel_order>=1 may need 900+. Timeout triggers fallback to HOTPANTS.
`sfft_total_timeout`	*null* (int|null)	Wall-clock budget (seconds) for the whole SFFT retry cascade inside one subtraction (first pass plus the optional post-anomaly, flux-discrepancy, ConstPhotRatio, kernel-size, flags, and self-match retries). Each pass is separately bounded by sfft_timeout; optional retries are skipped once this total is spent. null = auto (4 x sfft_timeout, i.e. ~40 min at the default).
`sfft_num_threads`	`0` (int)	CPU threads for the SFFT subprocess internals (pyFFTW plans and numba kernels in the Numpy backend, plus BLAS/OpenMP for the dense linear solve). 0 = auto: min(4, cpus) when the pipeline processes images serially (nCPU<=1), 1 when nCPU>1 so parallel image workers do not oversubscribe. SFFT's dense solve is memory-bound long before it is thread-bound, so values above ~4 gain little.
`sfft_max_memory_gb`	`0` (float)	Peak-RAM budget (GB) for one SFFT subtraction, enforced before the subprocess launches and re-checked inside run_sfft.py. SFFT allocates an NEQ x NEQ float64 least-squares matrix (twice under ConstPhotRatio) plus (2*kernel_hw+1) complex128 image planes to apply the solved kernel; for large PSF mismatches this can reach tens of GB and was the cause of OOM/hangs at this stage. What happens when the estimate exceeds the budget is set by sfft_memory_policy. 0 = auto (80% of MemAvailable, capped 24 GB).

continued

Key	Default (type)	Description
sfft_memory_policy	"shrink" (str)	Over-budget policy when the SFFT estimate exceeds sfft_max_memory_gb. "shrink" (default): drop kernel_order toward 0, then reduce kernel half-width toward its PSF-difference floor, and skip to HOTPANTS if even the minimum cannot fit - protects the host but degrades the subtraction kernel. "fallback": never shrink; use HOTPANTS with the SFFT kernel left intact - preserves subtraction quality at the cost of the algorithm choice. "ignore": never shrink; log the estimate and run the full kernel anyway - full SFFT quality but accepts the OOM/swap risk (the estimate is a conservative upper bound, so moderate overshoot often still fits).
inpaint_template_cores	False (bool)	Inpaint saturated template star cores before subtraction (helps when template has broken cores).
inpaint_method	biharmonic (str)	Inpainting algorithm name (e.g. biharmonic).
inpaint_saturate_frac	0.90 (float)	Saturation threshold as fraction of max/bit-depth for core mask creation.
inpaint_dilate_radius	6 (int)	Mask dilation radius (pixels) around saturated cores before inpainting.
subtraction_saturate_fraction	0.90 (float)	Saturation threshold as fraction of SATURATE for source filtering in template subtraction. Sources with peak_flux >= this fraction * SATURATE are excluded from kernel fitting to avoid non-linear regime. Matches PSF building default (0.9).
inpaint_max_mask_fraction	0.01 (float)	Safety limit: skip inpainting if mask covers more than this fraction of the image.

A.14 Background estimation

The two-pass sky estimate: the iterative source mask, the mesh-binning statistics, the clipped interpolation back to the grid, the optional MaxiMask defect classifier, and the target-local `local_*` fit.

```
autophot_input["background"]["use_maximask"] = True  # needs tensorflow
autophot_input["background"]["mesh_scale_min"] = 4.0  # mesh size in FWHM
autophot_input["background"]["mesh_scale_max"] = 10.0
```

Table 35: Background estimation — configuration reference.

Key	Default (type)	Description
crowded_bright_frac_min	0.40 (float)	Crowded-field trigger based on bright-source fraction (pipeline-specific metric).
fast_mode	False (bool)	If True, use faster, approximate background estimation settings.
fast_mode_skip_trail_mask	True (bool)	Only used if fast_mode is True; set false to run trail detection anyway.
mesh_scale_min	4.0 (float)	Minimum background mesh scale (in FWHM units unless otherwise specified).
mesh_scale_max	10.0 (float)	Maximum background mesh scale (in FWHM units unless otherwise specified).
mesh_filter_size_mesh_units	3 (int)	Background smoothing filter size expressed in mesh-cell units.
global_sigma_clip_maxiters	6 (int)	SigmaClip max iterations per mesh cell (lower = faster; use 10+ if noisy).

continued

Key	Default (type)	Description
global_mask_edge_flatten	True (bool)	If True, damp Background2D interpolation overshoot near masked regions by replacing a thin band around mask edges with a smooth estimate from nearby unmasked pixels.
global_mask_edge_flatten_radius_px	*null* (float|null)	Flatten band radius (pixels). If null, defaults to max(2, ~1*FWHM).
global_mask_edge_flatten_sigma_px	*null* (float|null)	Gaussian smoothing sigma (pixels) for mask-edge flattening. If null, defaults to radius/2.
source_mask_convolve	True (bool)	If True, Gaussian-smooth before detection (slower; can reduce mask fragmentation).
source_mask_convolve_first_iter_only	True (bool)	If source_mask_convolve=True, smooth only on iteration 0 (faster).
source_mask_iterative_bkg_update	True (bool)	If True, update residual between mask iterations using a quick coarse Background2D fit.
source_mask_dilate_factor	2.5 (float)	Dilation factor for source masking (multiplier of FWHM). Higher values mask more aggressively around sources. Use 2.0-3.0 for balanced masking, 3.0-4.0 for aggressive masking.
source_mask_dilate_iterations	2 (int)	Number of dilation iterations. Higher values extend masks further. Use 2 for balanced masking, 3 for aggressive masking.
source_mask_max_fraction	0.45 (float)	Upper bound on total masked fraction. When dilation would exceed this, the radius is progressively shrunk so all sources are still masked (cores at minimum) while preserving background pixels. Lower values (0.30-0.35) help in dense fields; higher values (0.50-0.55) mask more aggressively in sparse fields.
zero_threshold	1e-29 (float)	Pixels with |value| below this are treated as invalid (set to NaN) before background estimation. Set to 0 to disable, e.g. for difference images where tiny nonzero flux is meaningful.
saturation_mask_frac	0.90 (float)	Fraction of the saturation level at or above which a pixel is flagged saturated (used by the saturation core and bleed/streak masks). 0.80-0.95 typical; lower values catch more saturated pixels but risk masking bright-source cores.
fwhm_unit	pixels (str)	Units of the FWHM value passed to background estimation: "pixels" (already in detector pixels) or "arcsec" (converted using pixel_scale). Required to size Background2D mesh boxes correctly when FWHM comes from header keywords in arcsec.
max_defect_blobs	500 (int)	Maximum number of connected components processed per defect-mask stage (saturation streaks, satellite trails, defect columns). Blobs are ranked by pixel count (largest first); the cap prevents O(n_blobs) slowdown on crowded frames with many saturated stars. Set 0 to disable the cap.
mask_defect_columns	True (bool)	Mask short axis-aligned ridges (hot-pixel columns/rows, pixel-aligned cosmic-ray clusters) that fall below the satellite trail mask's minimum length. Masked pixels are seeded into the source mask and reported in hardware_defects_mask.
defect_column_n_sigma	3.0 (float)	Detection threshold above the sky median, in biweight sigma units, for the defect column/row mask.
defect_column_min_length_px	12 (int)	Minimum blob length (px) for the defect column/row mask. Below the satellite trail minimum (40 px) but above a compact source.
defect_column_min_aspect_ratio	3.0 (float)	Minimum length/width for a blob to count as a defect column/row.
defect_column_max_width_px	3.0 (float)	Maximum blob width (px) for the defect column/row mask; wider structures are real sources or trails.

continued

Key	Default (type)	Description
defect_column_axis_tolerance_deg	20.0 (float)	Maximum angle (deg) between a blob's principal axis and the row/column direction; genuine bad columns/ rows are detector-aligned, unlike edge-on galaxies.
use_maximask	False (bool)	Enable the MaxiMask deep-learning defect mask.
maximask_classes	[HCL, DCL, HP, DP, P, TRL, SAT, SP, OV] (list)	MaxiMask classes unioned into the defect mask. Default is every true defect/artifact class: the CNN catches faint instances the heuristic masks miss, at no extra inference cost. CR is omitted because astroscrappy (cosmic_rays.remove_cmrays) is the single CR detector and its mask is unioned downstream anyway. BG/BBG/FR/NEB mark scene content or sky structure, not bad pixels (BBG would flag legitimate bright sources).
maximask_batch_size	4 (int)	Tiles per TF call. Smaller is faster on CPU: measured bs=4 22s vs bs=8 36s vs bs=32 175s per ~4MPix frame (non-monotonic, CPU-dependent; tune per machine). Larger batches may help on GPU. # int — Inference batch size in 400x400 tiles; lower it if memory-limited.
maximask_dilate_iterations	1 (int)	3x3 dilation applied to the MaxiMask mask to cover detection edges.
maximask_allow_gpu	False (bool)	Allow TensorFlow to use a GPU. Default False forces CPU-only inference (safe on any host; ~0.02 MPix/s on CPU).
local_mesh_scale	4.0 (float)	Background2D box-size scale factor relative to FWHM (local background). Values below mesh_ scale_min (4) are clamped inside _compute_box_ sizes.
local_min_box	6 (int)	Minimum Background2D box size (pixels).
local_region_fraction_limit	0.15 (float)	Max fraction of image covered by local region before falling back (pipeline-specific).
local_box_size	*null* (int|null)	Optional explicit local background box size (pixels). If null, use adaptive sizing.
local_filter_size	*null* (int|null)	Optional explicit local Background2D filter size (pixels). If null, use computed value (capped by local_ filter_size_max).
local_filter_size_max	3 (int)	Maximum allowed filter size when adaptively increasing filter size.
local_source_mask_nsigma	5.0 (float)	Sigma threshold for masking sources during local background estimation.
local_source_mask_npixels	7 (int)	Minimum connected pixels for source mask creation.
local_source_mask_iterations	2 (int)	Number of mask/estimate iterations for local background.
local_exclude_percentile	80.0 (float)	photutils exclude_percentile: a mesh node is kept while more than (100 - p)% of box pixels are usable. p=95 lets a 95%-masked box contribute an estimate from a few pixels.
local_sigma_clip	3.0 (float)	Sigma clipping threshold for background statistics.
local_sigma_clip_maxiters	10 (int)	Maximum sigma-clipping iterations.
local_nonnegative_target_offset	False (bool)	If True, after local subtraction in the cutout, add a uniform constant to the residual so a chosen cutout statistic meets a floor. DISABLED by default: the lift inflates the Poisson term in calc_total_error (abs(image*gain)), overestimating noise and underestimating SNR on difference images.
local_cutout_floor_stat	median (str)	Which statistic to floor in the locally subtracted cutout: "median" (recommended; allows some negatives) or "min" (forces all finite pixels >= floor; stronger bias).

continued

Key	Default (type)	Description
`local_cutout_min_value`	`0.0` (float)	Floor value for the chosen statistic in the locally subtracted cutout. Use 0.0 for nonnegative median; small positive if you truly require >0.
`local_cutout_annulus_sigma_mult`	`2.0` (float)	Additional requirement: raise the local cutout floor so the background-like median is >= (this * annulus_std), where annulus_std is sigma-clipped std in an annulus around the target on the locally subtracted cutout.
`local_cutout_annulus_in_scale_fwhm`	`3.0` (float)	Inner radius for the annulus_std estimate, in units of FWHM (pixels). The effective inner radius is max(exclude_inner_radius, this*FWHM).
`local_cutout_annulus_width_scale_fwhm`	`2.0` (float)	Width of the annulus used for annulus_std estimate, in units of FWHM (pixels).

A.15 Post-processing

What happens after the per-image pipeline: light-curve colour-term correction, RGB composite images, the Lomb–Scargle periodogram, and instrument-alias handling in the aggregated table.

```
autophot_input["postprocessing"]["color_image"] = True
autophot_input["postprocessing"]["periodogram"] = True
autophot_input["postprocessing"]["transient_color_correction"] = True
```

Table 36: Post-processing — configuration reference.

Key	Default (type)	Description
`transient_color_correction`	`False` (bool)	If True, after the light curve is collected, apply per-image zeropoint color terms to the transient using its own paired color-index observations (same telescope+instrument, within color_correction_dt days; solved iteratively). Writes LightCurve_Output_colorcorrected.csv.
`color_correction_dt`	`1.0` (float)	Max |t1 - t2| in days for pairing color-index filters (e.g. ZTF g and r) in the transient color correction.
`color_correction_max_err`	`0.5` (float)	Max magnitude error (mag) for both members of a color-index pair; non-detections and low-S/N points cannot anchor a color.
`color_image`	`False` (bool)	If True, after the light curve is collected, build an RGB composite (Color_Image.<plot_format>) in the reduced directory: every band's frames are resampled onto the reddest channel's WCS, median-combined, scaled by exposure time and zeropoint, and stretched. Requires >=1 band with a usable WCS; skipped bands log a warning.
`color_image_stretch`	`asinh` (str)	Stretch applied after normalization: "asinh" (linear near the noise floor, logarithmic for bright structure), "log", "sqrt", or "linear".
`color_image_softening`	`10.0` (float)	asinh/log stretch strength; higher values keep more dynamic range visible.
`color_image_black_percentile`	`1.0` (float)	Percentile of finite pixels mapped to black per band.
`color_image_white_percentile`	`99.5` (float)	Percentile of finite pixels mapped to white per band.
`color_image_gamma`	`1.0` (float)	Gamma correction applied to the stretched image (>1 brightens midtones).
`color_image_saturation`	`1.0` (float)	Color saturation multiplier (1 = natural).

continued

Key	Default (type)	Description
`color_image_channels`	*null* (dict|null)	Optional channel-to-band map, e.g. {R: i, G: r, B: g}; a list mixes bands ({G: [g, V]}). Default: effective-wavelength order (reddest -> R, bluest -> B, G synthesized as (R+B)/2 with two bands).
`color_image_max_frames_per_band`	`0` (int)	Cap on frames stacked per band (sharpest FWHM first); 0 = use all frames.
`color_image_save_fits`	`False` (bool)	Also write the pre-stretch linear RGB cube as Color_Image.fits carrying the reference WCS.
`color_image_dpi`	`200` (int)	Output image resolution in dots per inch.
`periodogram`	`False` (bool)	If True, after the light curve is collected, run a Lomb-Scargle period search per band on the target (prefers Differential_Photometry_<method>.csv relative fluxes, else calibrated magnitudes converted to flux). Writes Periodogram_<band>.<plot_format> and periodogram_<band>.csv peak tables.
`periodogram_method`	`PSF` (str)	Photometry method whose series is analyzed: "PSF" or "AP".
`periodogram_min_period`	*null* (float|null)	Shortest trial period in days; null = twice the median cadence.
`periodogram_max_period`	*null* (float|null)	Longest trial period in days; null = half the observing baseline.
`periodogram_samples_per_peak`	`5` (int)	Frequency-grid oversampling factor for the periodogram.
`periodogram_top_peaks`	`5` (int)	Number of top periodogram peaks reported per band.
`lightcurve_instrument_column`	`True` (bool)	If True, append an "Instrument" column ("<TELESCOP>+<INSTRUME>" from the FITS header) to the LightCurve_<method>_<target>.dat photometry table; unknown header values write "-". Canonical survey abbreviations shorten common labels (e.g. Palomar 48-inch+ZTF/MOSAIC -> P48+ZTF).
`lightcurve_instrument_aliases`	`{}` (dict|null)	Optional renames for the joined telescope+instrument label, e.g. {"LCO 2m0-02+Sinistro": "FTS+Sinistro"}.

B Auxiliary YAML schemas

B.1 `telescope.yml`

Located in `wdir` (created by `autophot-inspect-telescope` or `inspect_telescope`). Hierarchy: `<TELESCOP>` → `location` (name/lon/lat/alt for observatory metadata) and `INSTRUME` → `<INSTRUME>` → instrument block with keys `Name` (canonical `<TEL>+<INST>`), the header keywords `gain`, `readnoise`, `saturate`, `airmass`, `mjd`, `date`, `exptime`, `pixel_scale` (numeric, arcsec/pixel), an ordered filter-key chain `filter_key_0`, `filter_key_1`, … tried until a recognised band name appears, and one entry per raw filter value mapping to a canonical band. Canonical band names are case-significant (Section 4); the special value `not_given_by_user` marks a keyword absent from the header convention.

B.2 `catalog.yml`

Bundled with the package. Maps each catalogue backend (`gaia`, `refcat`, `pan_starrs`, `sdss`, `apass`, `2mass`, `legacy`, `skymapper`, `custom`) to its column names for `RA`, `DEC`, and every supported band plus its error column; `custom` also maps `name`, `RA_err`, `DEC_err`. Band keys are case-significant so `R` (Cousins) and `r` (Sloan) map to different columns.

B.3 `filters.yml`

Bundled. `default_dmag` lists, per canonical band, the ordered catalogue colour pairs used as the independent variable in the zero-point colour term (e.g. for V: $B-V$, $V-R$, $V-g$, $V-r$); the first pair whose columns are both present in the downloaded catalogue is fitted.

B.4 TNS and API credentials

`wcs.TNS_BOT_ID`, `TNS_BOT_NAME`, `TNS_BOT_API` configure the Transient Name Server (Transient Name Server, n.d.) lookup used for target resolution; `catalog.MASTcasjobs_wsid`/`pwd` authenticate MAST Casjobs for Refcat2 queries. Credentials may also be supplied through environment variables or `autophot_tokens.py`; they are never written to outputs.

Code availability. AutoPhOT is public at `https://github.com/Astro-Sean/autophot` (Brennan and Fraser, 2022).

References

C. Alard and R. H. Lupton. A Method for Optimal Image Subtraction. *ApJ*, 503(1):325–331, July 1998. doi: 10.1086/305984.

Jay Anderson and Ivan R. King. Toward High-Precision Astrometry with WFPC2. I. Deriving an Accurate Point-Spread Function. *PASP*, 112(776):1360–1382, October 2000. doi: 10.1086/316632.

Astropy Collaboration, A. M. Price-Whelan, P. L. Lim, N. Earl, N. Starkman, L. Bradley, D. L. Shupe, A. A. Patil, L. Corrales, C. E. Brasseur, and others. The Astropy Project: Sustaining and Growing a Community-oriented Open-source Project and the Latest Major Release (v5.0) of the Core Package. *ApJ*, 935(2):167, August 2022. doi: 10.3847/1538-4357/ac7c74.

A. Becker. HOTPANTS: High Order Transform of PSF ANd Template Subtraction. *ASCL*, 1504: 004, May 2015.

Timothy C. Beers, K. Flynn, and K. Gebhardt. Measures of Location and Scale for Velocities in Clusters of Galaxies: A Robust Approach. *AJ*, 100:32–46, July 1990. doi: 10.1086/115470.

Martin Beroiz, Juan B. Cabral, and Bruno Sanchez. Astroalign: A Python module for astronomical image registration. *Astronomy and Computing*, 32:100384, July 2020. doi: 10.1016/j.ascom.2020.100384.

E. Bertin. SCAMP: Automatic astrometric and photometric calibration. *ASCL*, 1010:063, October 2006.

E. Bertin and S. Arnouts. SExtractor: Software for source extraction. *Astronomy and Astrophysics Supplement Series*, 117:393–404, June 1996. doi: 10.1051/aas:1996164.

E. Bertin, Y. Mellier, M. Radovich, G. Missonnier, P. Didelon, and B. Morin. The TERAPIX pipeline. *ASPC*, 281:228, 2002.

Paul T. Boggs, Richard H. Byrd, and Robert B. Schnabel. A stable and efficient algorithm for nonlinear orthogonal distance regression. *SIAM Journal on Scientific and Statistical Computing*, 8(6):1052–1078, November 1987. doi: 10.1137/0908085.

Larry Bradley, Brigitta Sipőcz, Thomas Robitaille, Erik Tollerud, Marco Viana, Christoph Deil, Kyle Barbary, Matthew Newville, Simon Tsien, Hans Moritz G"unther, and others. photutils: Astropy's package for detection and photometry of astronomical sources. *JOSS*, 7(78):4504, September 2022. doi: 10.21105/joss.04504.

S. J. Brennan and M. Fraser. The AUTOmated Photometry Of Transients pipeline (AutoPhOT). *Astronomy and Astrophysics*, 667:A62, November 2022. doi: 10.1051/0004-6361/202243067.

M. R. Calabretta and E. W. Greisen. Representations of celestial coordinates in FITS (Paper II). *Astronomy and Astrophysics*, 395:1077–1122, December 2002. doi: 10.1051/0004-6361:20021327.

K. C. Chambers, E. A. Magnier, N. Metcalfe, H. A. Flewelling, M. E. Huber, C. Z. Waters, L. Denneau, P. W. Draper, D. Farrow, D. P. Finkbeiner, and others. The Pan-STARRS1 Surveys, 2016.

Arjun Dey, David J. Schlegel, Dustin Lang, Robert Blum, Kaylan Burleigh, Xiaohui Fan, Joseph R. Findlay, Douglas Finkbeiner, David Herrera, Stephanie Juneau, and others. Overview of the DESI Legacy Imaging Surveys. *AJ*, 157(5):168, May 2019. doi: 10.3847/1538-3881/ab089d.

Martin A. Fischler and Robert C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, June 1981. doi: 10.1145/358669.358692.

Daniel Foreman-Mackey, David W. Hogg, Dustin Lang, and Jonathan Goodman. emcee: The MCMC Hammer. *PASP*, 125(925):306–312, March 2013. doi: 10.1086/670067.

Gaia Collaboration, A. Vallenari, A. G. A. Brown, T. Prusti, J. H. J. de Bruijne, F. Arenou, C. Babusiaux, C. A. L. Bailer-Jones, M. Biermann, A. Brandeker, and et al. Gaia Data Release 3. Summary of the content and survey properties. *Astronomy and Astrophysics*, 674:A1, June 2023. doi: 10.1051/0004-6361/202243940.

A. Ginsburg, B. M. Sipőcz, C. E. Brasseur, P. S. Cowperthwaite, M. W. Craig, C. Deil, J. Guillochon, G. Guzman, S. Liedtke, P. Lian Lim, K. E. Lockhart, M. Mommert, B. M. Morris, H. Norman, M. Parikh, M. V. Persson, T. P. Robitaille, J.-C. Segoville, L. P. Singer, E. J. Tollerud, M. de Val-Borro, I. Valtchanov, J. Woillez, and Astroquery Collaboration. astroquery: An Astronomical Web-querying Package in Python. *AJ*, 157(3):98, March 2019. doi: 10.3847/1538-3881/aafc33.

Manuel Guizar-Sicairos, Samuel T. Thurman, and James R. Fienup. Efficient subpixel image registration algorithms. *Opt. Lett.*, 33(2):156–158, February 2008. doi: 10.1364/OL.33.000156.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, and others. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2.

A. A. Henden, M. Templeton, D. Terrell, T. C. Smith, S. Levine, and D. Welch. AAVSO Photometric All Sky Survey (APASS) DR9. VizieR Online Data Catalog, II/336, 2016.

Steve B. Howell. Two-dimensional aperture photometry: Signal-to-noise ratio of point-source observations and optimal data-extraction techniques. *PASP*, 101:616–622, June 1989. doi: 10.1086/132457.

Lei Hu, Lifan Wang, Xiaofeng Chen, and Jie Yang. Image Subtraction in Fourier Space. *ApJ*, 936 (2):157, September 2022. doi: 10.3847/1538-4357/ac7394.

J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science and Engineering*, 9 (3):90–95, 2007. doi: 10.1109/MCSE.2007.55.

Sebastian Jester, Donald P. Schneider, Gordon T. Richards, Paul J. Green, Maarten Schmidt, Patrick B. Hall, Michael A. Strauss, Daniel E. Vanden Berk, Chris Stoughton, James E. Gunn, and others. The Sloan Digital Sky Survey View of the Palomar-Green Bright Quasar Survey. *AJ*, 130(3):873–895, September 2005. doi: 10.1086/432466.

Richard G. Kron. Photometry of a complete sample of faint galaxies. *ApJS*, 43:305–325, June 1980. doi: 10.1086/190669.

Dustin Lang, David W. Hogg, Keir Mierle, Michael Blanton, and Sam Roweis. Astrometry.net: Blind astrometric calibration of arbitrary astronomical images. *AJ*, 139(5):1782–1800, June 2010. doi: 10.1088/0004-6256/139/5/1782.

Legacy Surveys Collaboration. DESI Legacy Imaging Surveys Data Release 10: photometric system description. https://www.legacysurvey.org/dr10/description/, 2023.

N. R. Lomb. Least-squares frequency analysis of unequally spaced data. *Ap&SS*, 39:447–462, February 1976.

R. H. Lupton. SDSS photometric equations: transformations between SDSS and Johnson-Cousins systems. https://www.sdss4.org/dr14/algorithms/sdssUBVRITransform/, 2005.

Curtis McCully and others. astroscrappy: Speedy Cosmic Ray Annihilation Pack in Python. ASCL ascl:1807.031, 2018.

Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

A. F. J. Moffat. A Theoretical Investigation of Focal Stellar Images in the Photographic Emulsion and Methods of Reconstructing the Stellar Intensity Distribution. *Astronomy and Astrophysics*, 3:455, December 1969.

F. Ochsenbein, P. Bauer, and J. Marcout. The VizieR database of astronomical catalogues. *Astronomy and Astrophysics Supplement Series*, 143:23–32, April 2000. doi: 10.1051/aas:2000169.

M. Paillassa, E. Bertin, and H. Bouy. MAXIMASK and MAXITRACK: Two new tools for identifying contaminants in astronomical images using convolutional neural networks. *Astronomy and Astrophysics*, 634:A48, February 2020. doi: 10.1051/0004-6361/201936345.

Chien Y. Peng, Luis C. Ho, Chris D. Impey, and Hans-Walter Rix. Detailed Structural Decomposition of Galaxy Images. *AJ*, 124(1):266–293, July 2002. doi: 10.1086/340952.

Thomas Robitaille, Erik Tollerud, Marco Viana, Christoph Deil, K. A. Bostroem, P. Greenfield, V. Zabalza, A. Conley, M. Bachetti, B. J. Baptista, G. Brammer, S. Brigitta, D. J. Burke, D. Cara, M. Droettboom, P. Erlanson, H. M. G"unther, M. A. Gurwell, K. Inglis, S. Laine, P. Llerena, N. MacDonald, Z. Meadows, T. E. Pickering, A. Pontzen, A. Ptak, B. Refsdal, A. Scholz, M. Slavich, M. Tanga, E. J. Tollerud, and H. Yorke. reproject: astronomical image reprojection in Python. *JOSS*, 5(47):1464, March 2020. doi: 10.21105/joss.01464.

C. Rodrigo and E. Solano. The SVO Filter Profile Service. *ASPC*, 461:461, 2012. doi: 10.48550/arXiv.1201.3114.

Thomas V. Russell, Guy R. Rixon, Daniel M. Bramich, Benjamin Holl, Lukasz Wyrzykowski, Niko Ihanec, Francesca De Angeli, Simon T. Hodgkin, Floor van Leeuwen, Roger Burgon, Georgie Blow, and Richard G. McMahon. SFFT: The Sparse Field Flux Transport algorithm for image subtraction and transient detection. *MNRAS*, 515(4):5010–5024, November 2022. doi: 10.1093/mnras/stac2009.

J. D. Scargle. Studies in astronomical time series analysis. II. Statistical aspects of spectral analysis of unevenly spaced data. *ApJ*, 263:835–853, December 1982.

D. L. Shupe, M. Moshir, J. Li, D. Makovoz, R. Narron, and R. N. Hook. The SIP Convention for Representing Distortion in FITS Image Headers. In *Astronomical Data Analysis Software and Systems XIV, ASP Conf. Ser.*, volume 347, page 491, 2005.

M. F. Skrutskie, R. M. Cutri, R. Stiening, M. D. Weinberg, S. Schneider, J. M. Carpenter, C. Beichman, R. Capps, T. Chester, J. Elias, and others. The Two Micron All Sky Survey (2MASS). *AJ*, 131(2):1163–1183, February 2006. doi: 10.1086/498708.

spalipy. spalipy: detection-based astronomical image registration with quad matching and spline-warp resampling. https://pypi.org/project/spalipy/, n.d.

Peter B. Stetson. DAOPHOT: A Computer Program for Crowded-Field Stellar Photometry. *PASP*, 99:191–222, March 1987. doi: 10.1086/131977.

Peter B. Stetson. On the Growth-Curve Method for Calibrating Stellar Photometry with CCDs. *PASP*, 102:932–948, August 1990. doi: 10.1086/132731.

STScI. tweakwcs: image alignment with adjustable WCS. https://tweakwcs.readthedocs.io/, n.d.

J. L. Tonry, C. W. Stubbs, K. R. Lykke, P. Doherty, I. S. Shivvers, W. S. Burgett, K. C. Chambers, K. W. Hodapp, N. Kaiser, R.-P. Kudritzki, and others. The Pan-STARRS1 Photometric System. *ApJ*, 750(2):99, May 2012. doi: 10.1088/0004-637X/750/2/99.

J. L. Tonry, L. Denneau, H. Flewelling, A. N. Heinze, C. A. Onken, S. J. Smartt, B. Stalder, H. J. Weiland, and C. Wolf. The ATLAS All-Sky Stellar Reference Catalog. *ApJ*, 867(2):105, November 2018. doi: 10.3847/1538-4357/aae386.

V. Toptun and et al. DECaLS and MzLS+BASS to SDSS photometric transformations, 2024. piecewise relations in $(g - r)$ and $(r - z)$, as implemented in `autophot.calibration.filter_transformations`.

Transient Name Server. Transient Name Server. https://www.wis-tns.org/, n.d.

Pieter G. van Dokkum. Cosmic-Ray Rejection by Laplacian Edge Detection. *PASP*, 113(789): 1420–1427, November 2001. doi: 10.1086/323894.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, and others. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, March 2020. doi: 10.1038/s41592-019-0686-2.

Miguel Weiler, Francesca De Angeli, Paolo Montegriffo, Carine Babusiaux, Ivan De Loon, Paola Sartoretti, Jose Manuel Carrasco, Mike B. Davies, Floor van Leeuwen, and Marco Riello. GaiaXPy: A Python package for Gaia spectrophotometry. *MNRAS*, 521(2):2941–2949, May 2023. doi: 10.1093/mnras/stad678.

M. Wenger, F. Ochsenbein, D. Egret, P. Dubois, F. Bonnarel, S. Borde, F. Genova, G. Jasniewicz, S. Laloë, S. Lesteven, and others. The SIMBAD astronomical database. The CDS reference database for astronomical objects. *Astronomy and Astrophysics Supplement Series*, 143:9–22, April 2000. doi: 10.1051/aas:2000332.

Christian Wolf, Christopher A. Onken, Lance C. Luvaul, Brian P. Schmidt, Michael S. Bessell, Seo-Won Chang, Gary S. da Costa, Dougal Mackey, Tony Martin-Jones, Simon J. Murphy, and others. SkyMapper Southern Survey: First Data Release (DR1). *PASA*, 35:e010, February 2018. doi: 10.1017/pasa.2018.5.

Donald G. York, J. Adelman, Jr. Anderson, John E., Scott F. Anderson, James Annis, Neta A. Bahcall, J. A. Bakken, Robert Barkhouser, Steven Bastian, Eileen Berman, and others. The Sloan Digital Sky Survey: Technical Summary. *AJ*, 120(3):1579–1587, September 2000. doi: 10.1086/301513.

Barak Zackay, Eran O. Ofek, and Avishay Gal-Yam. Proper image subtraction–optimal transient detection, photometry and hypothesis testing. *ApJ*, 830(1):27, June 2016. doi: 10.3847/0004-637X/830/1/27.